Crash Course Intro

The levels in Powerslave are very similar to BUILD engine games, where a level is composed of sectors consisting of planes that make up the walls, ceilings and floors. Like BUILD, only one ceiling and floor can exist per sector and walls are always fully vertical. Some walls may be flagged to act as a portal in order to connect to other sectors and like with BUILD, portals are used for traversing the scene and culling out objects that are not visible. However, unlike BUILD, a wall may be linked to multiple sectors, allowing unlimited 3D geometry.

Anatomy of a Sector

A sector is composed of planes, which also makes up faces. Unlike BUILD, sectors MUST be convex, in other words, the walls making up the sector must be curved like the exterior of a circle or sphere, or in other words, the interior angles of all the walls measuring less than 180 degrees.
Collision and how sprites are rendered relies on the convexity of sectors. If a sector is NOT convex, then it will flash red in the 2D viewer of the editor. However it will still allow the user to keep on editing the sector. In 3D mode, the ceilings/floors may not render correctly if the sector is not convex.

Anatomy of a Face

A face is a plane composed of 4 vertices that serves as collision. The face can have some attributes such as block AI, act as a force field, or serve as a portal. Faces may or may not contain tiles.

Anatomy of a Tile

A tile is a textured quad composed of 4 vertices. The amount of tiles depends on the length of the face, and how often the texture repeats. For example, a wall with a 64x64 texture that's 512 units in width and 256 units in height will require 8 tile columns and 2 tile rows. However there may be multiple tiles with different textures present per face. Tiles can also have attributes for manipulating the orientation of the texture, such as flipping, panning and rotating.

NOTE: Faces can only store up to 255 tiles. Be sure your walls and sectors are broken up to minimize the number of tiles, otherwise they will not load correctly in the game.

Setting up the editor

Launch the game normally and bring down the console with the “**~**” key. Next type **developer 1** and press enter. This will enable developer mode which will allow the engine to load loose files not present in the KPF archive. Next exit the game and relaunch for the changes to take effect. To start the editor, bring down the console with the “**~**” key and type **editor** and then press enter.

Controls

General

Keypad Enter	Toggles between 2D and 3D mode
0 - 1	Sets the grid snap size
Shift + 0-4	Sets the grid snap size

2D Mode

Left Mouse	Selects / Drag / Pans
Right Mouse	Context menu
Wheel Up/Down	Zooms in/out
Space	Pans canvas
Shift + Space	Enters sector drawing mode
Ctrl + Space	CSG Subtract (Walls Only)
u	CSG Subtract
delete	Delete selected item
h	hide selected item
Shift + h	unhide items
c	Clear selections
d	Duplicate selected items
i	Invert selections

insert	Inserts a point on a wall and splits it
end	Cancels draw mode

3D Mode

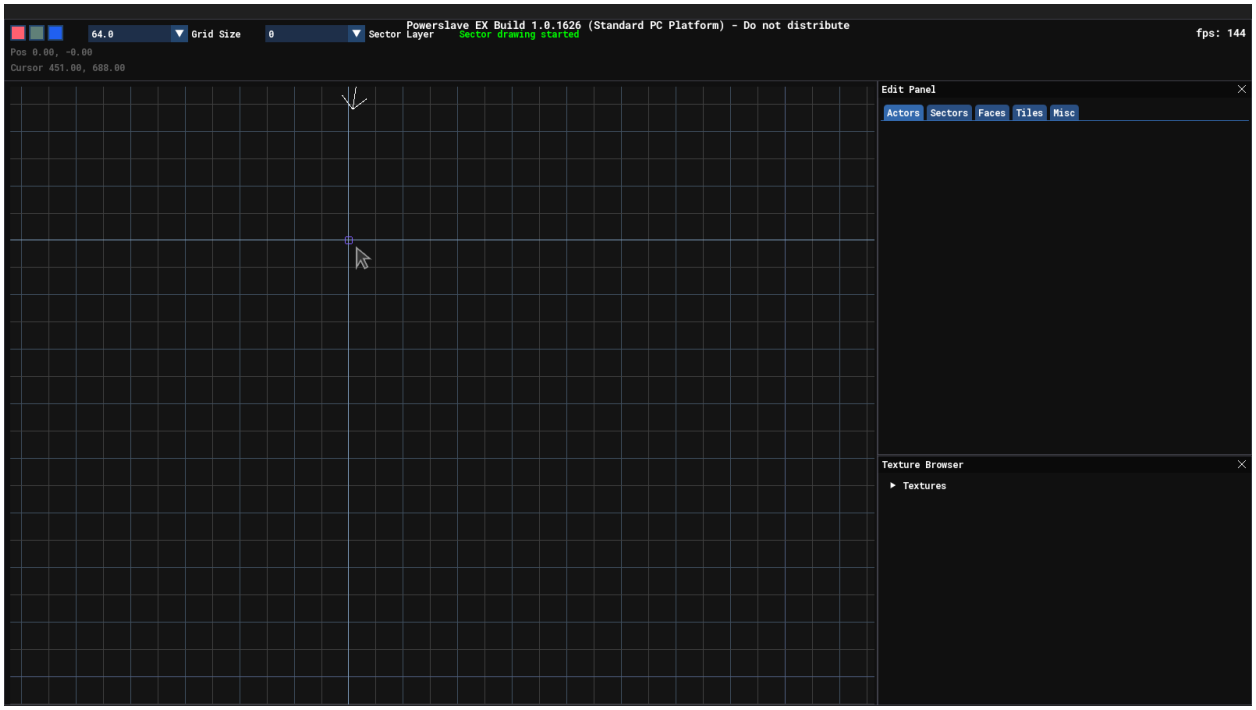
f	Enter camera active mode (also enables many of the functionality while in 3D mode)
Left Mouse	Selects faces / actors
Shift + Left Mouse	Selects tiles
Ctrl + Left Mouse	Places marker at cursor and snaps to grid
Right Mouse	Context menu
Mouse wheel	Moves selected ceiling/floor up/down
Shift + Mouse wheel	Rotates selected ceiling/floor up/down
Ctrl + Mouse wheel	Rotates selected ceiling/floor up/down
WASD keys	Move camera around
c	Clears selection
l	Toggle lock on selected face(s) (Explained later)
t	Sets target (Explained later)
r	Rebuild face geometry
shift + r	Delete and fully reconstruct face geometry and reorder in sector's list of faces
z	Copy tile properties of selected face
shift + z	Copy slope properties of selected ceiling/floor
x	Paste tile properties to selected face
shift + x	Paste slope properties to selected ceiling/floor
h	hide selected item
shift + h	unhide items

u	CSG Subtract
Backspace	Clear all tiles from selected face(s)
p	Rebuild portal (forces the editor to recompute and link sectors to selected portal face)
shift + home	Reset slope properties of ceiling
shift + end	Reset slope properties of floor
home	Align ceiling slope to marker
end	Align floor slope to marker
Keypad Multiply	Set rotation pivot of ceiling/floor to marker position
ctrl + left/right	Nudge marker on x axis
ctrl + up/down	Nudge marker on y axis
ctrl + shift + up/down	Nudge marker on z axis
insert	Split selected face by marker position
e	Snap selected ceiling/floor to marker position
delete	Delete selected item

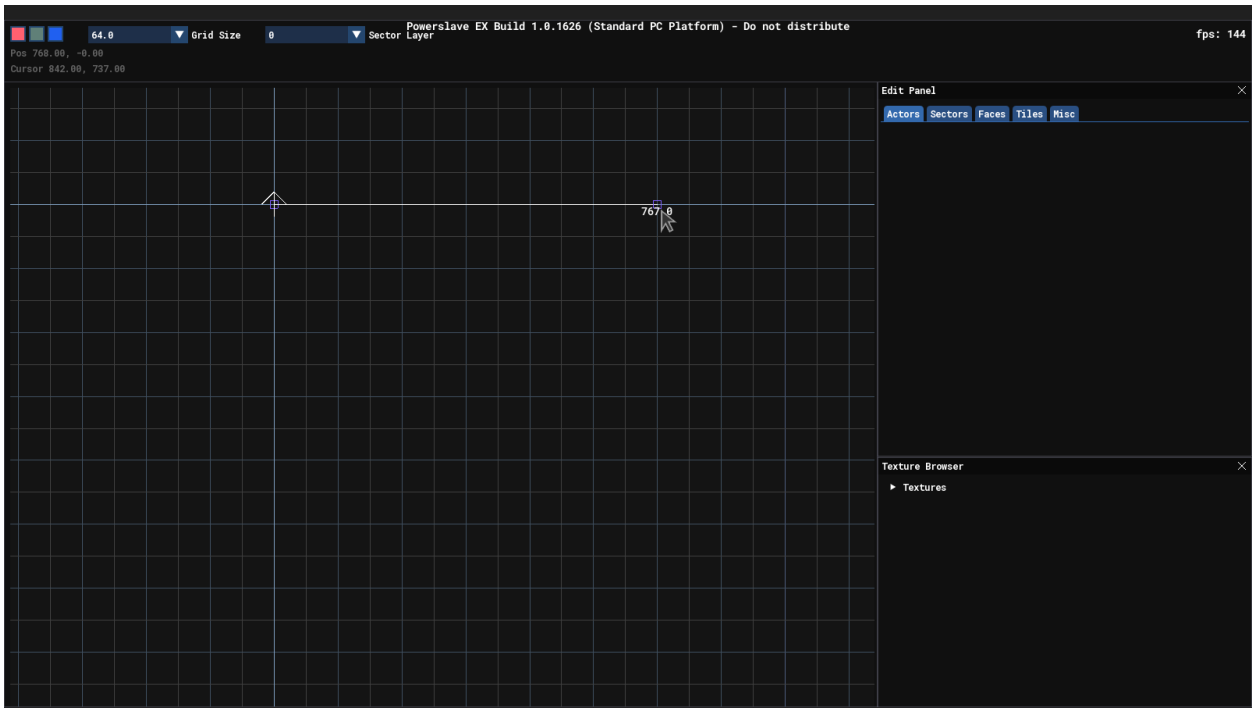
Making your first sector/room

While in 2D mode, press **shift + space** to enter sector drawing mode. You should see an outline of a vertex on your cursor. Pressing the **left mouse** will insert a vertex. You can keep inserting vertices to form your sector, but clicking on the first vertex inserted will complete the sector drawing process and your sector will be created.

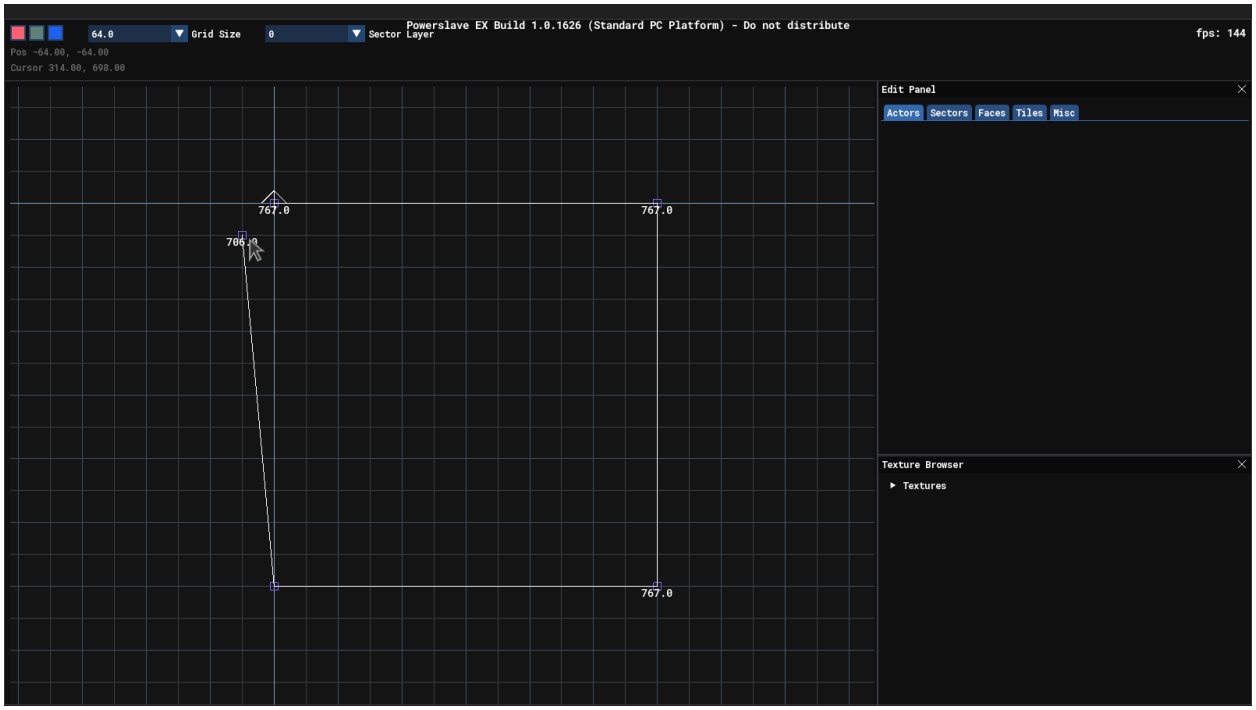
For simplicity, create a simple square room as shown in the screenshot. Start off by pressing **shift + space** and then position your cursor at coordinate 0, 0.

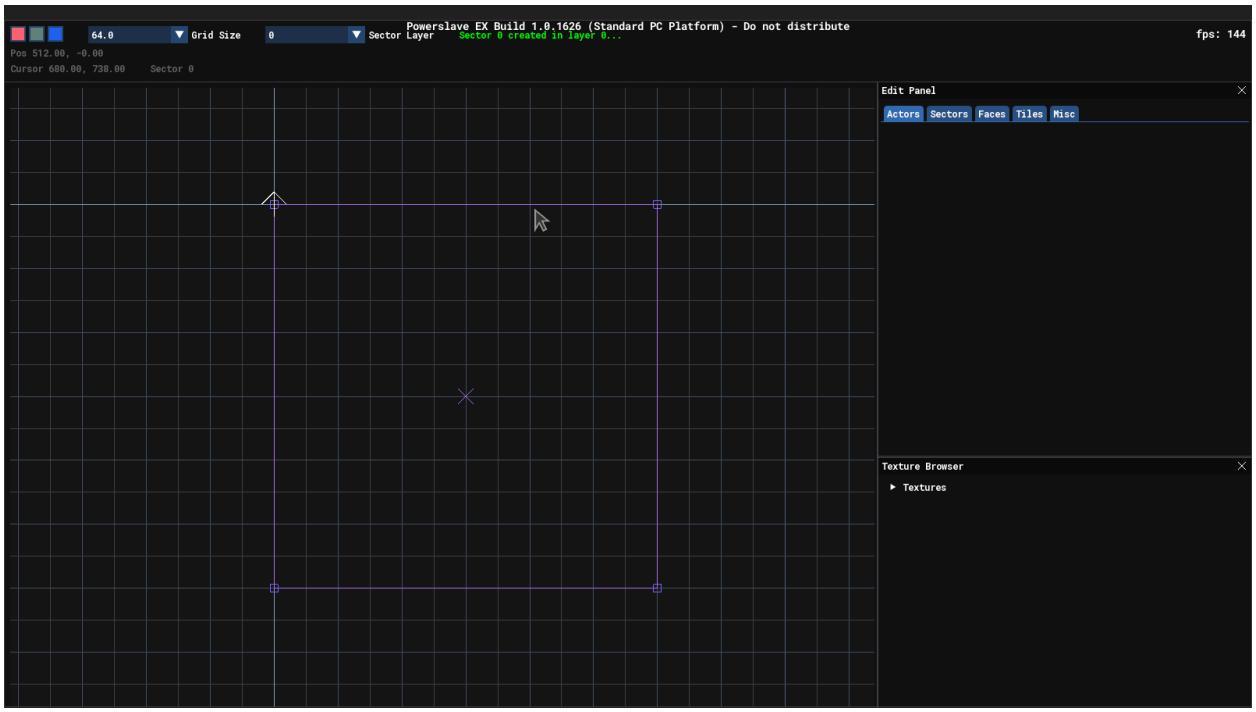


Next press the **left mouse button** to insert a vertex, then move your cursor to coordinate 768, 0 and insert another vertex there (see screenshot).



Repeat the same so you end up with a 4-sided square room with the walls all being ~768 units in length. **Left-clicking** on the first vertex inserted will complete the sector drawing process and the sector will be created.

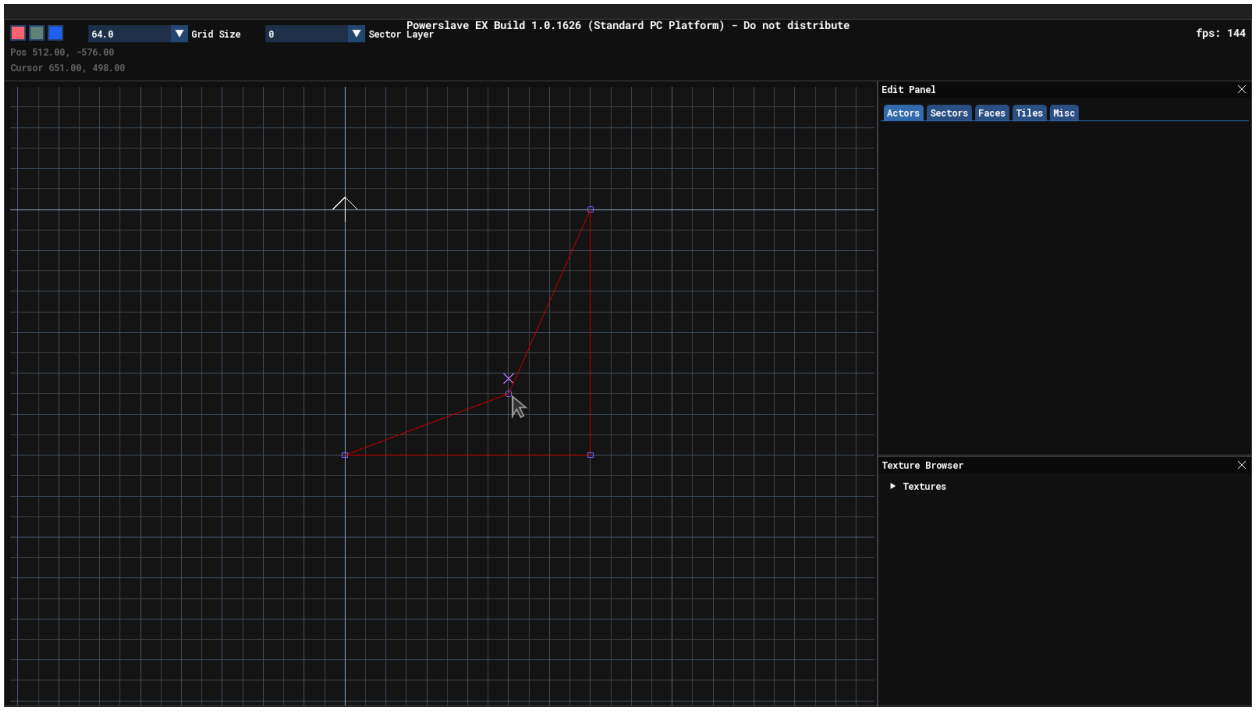




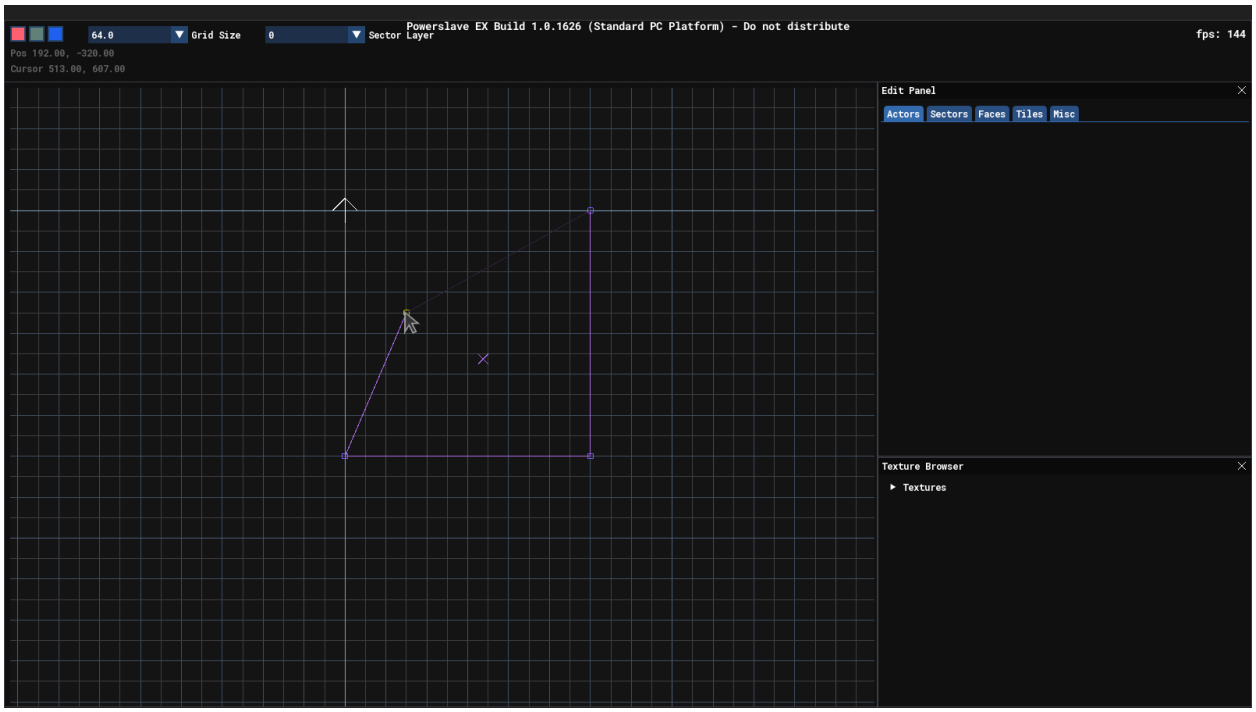
You can cancel the drawing process by pressing **END** on your keyboard. You can select the sector by clicking on the center of the sector. You can also select the individual vertices and drag them around. Pressing '**C**' will clear selections.

All sectors in Powerslave MUST be convex shaped. Otherwise the sector will flash red, indicating that you must correct it.

Non-Convex:

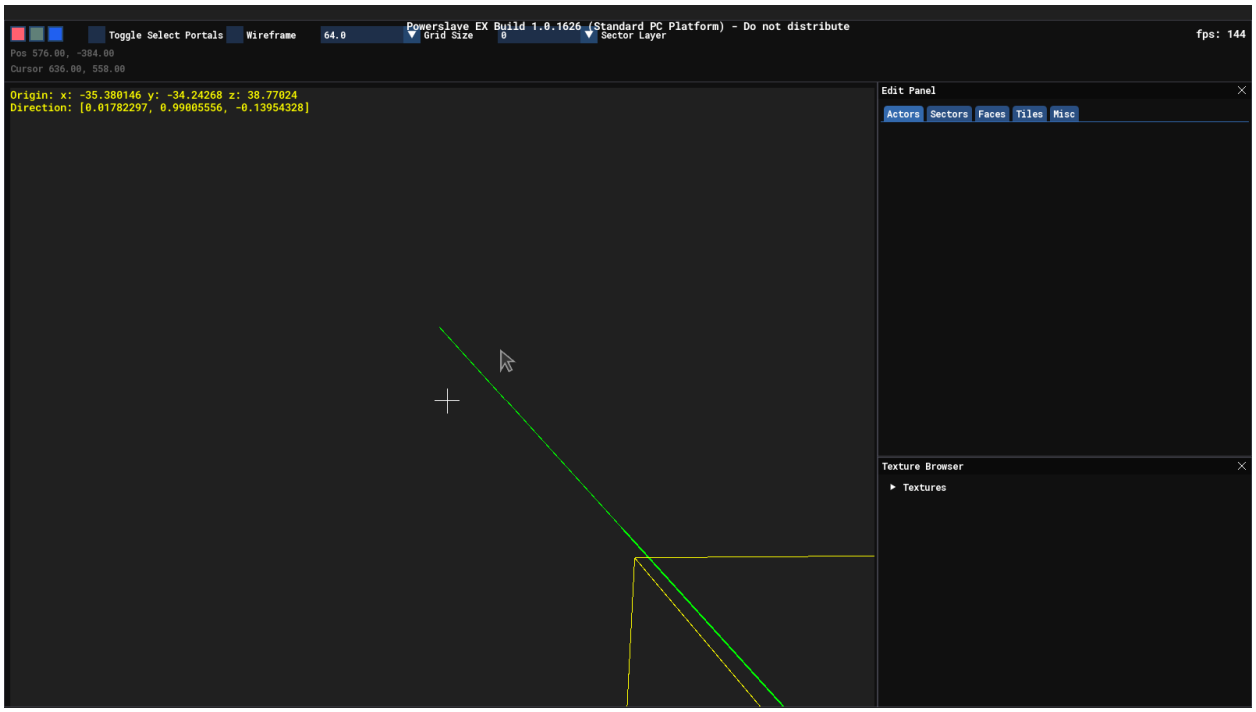


Convex:



Note that an average-sized room is about 512 units in length so ideally you want to make a room at that length or larger as the player's size
is about 192 in length.

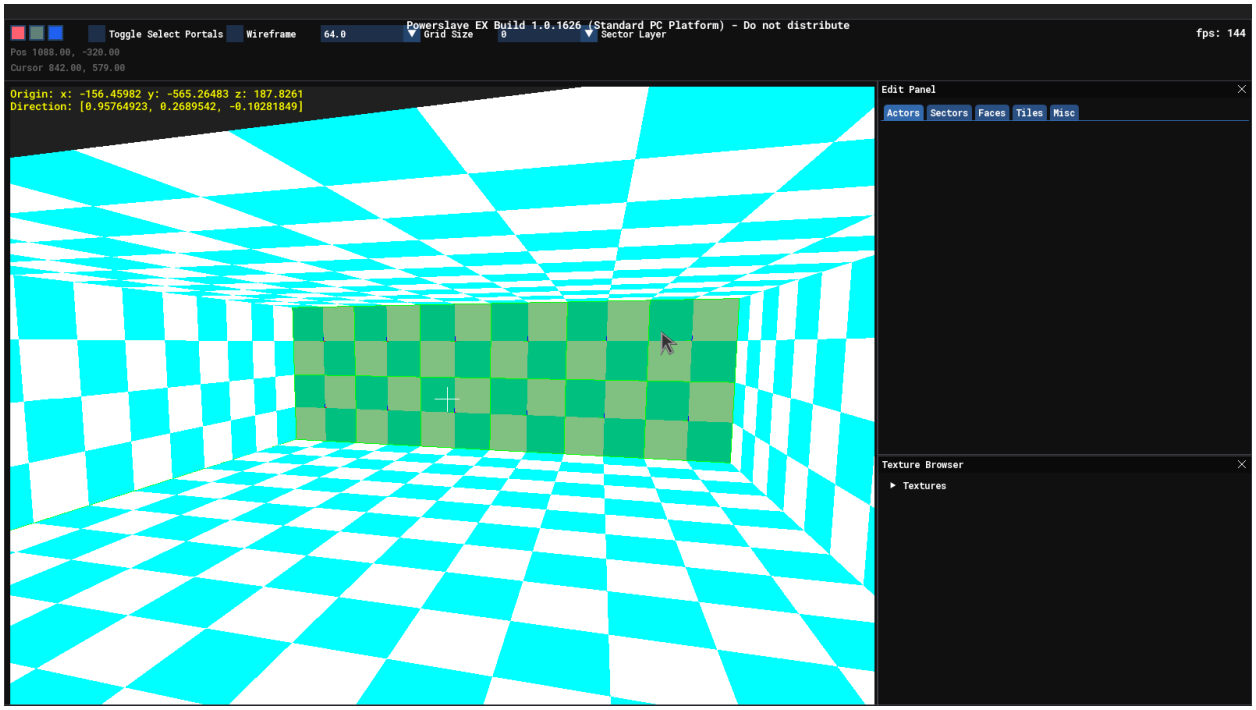
Now that you made your first sector, press **keypad-enter** to enter 3D mode. You should see something like this (see screenshot).



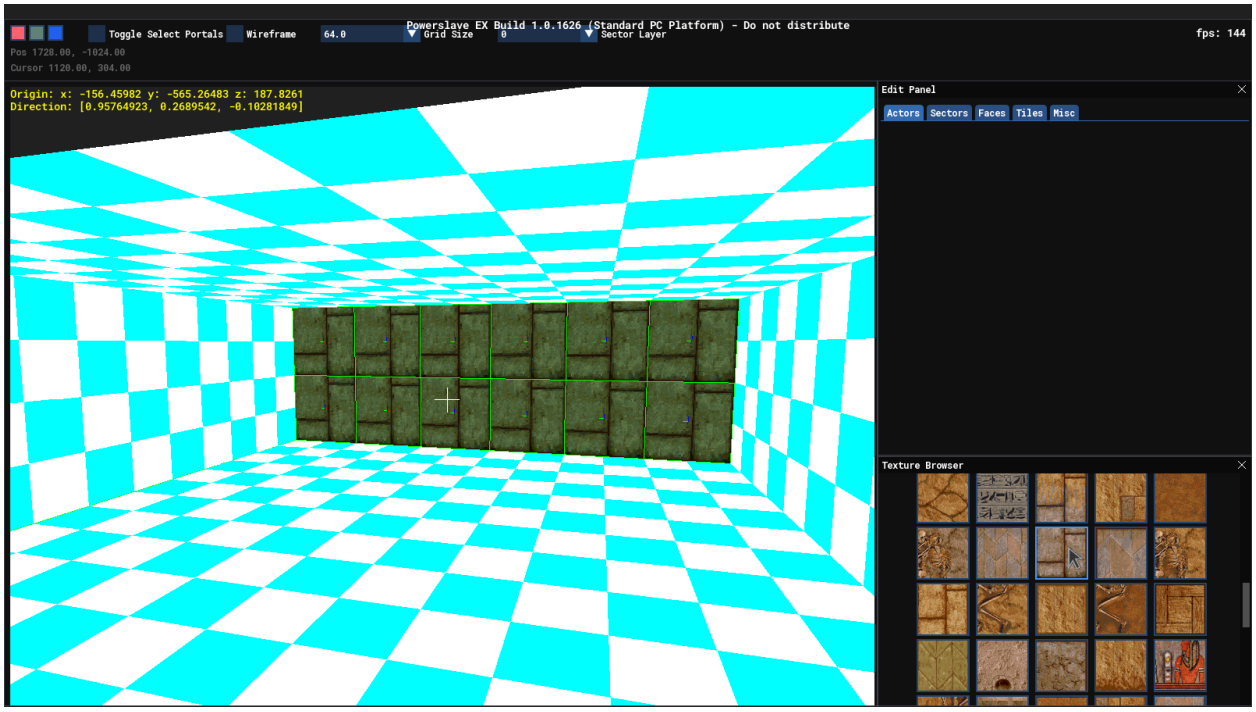
Press the **F** key to enter camera active mode. Press '**D**' to back up until you see your room. Use the mouse to turn, and use **'WASD'** keys to move around. Pressing the **F** key again turns off camera active mode. You can only select things while camera active mode is on, but can only insert actors when camera active mode is turned off.

Texturing your room

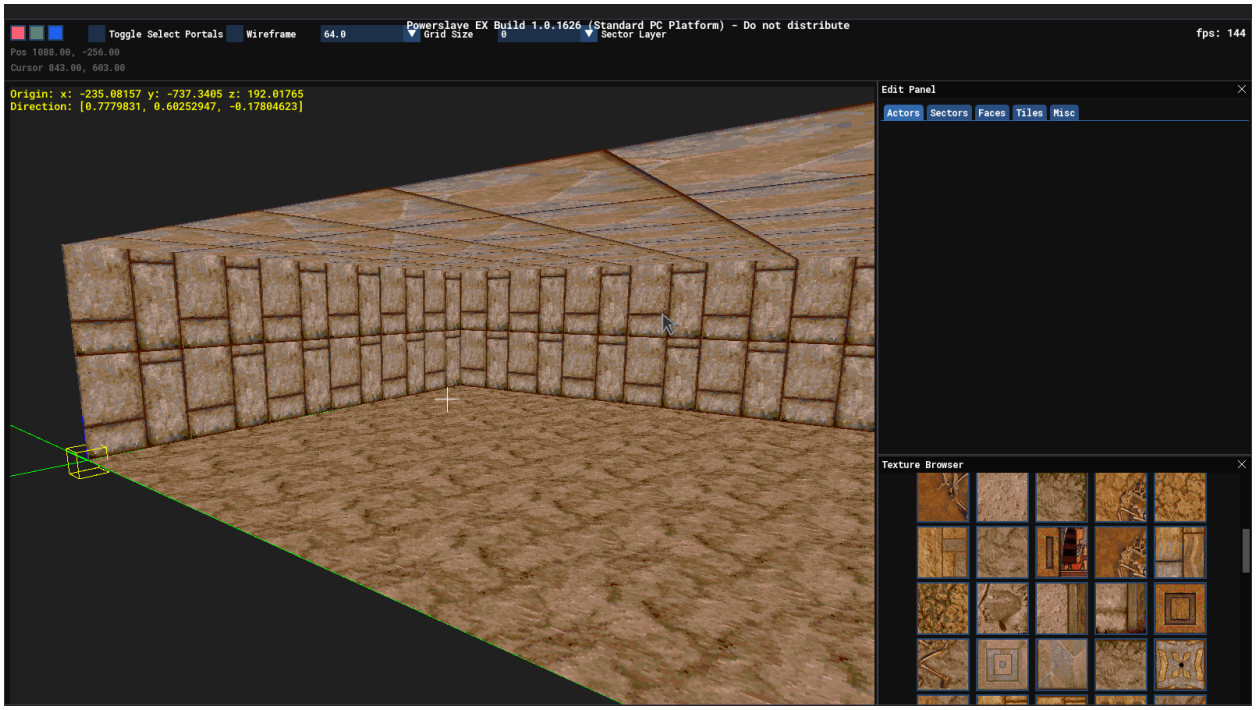
While in 3D mode and in camera active mode, press **shift + left mouse** on a face. It should highlight green as shown.



While still selected, press the **F** key to disable camera active mode and move your cursor over into the **Texture Browser** panel. Expand **'Textures'** and then choose **'Karnak'**. Selecting this will expand and list all available textures in that folder. Feel free to choose whatever texture of your choice.

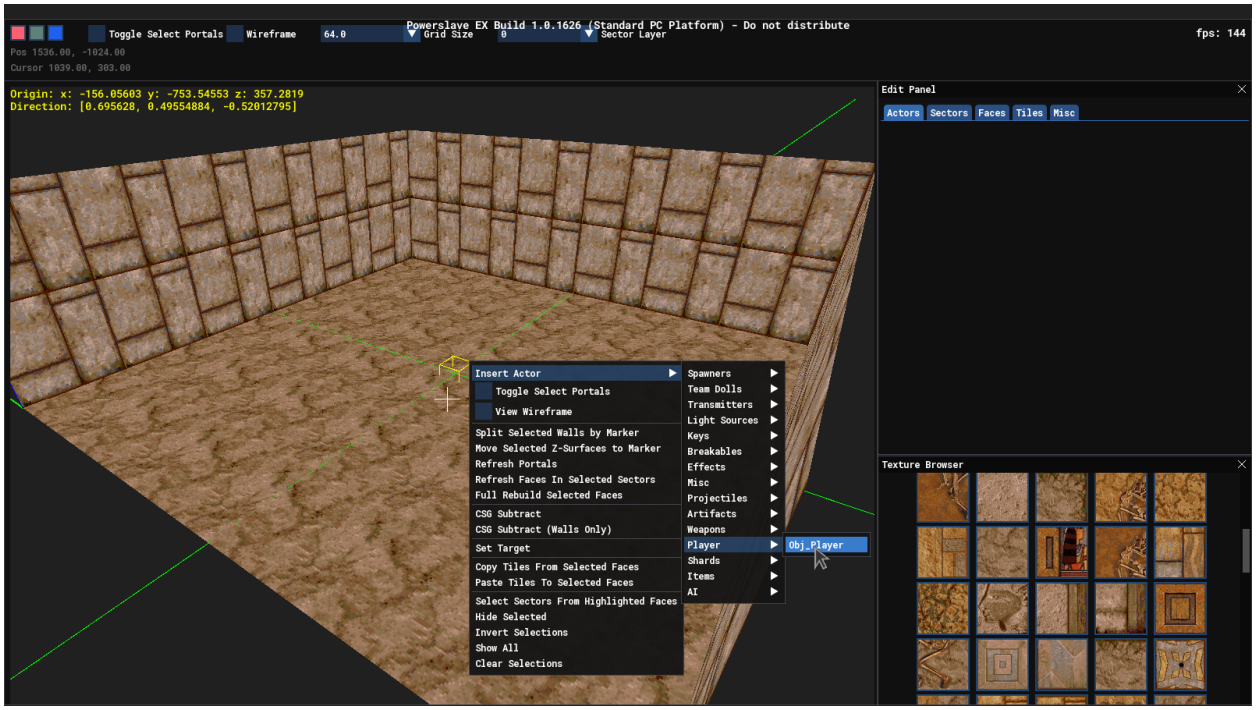


Repeat the steps for the other faces until the room is fully textured.

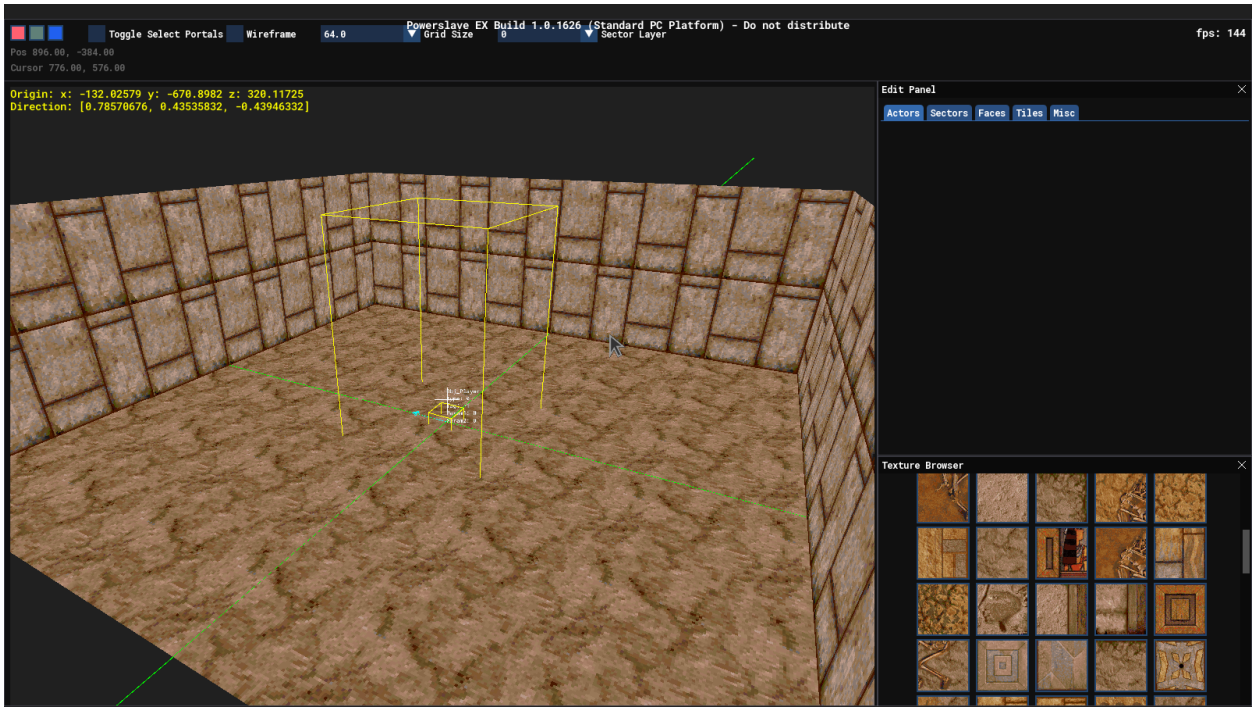


Adding the Player Object

While in 3D mode and in camera active mode, aim your camera at the center of the room and press **ctrl-left** mouse button to position the marker in the center of the room. Press the **F** key to exit out of camera active mode and press the right mouse button and select **Insert Actor->Player->Obj_Player**.

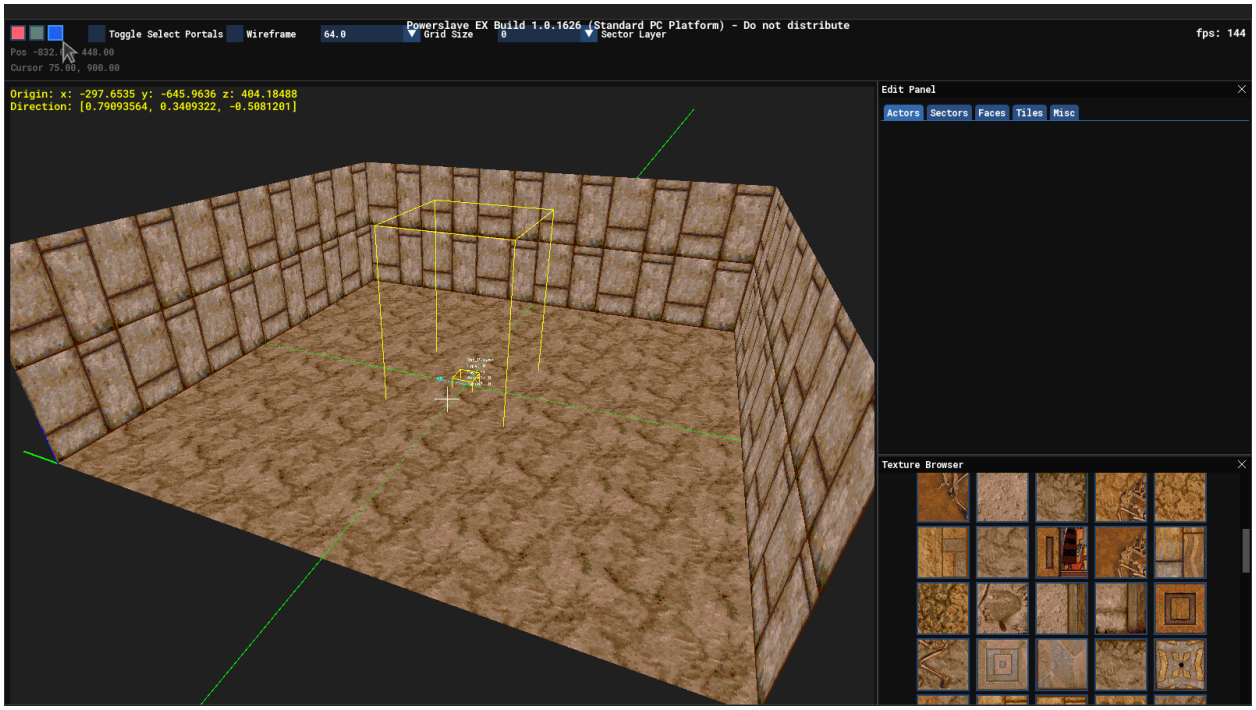


This will insert a player object in the center of the room.



Saving your Level

At the top-left corner of your screen, select the **blue-colored button** (see screenshot).

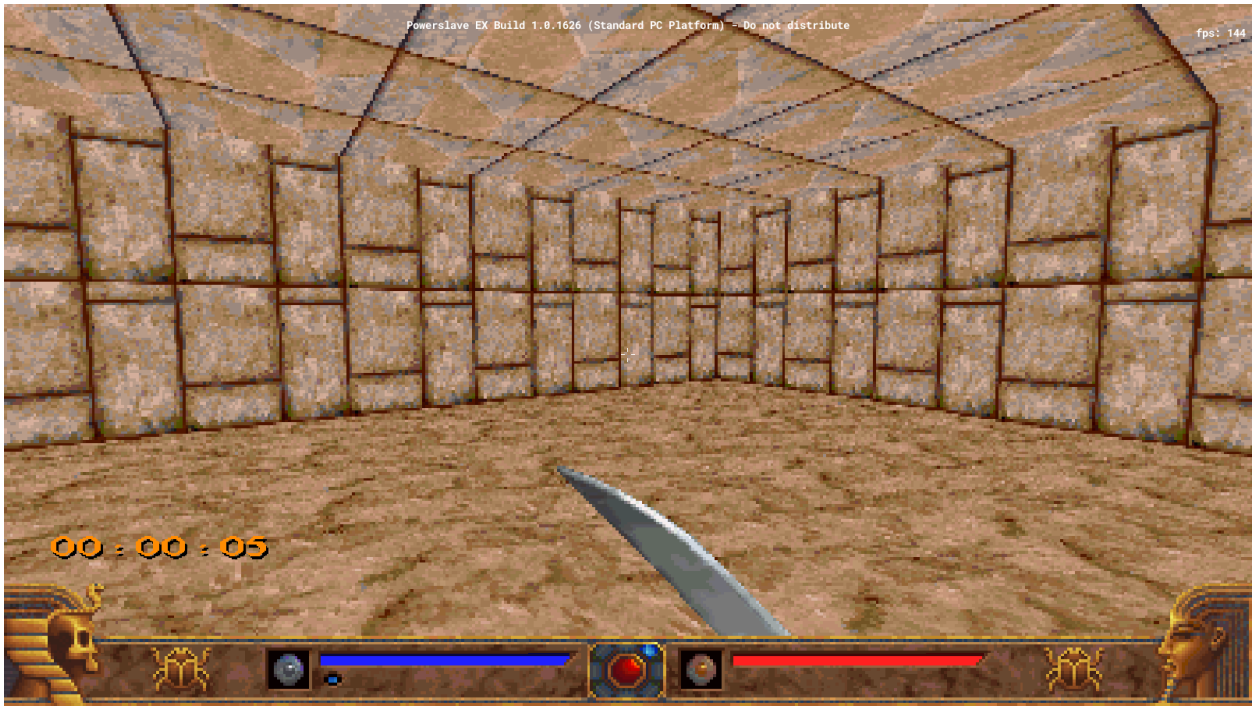


This opens up the save-as dialog window. Ideally, you want to save your map relative to where the game's executable is located (ie: **C:\Program Files (x86)\Steam\steamapps\common\PowerSlave Exhumed**). Save the map as 'my_level' and select 'ok'. Doing this will create a .DMAP file and a .MAP file. The DMAP file is the 'uncompiled' version of your level which can be read in a text editing program like notepad. The .MAP file is the compiled binary map file understood by the game engine.

Testing your level

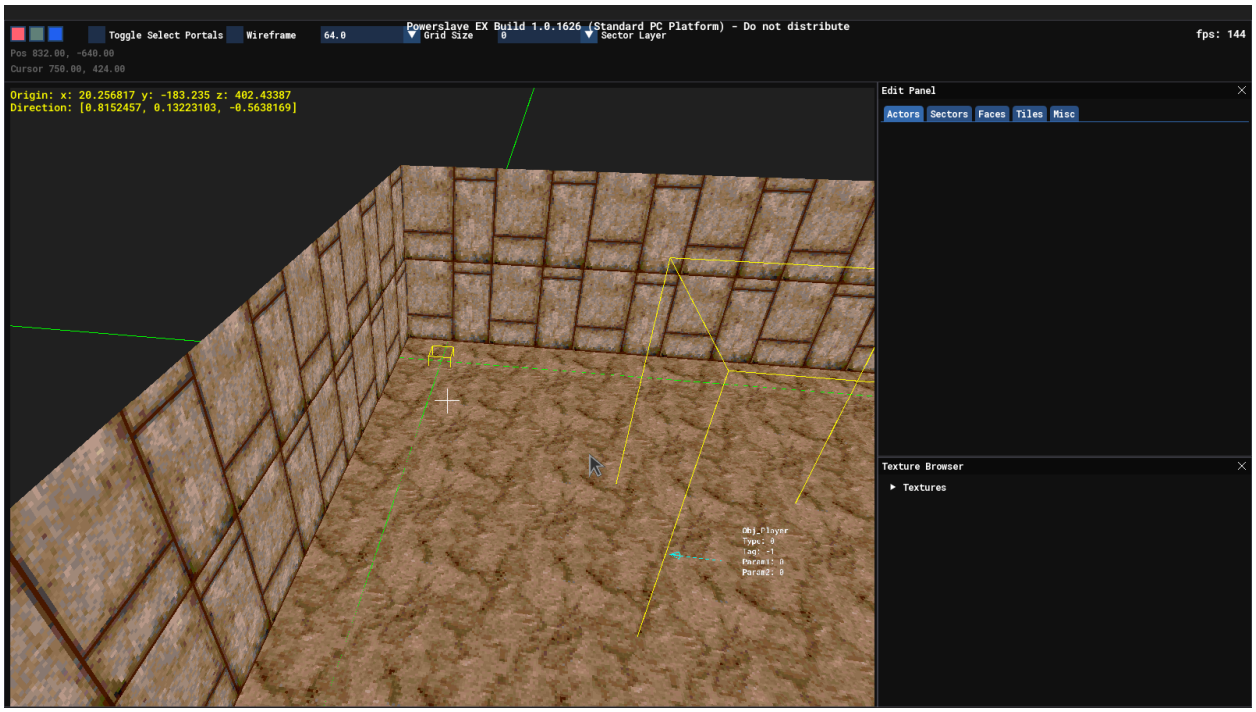
The game expects to locate maps relative to the executable, treating the executable path as the base location. So if you saved the map in the same folder as the executable, then bring down the console with the '**~**' key and type: "**map my_level.map**" (without quotes).

Congrats, you just made your first Powerslave level.

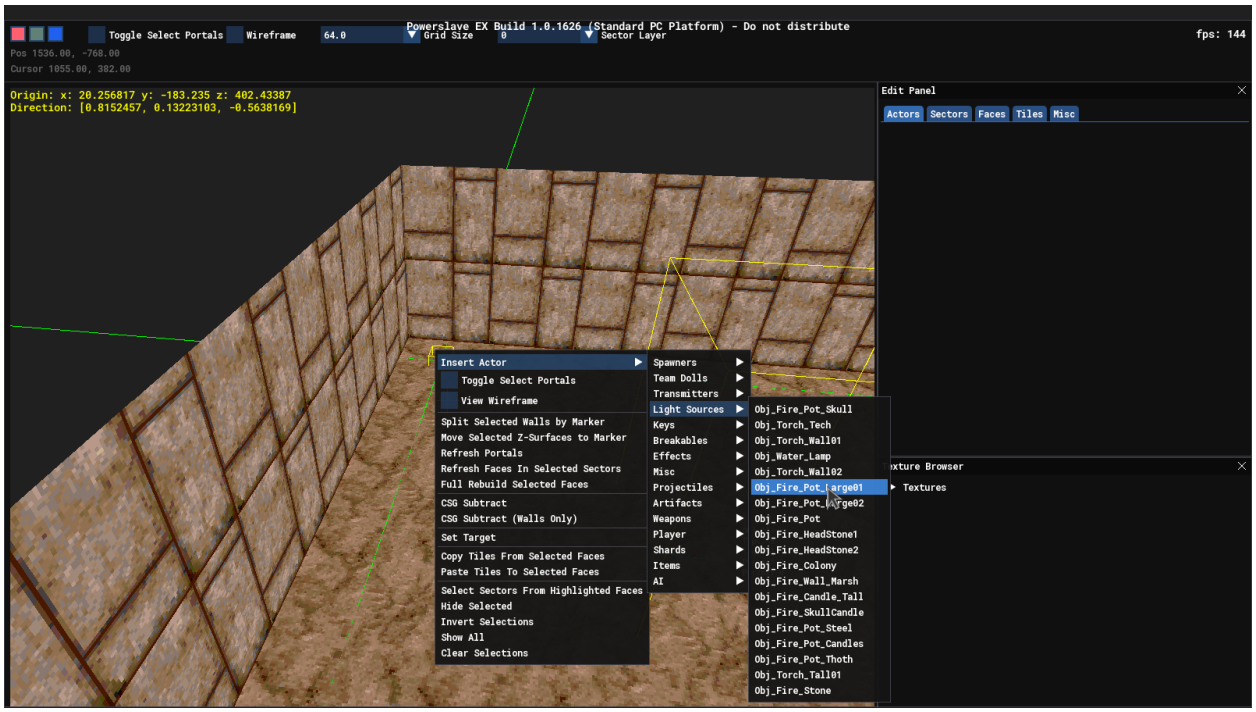


Lighting

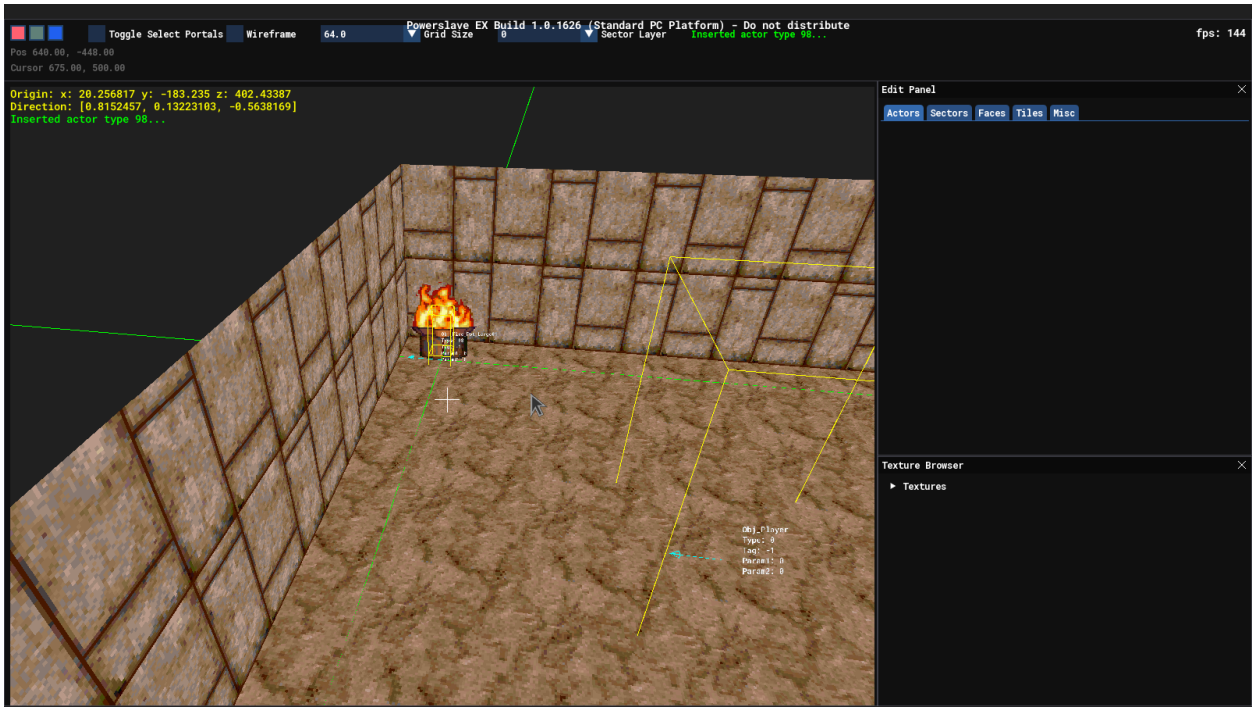
Place the marker at a corner of the room



Bring up the context menu by right clicking and select **Insert Actor->Light Sources->Obj_Fire_Pot_Large01**



Doing this will place a fire pot object at that location.



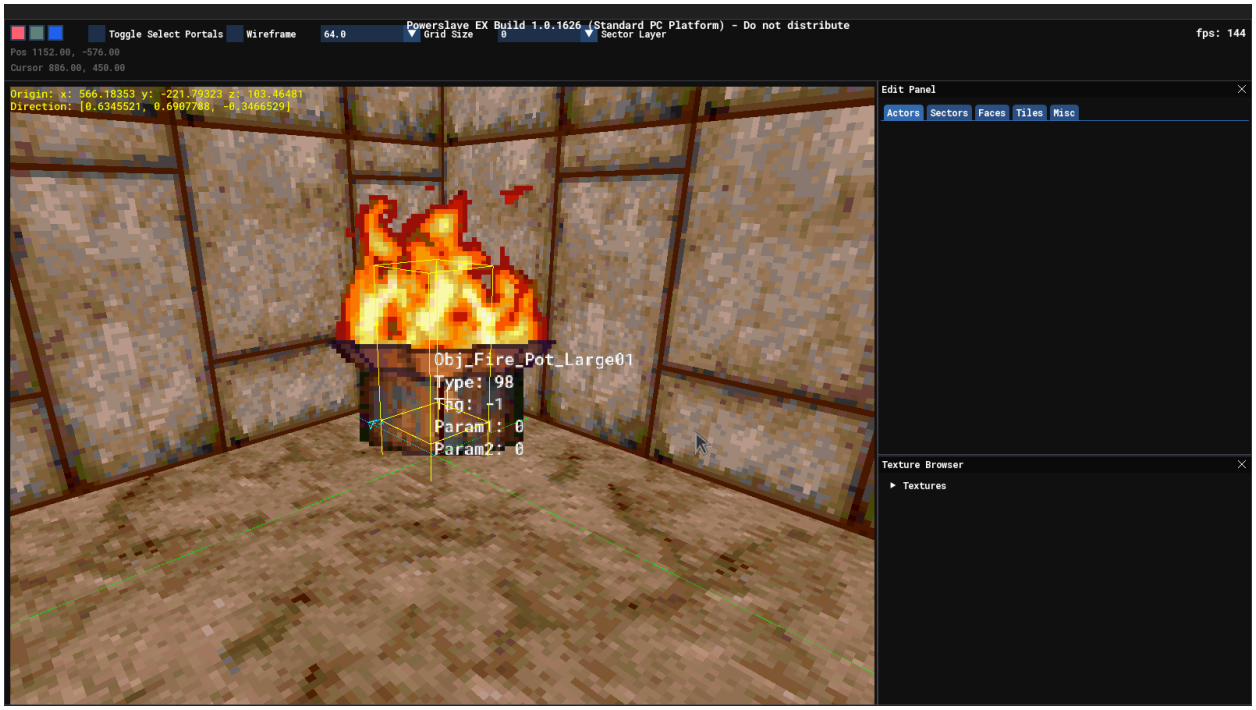
In order for lights to be compiled, you will need to create a **light script file** for the light builder to understand. Where you have my_level.dmap saved, create a file called **my_level_light.txt**. The name of this file must match your DMAP file and must have **_light** appended to the name.

Next open up this file in your text editor like Notepad. Next copy and paste the contents listed here into your file:

```
world
{
   sunlight_direction  { 0.95 0.4 1.0 }
   ambienceColor 32 32 32
   sunlight_ambientRange 2048.0
   sunlight_ambienceColor 32 32 32
   sunlight_color 248 248 248
   ambientOcclusion 0.7
}

object
{
   type 98
   Index -1
   radius 1024
   intensity 1.0
   falloff 2.0
   color 255 255 255
   zOffset 24.0
}
```

What this does is it tells the builder that all objects whose type is 98 should emit a 1024 radius light with a 2.0 falloff and 24 units off the ground. You can find out the object's type number by looking at the actor you just inserted, which will have some info displayed.



The fields inside the **world** block tells the light builder how sunlight affects the lighting. Any ceiling face of a sector that contains no tiles will emit sunlight. Below explains what each field does:

ambienceColor	Specifies the minimum color value to unlit areas
sunlight_ambientRange	Affects a gradient color value for faces that are in sunlight. This controls how far this gradient shade goes.
sunlight_ambienceColor	Specifies the gradient color value for faces that are in sunlight.
sunlight_color	Specifies the color of outdoor light and light emitted by sunlight.
ambientOcclusion	Specifies how much to darken creases and corners in geometry. Lower value results in darker shading.
sunlight_direction	Normalized directional vector that indicates the direction of the sun

The fields inside the **object** block tells the light builder how to apply light for this specific type of actor. Multiple object blocks can be applied to the same actor type.

type	What actor type this should affect
index	Unused, **must be left as -1** or the light won't be built at all.
radius	The size of the light. The **angle** of the actor will also control the scale of the radius. So an angle of 180 will emit the light at half the radius, where an angle of 359 will emit the light at near zero radius.
intensity	Sets intensity. 1.0 being the default, higher being brighter and < 1.0 being darker/weaker.
falloff	Sets a falloff for the light, where it fades out much more from the center.
color	Sets color for the light
zOffset	Sets a vertical offset from the ground.

Not listed in this tutorial is the **surface** block, which applies light to a specific texture

name	Texture path (in quotes) to affect (example: "textures/magma/magma412.png")
color	Color to apply
range	How large of a light this surface emits

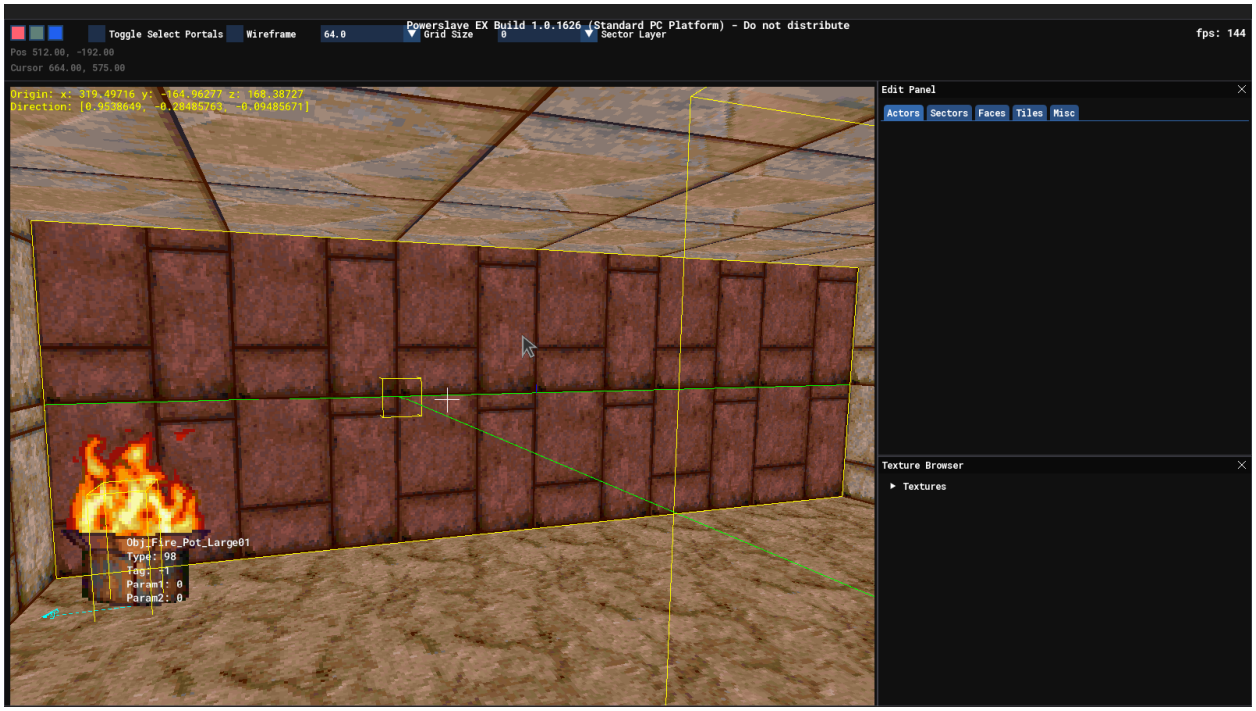
Next save the file and return back to the editor. Save your map by overwriting my_level.
After you saved your level, bring down the console with '**~**' and type: **buildlights my_level.map**

Then test your level by typing **map my_level.map** in the console and press enter.

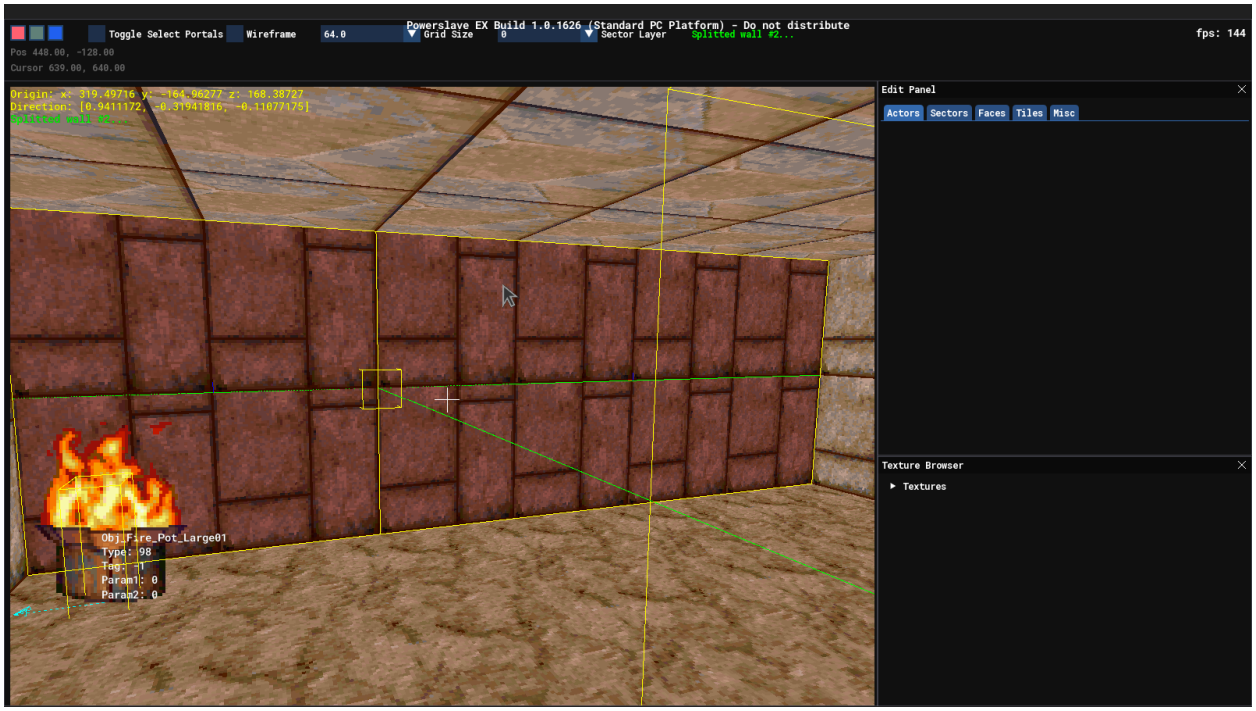
Congrats, you just built lighting in your level

Creating a corridor

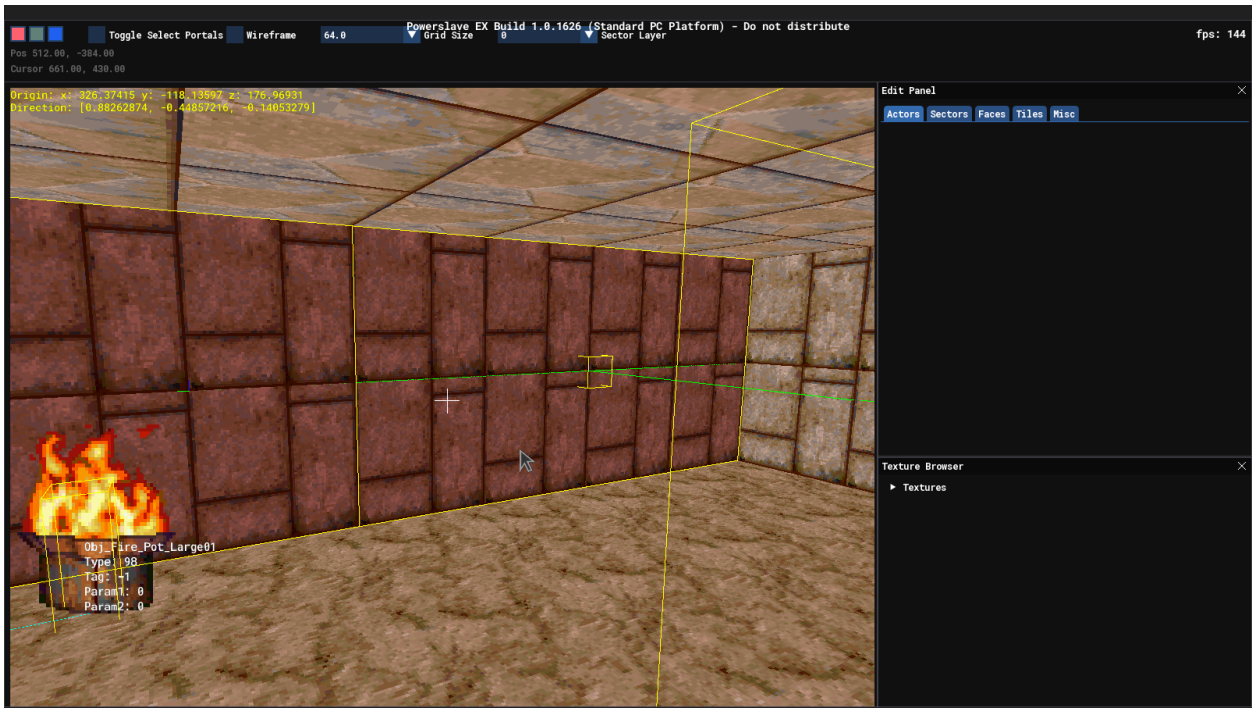
While in 3D mode with active camera mode enabled, select a face with **left-mouse button** and position the marker here:



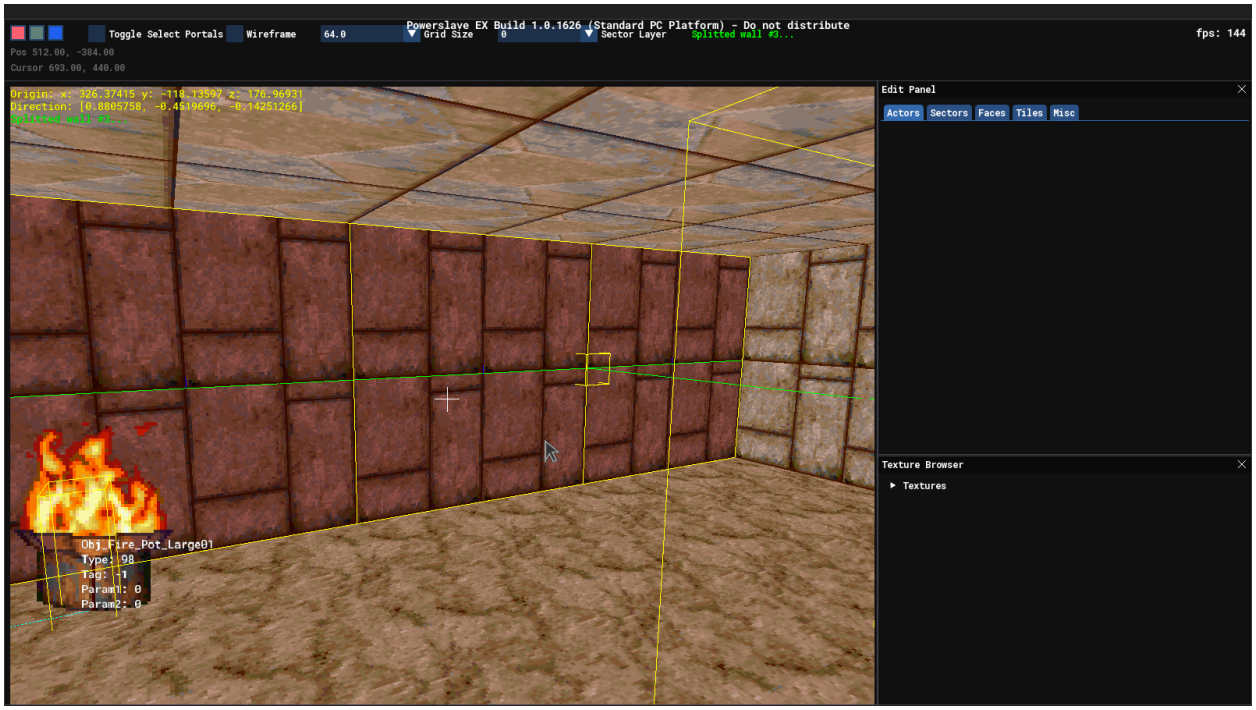
Press **insert** to split the face at the marker position, which should look like this:



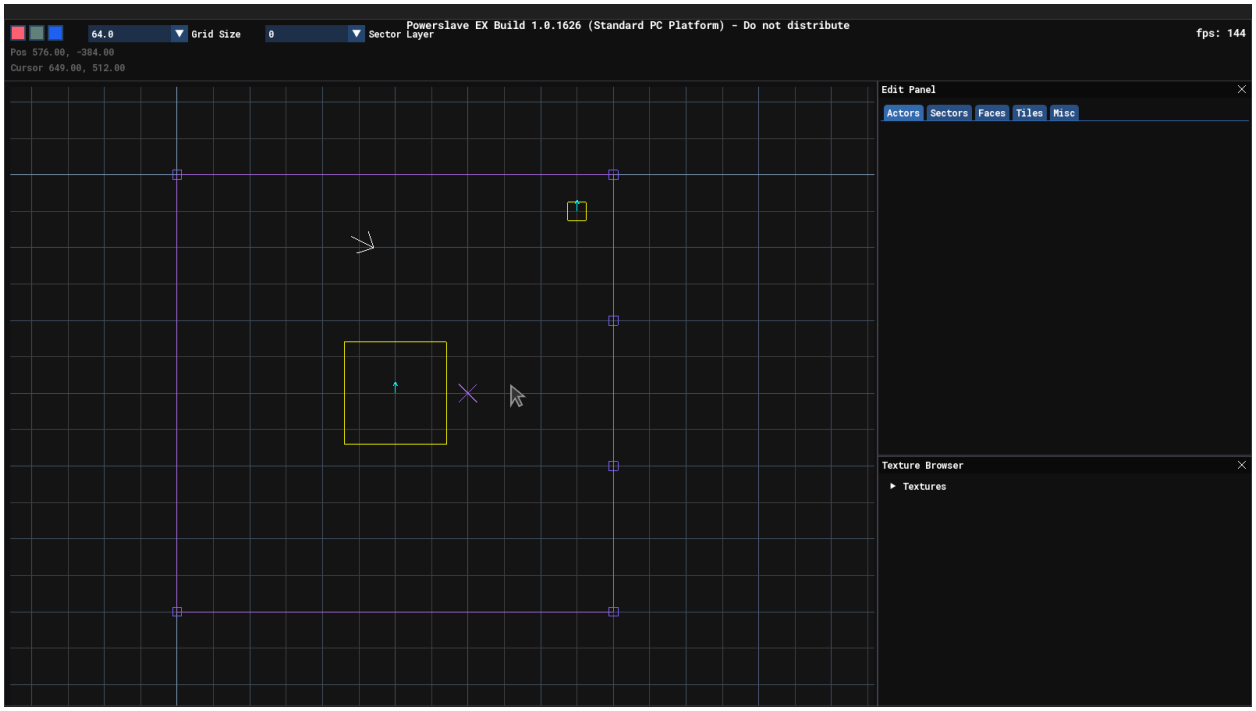
Next place the marker here:



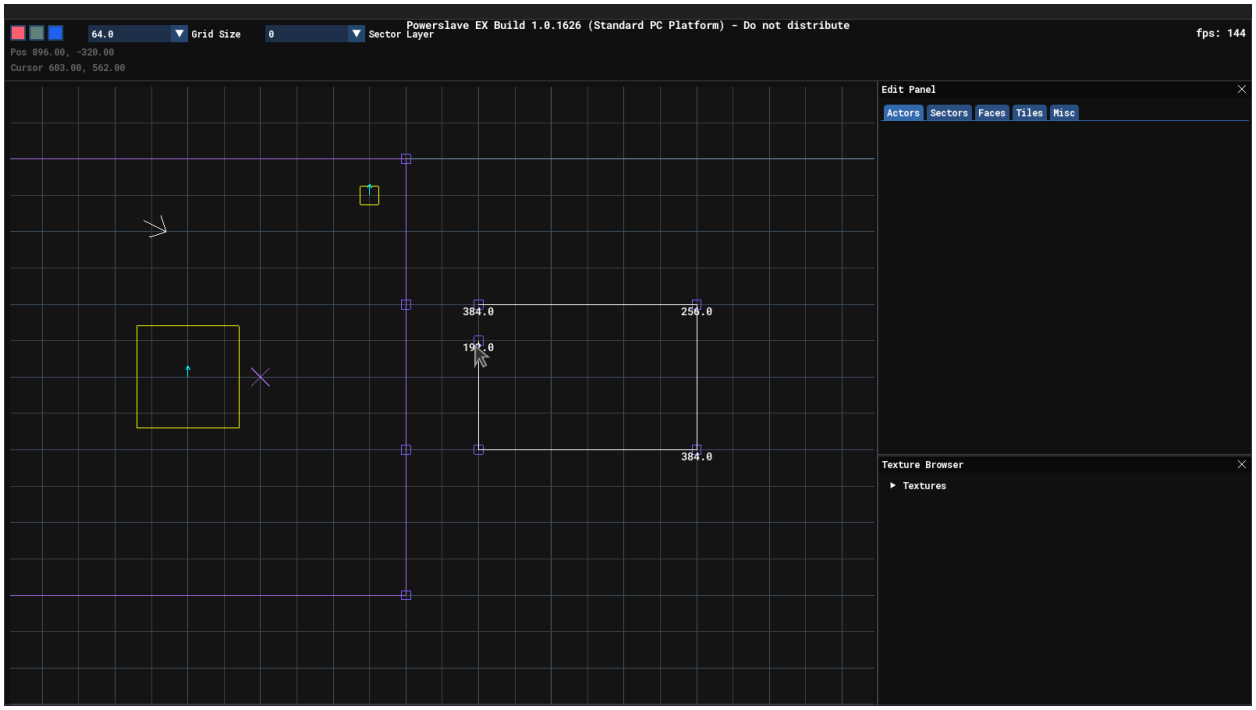
As before, press **insert** to split that face. Press **c** to clear selections. Note you can also split walls in 2D mode as well.

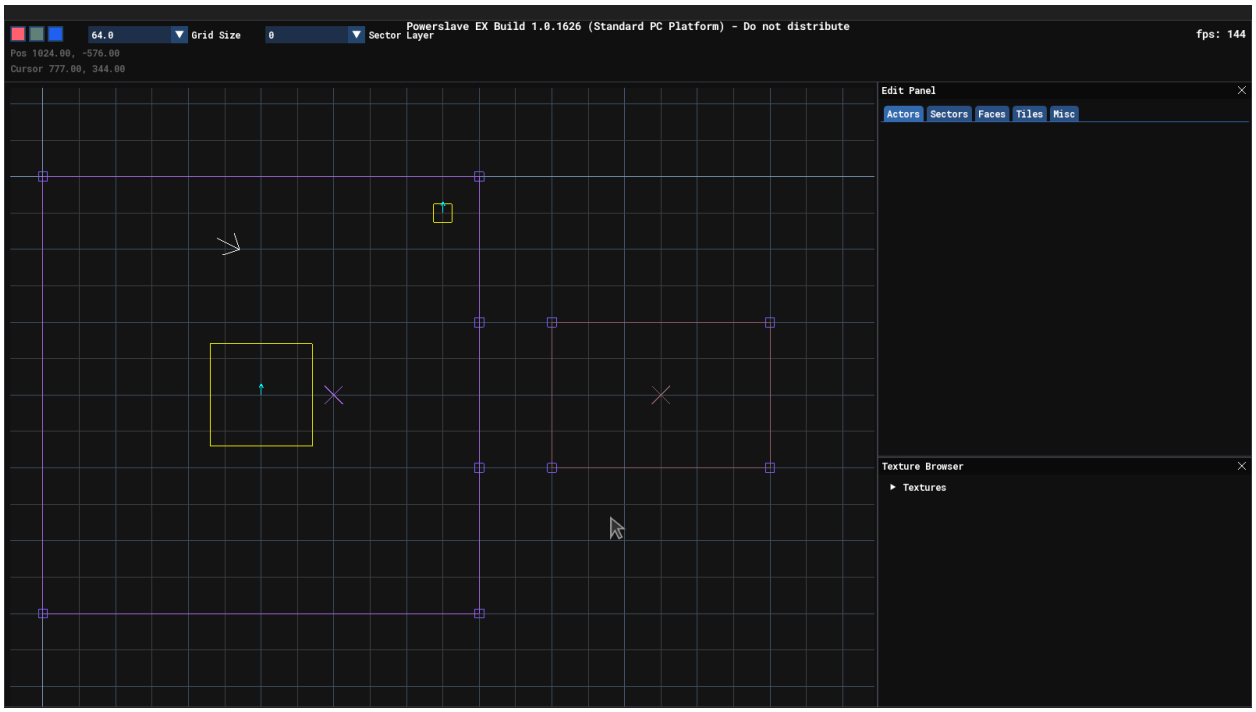


If done right, you should now have 3 walls

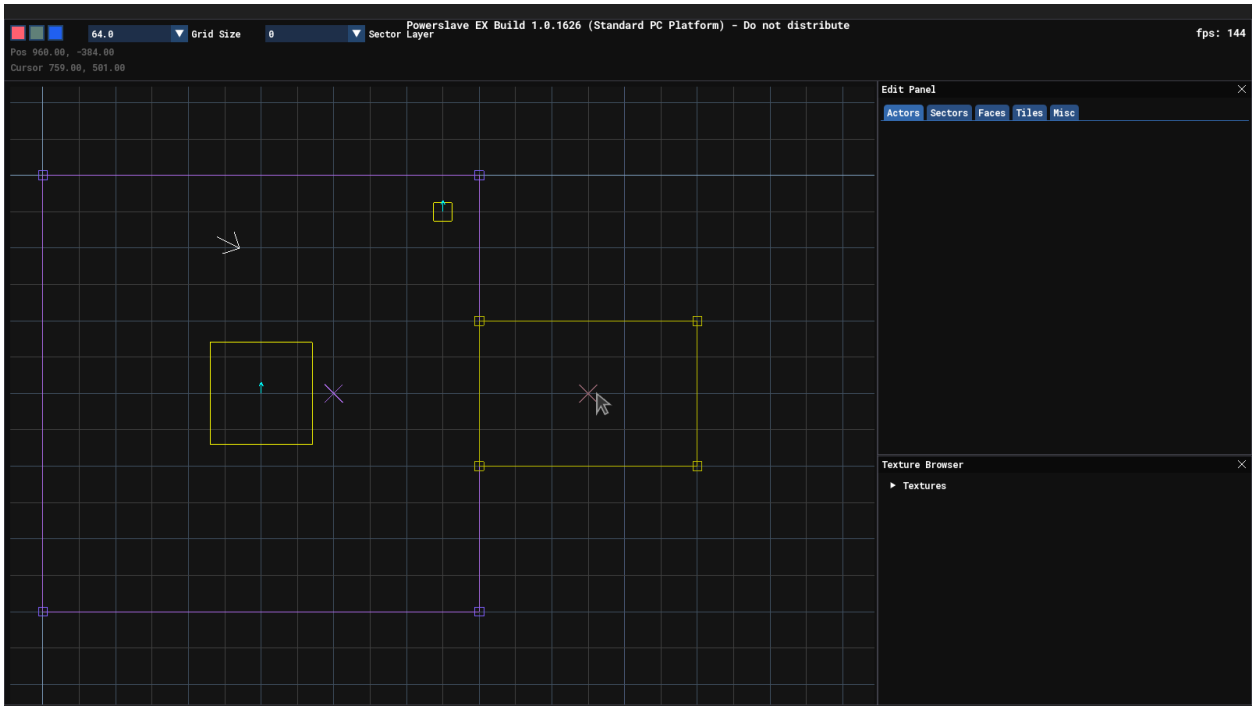


While in 2D mode, begin creating another sector as seen here



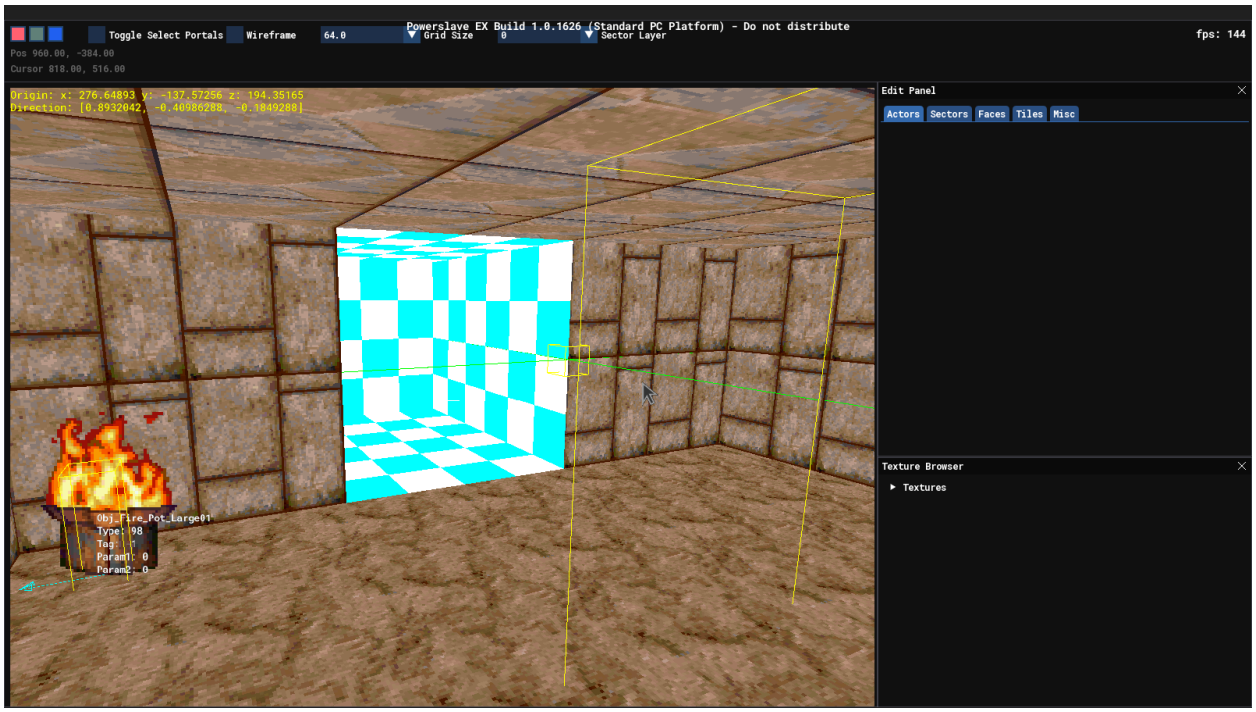


Next thing you want to do now is **select your new sector** and then drag it over until the far left side of the wall is lying on top of the other wall that was created when you performed the splits:

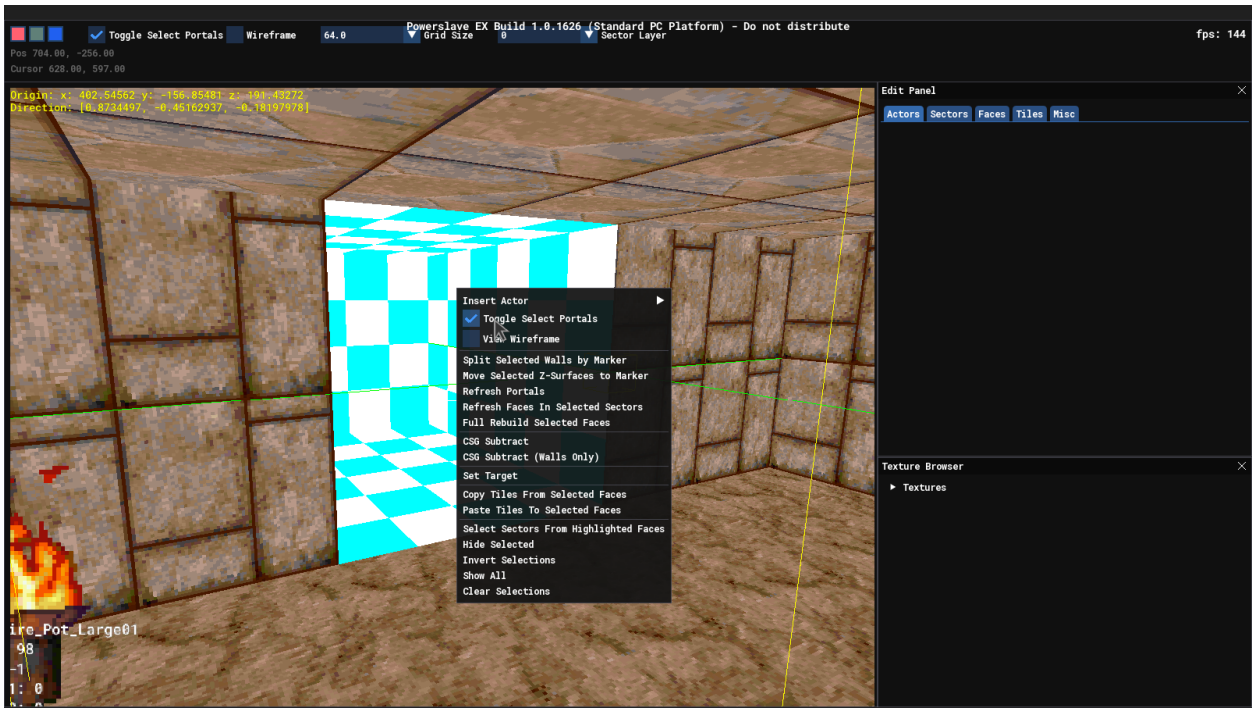


Release the left mouse button and press **C** to clear selections.

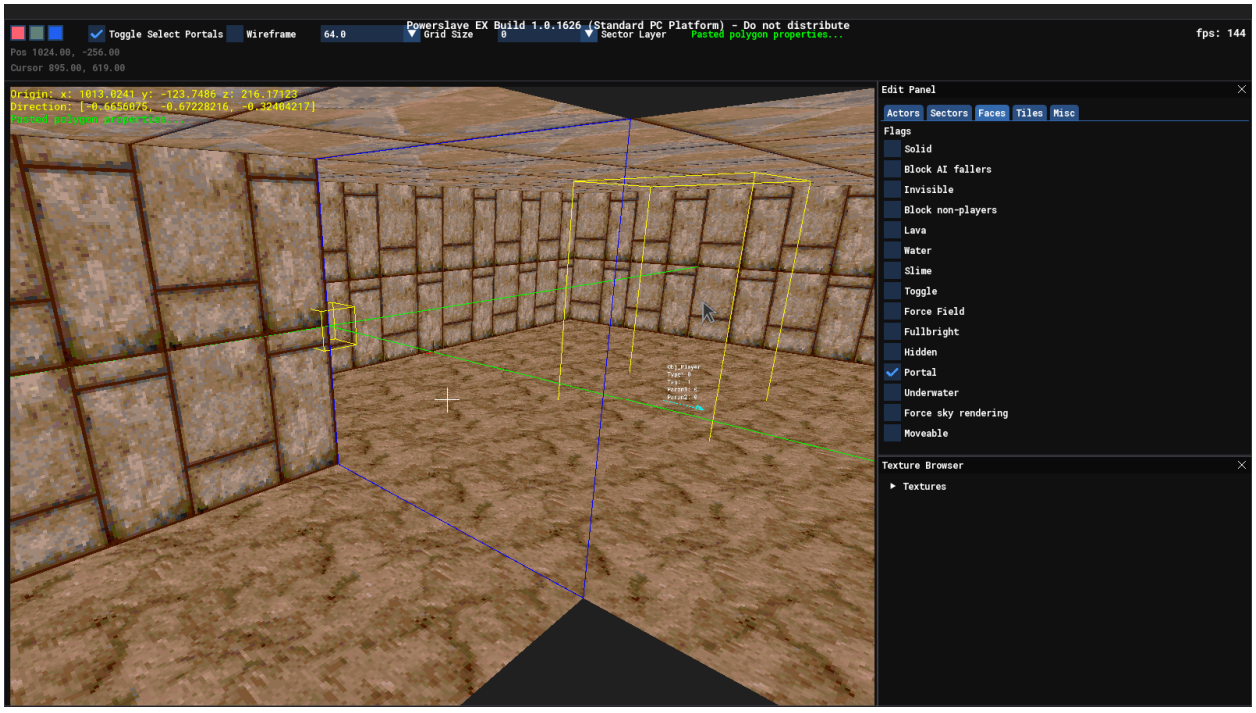
When you enter 3D mode, you should see that now a portal has been created between the two sectors. If the new sector hasn't been textured already, you might want to texture it now.



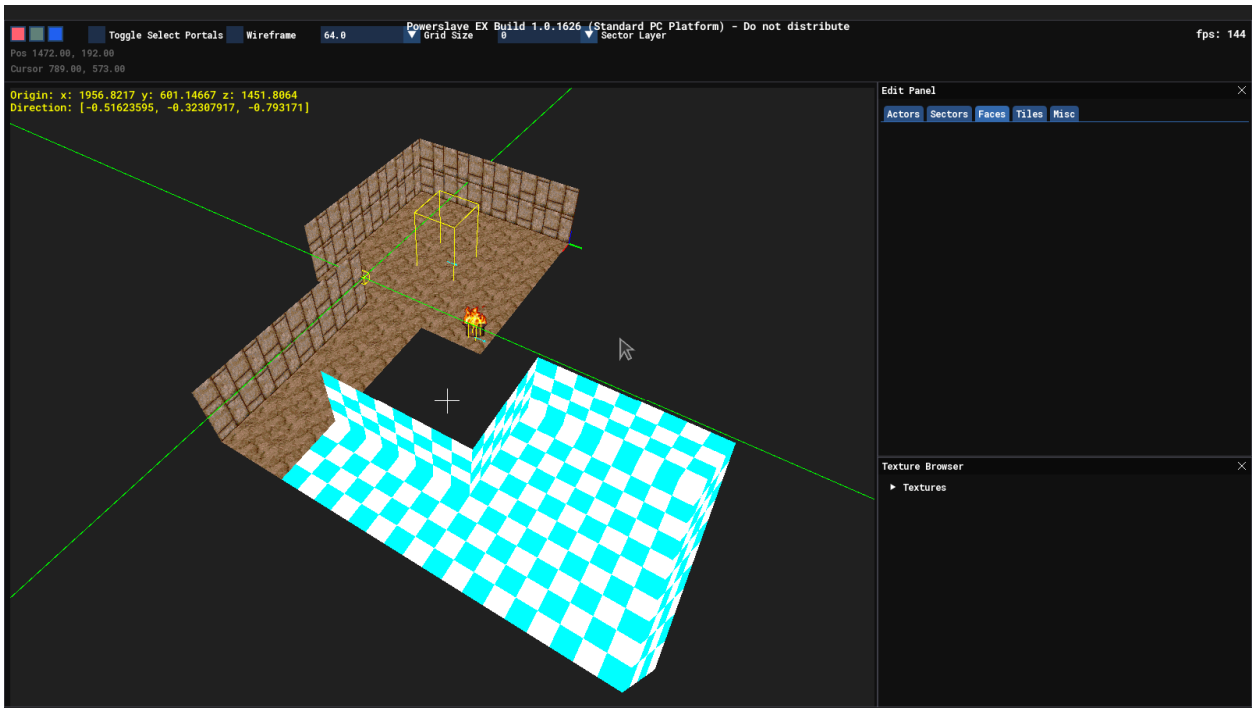
Since a portal has been created, the face used to represent the portal **can no longer be selected**, however you can enable it by right clicking and select **Toggle Select Portals**

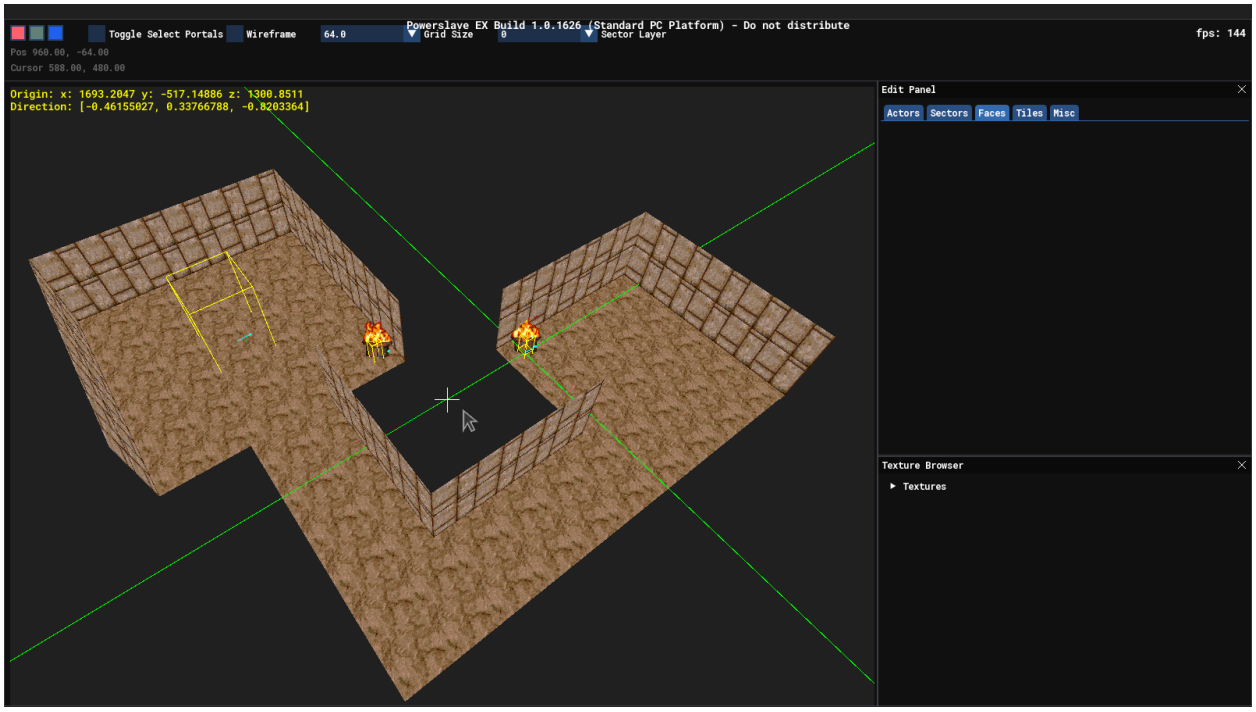


Enabling this will now allow you to select faces marked as portals, which will be displayed in a blue outline when selected.



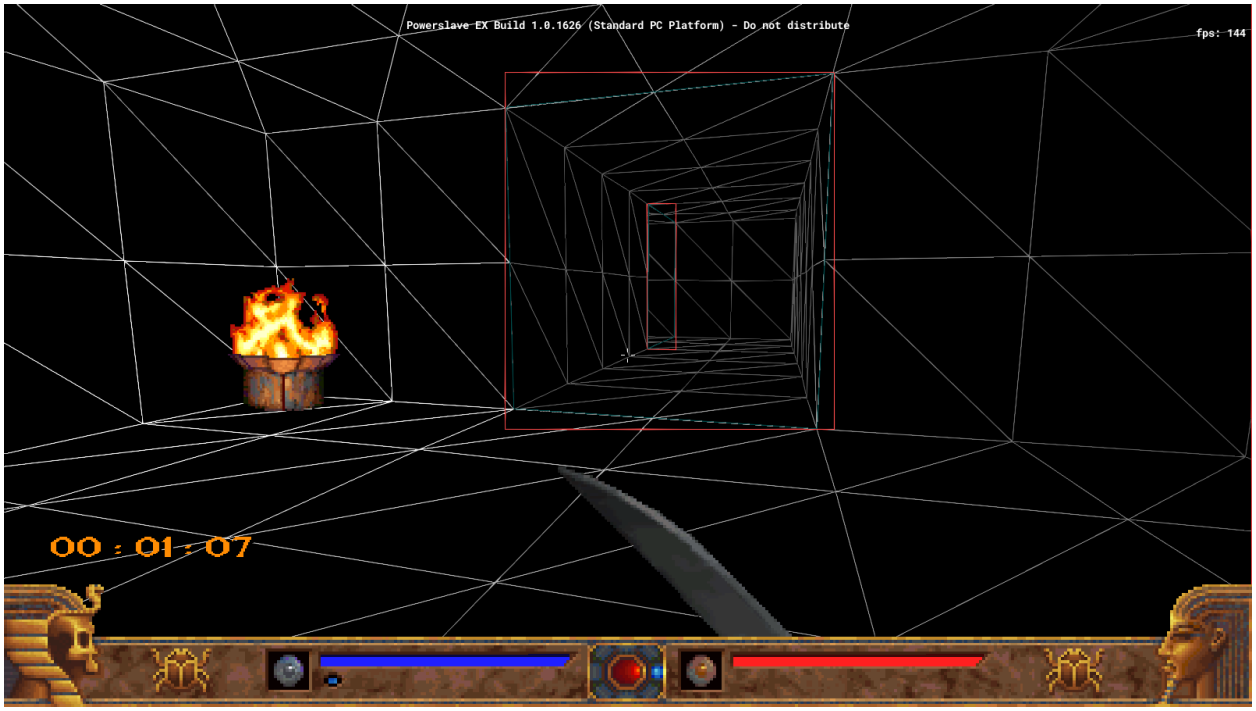
To quickly summarize, if there are two sectors whose faces are at the same length and are lying directly on top of one another will automatically link the two sectors together and the faces will be turned into portals. Try creating a few more sectors and linking them together to make two rooms connected by a hallway.

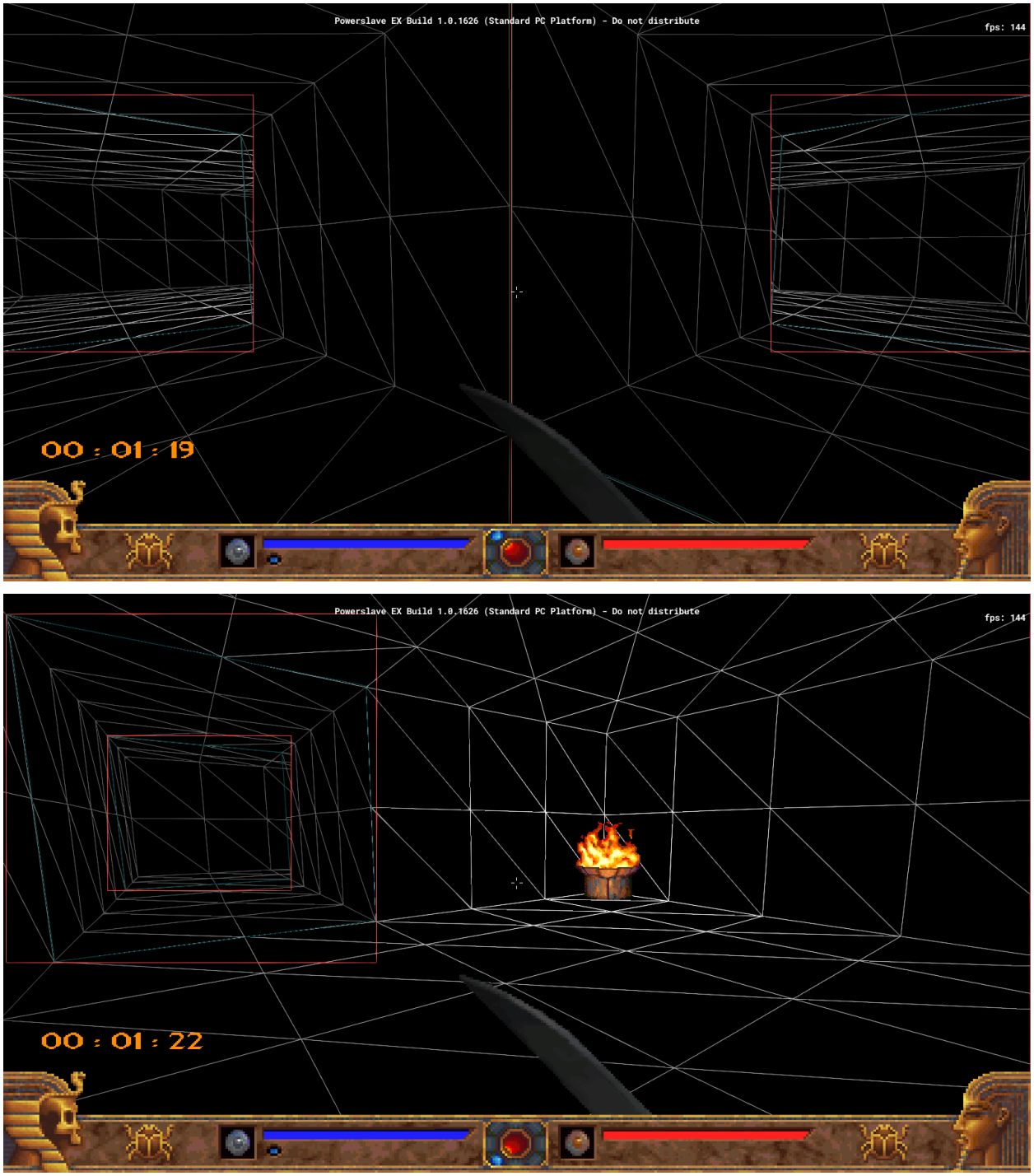




Don’t forget to rebuild lights before testing

As mentioned before, portals are extremely important for visibility culling and are needed for linking sectors together. To better understand how it works, bring down the console and type **r_wireframe 1**, press enter and then **r_showportals 1** and press enter as well. This will visualize how portals affect your visibility into other sectors.

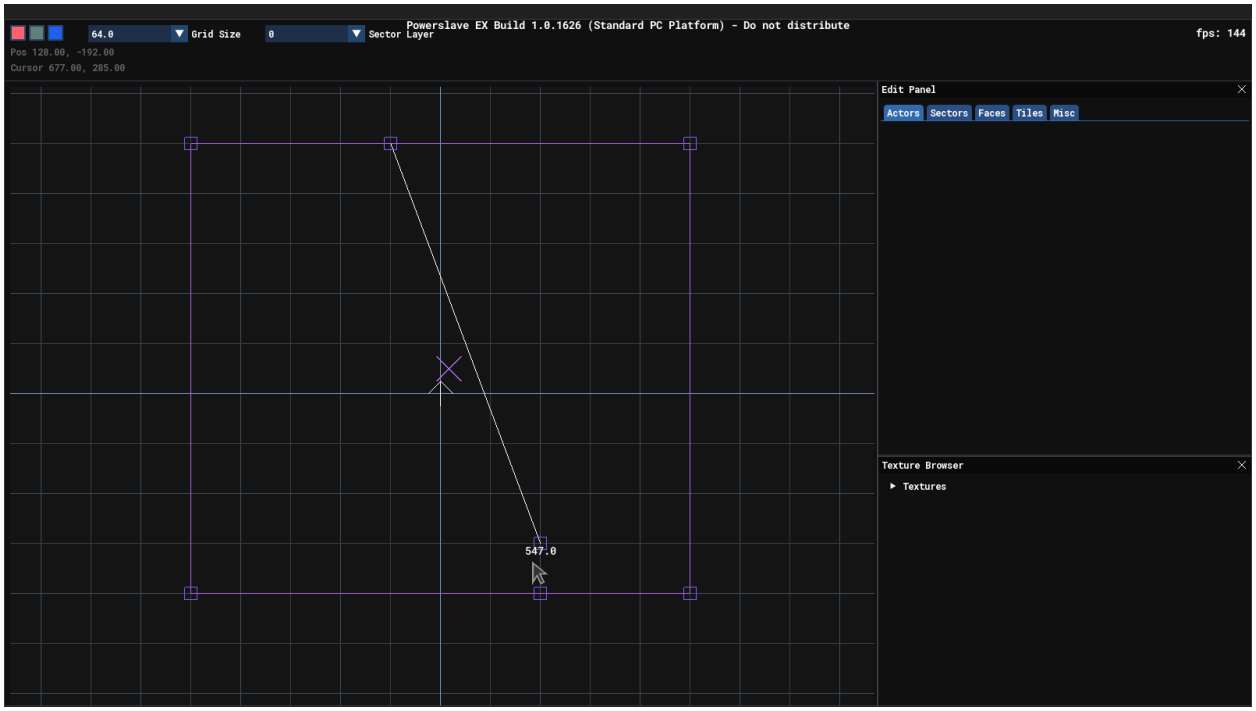


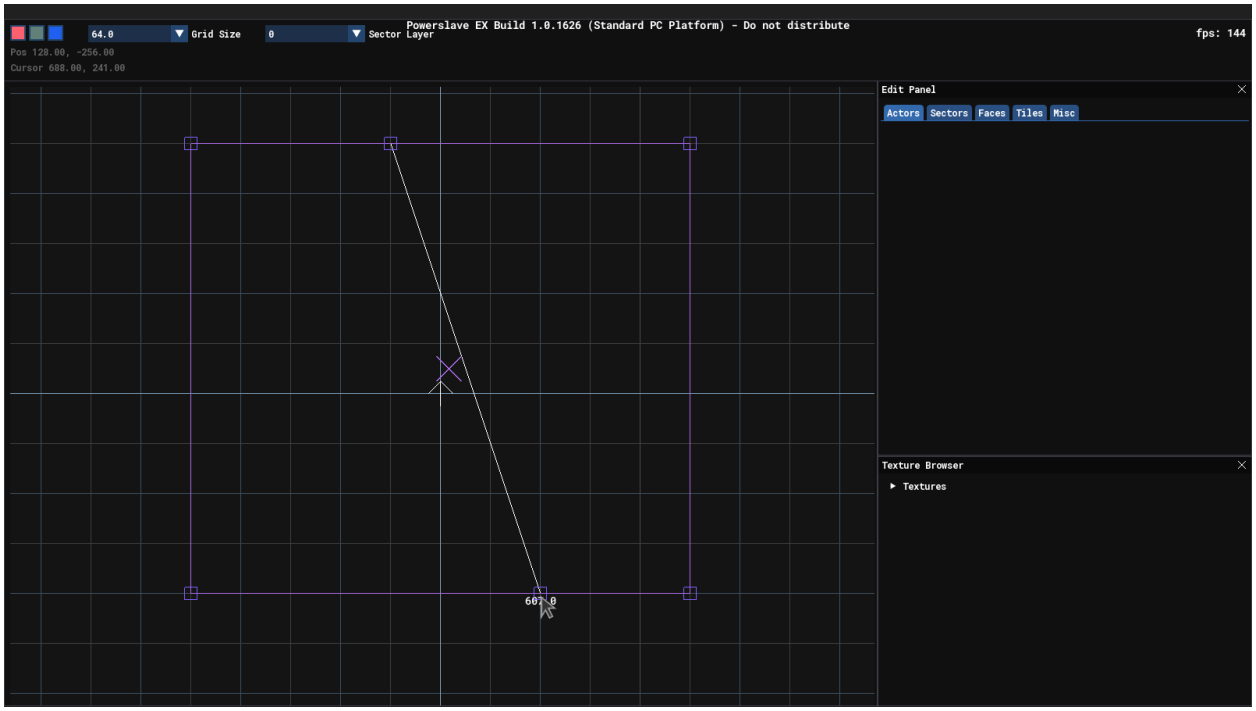


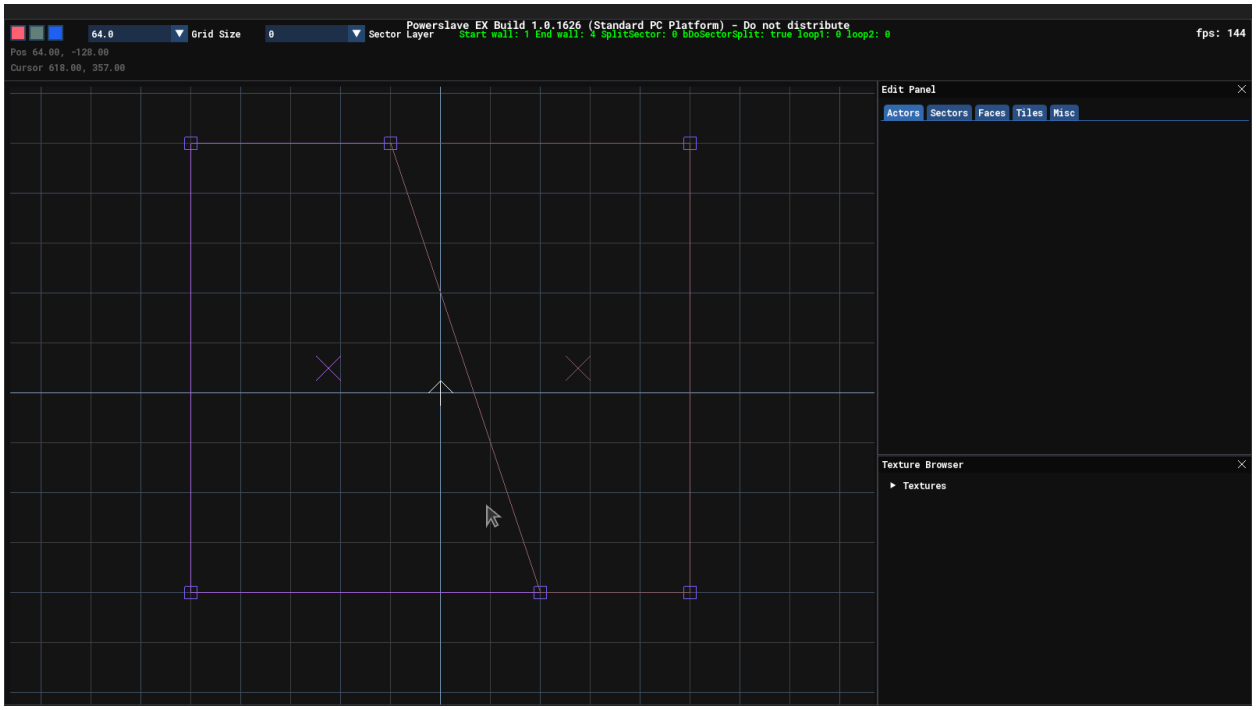
Splitting a sector

In order to split a sector into two, you will need to be in 2D mode. Splitting is done by hitting **shift + space** to enter draw sector mode and then drawing a line from one point of a sector to

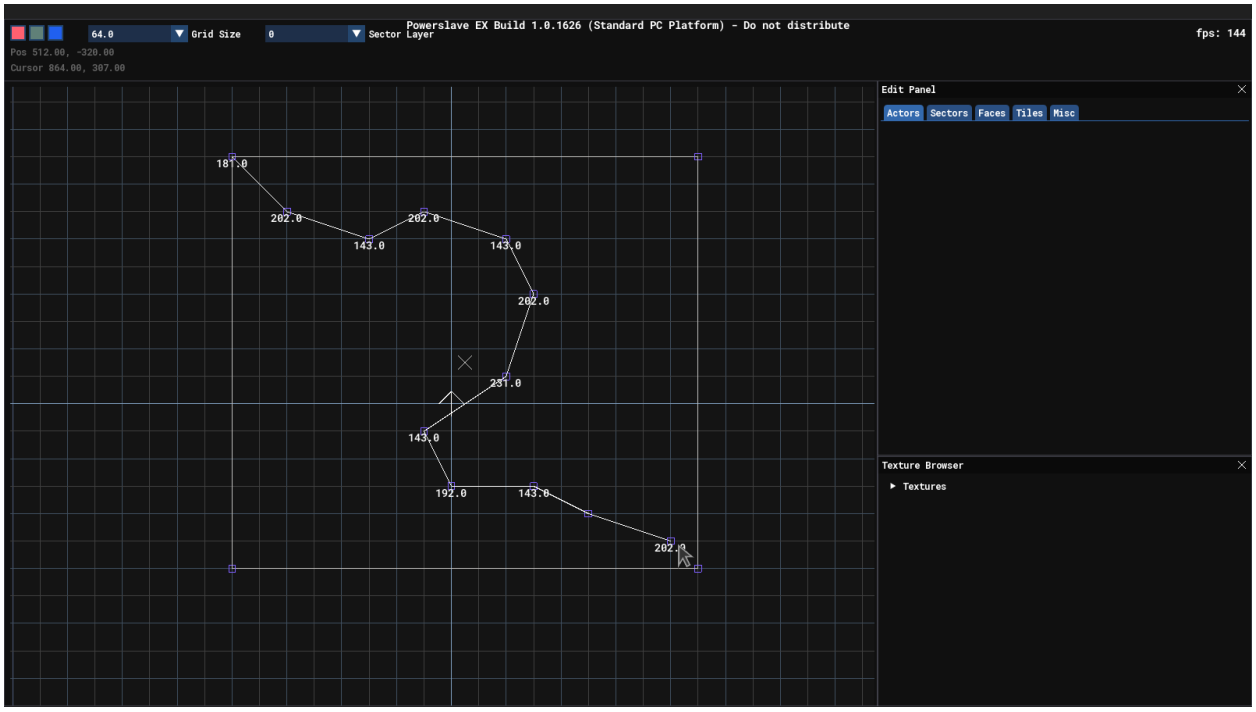
another point in the same sector. Upon clicking the left mouse button on the second point in that sector, it will be split and will automatically exit out of draw mode.



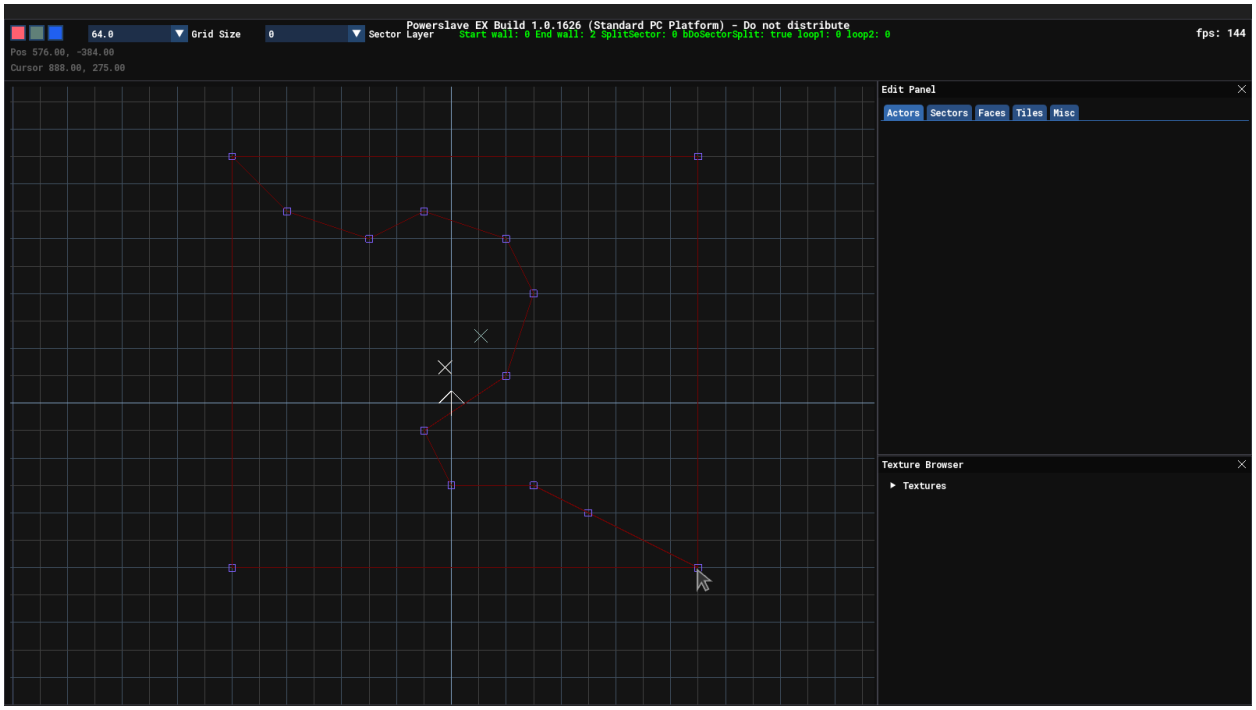




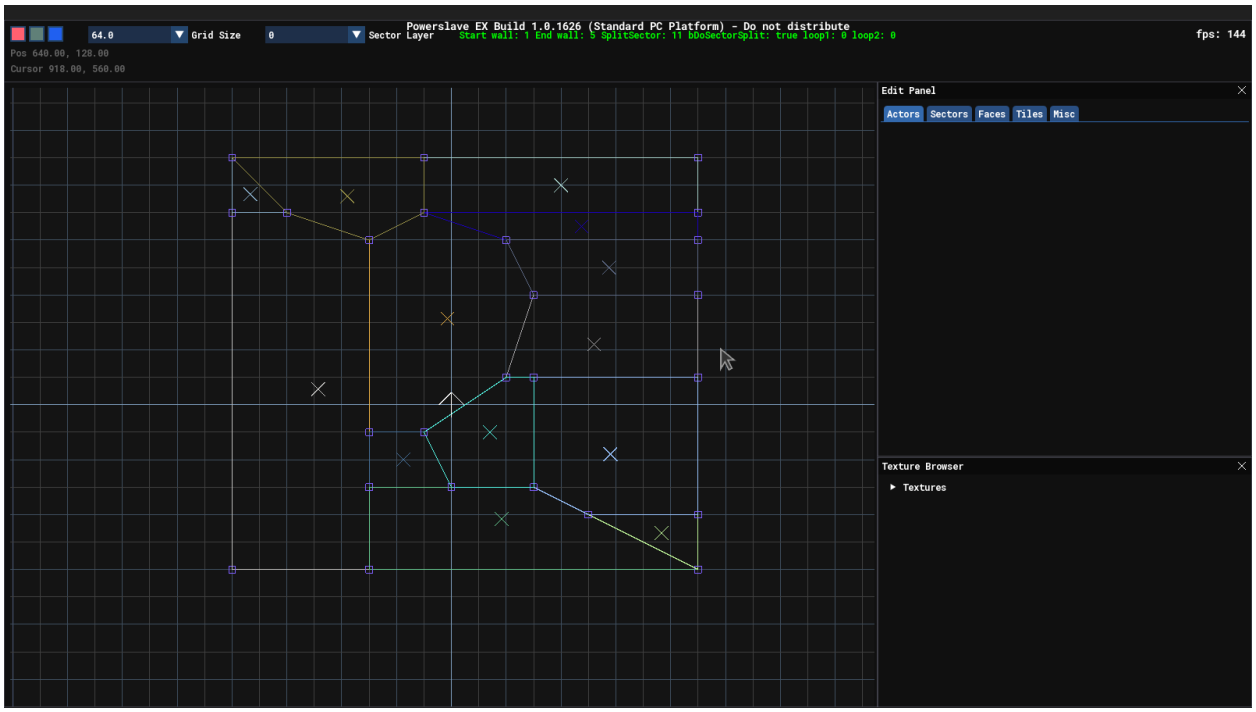
Note that you are not restricted to drawing a single straight line to split a sector. You can use any kind of shape as long as the last connection is on an existing point in the sector.



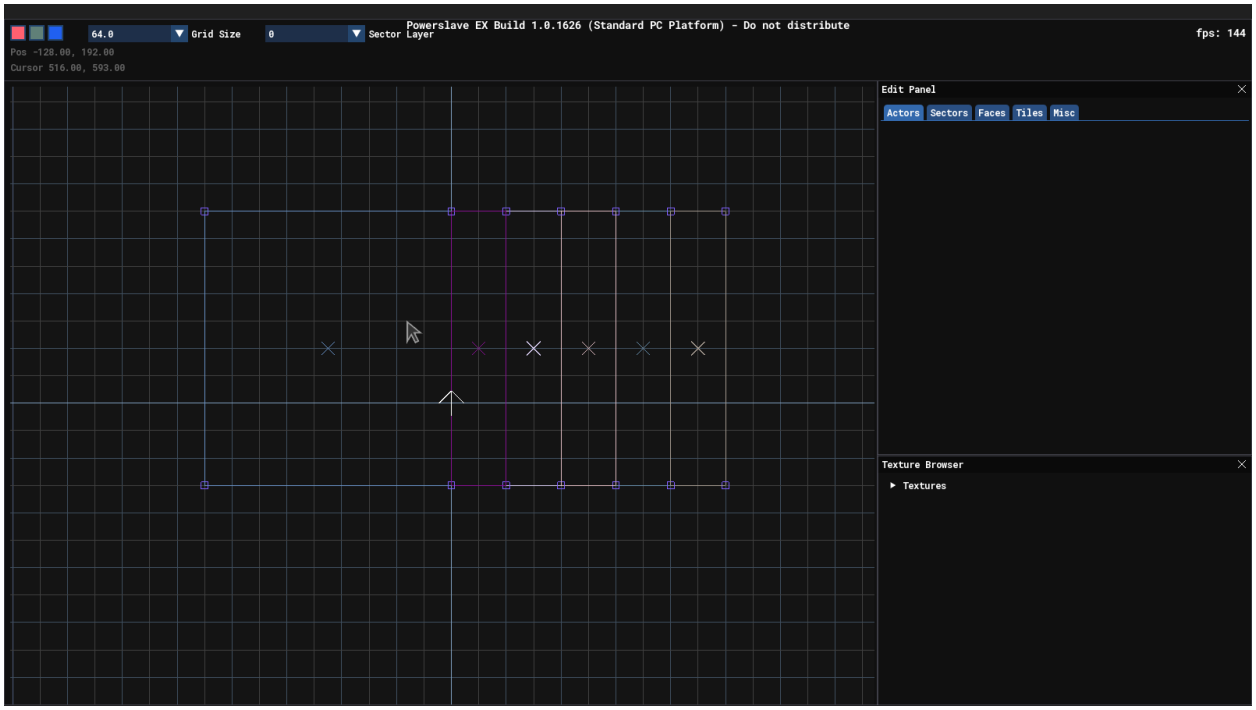
Of course doing it like this will result in both sectors being non-convex.

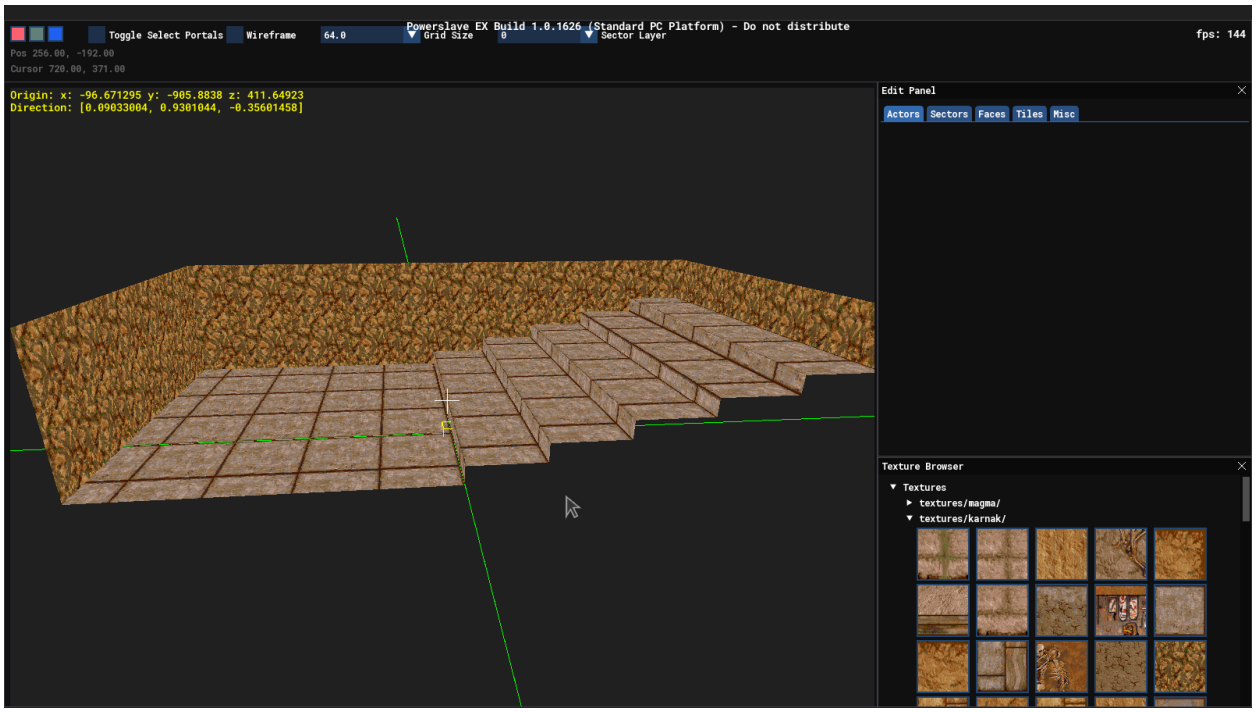


So afterwards, you will need to further split your sectors so they are all convex, resulting in a lot more sectors being produced.



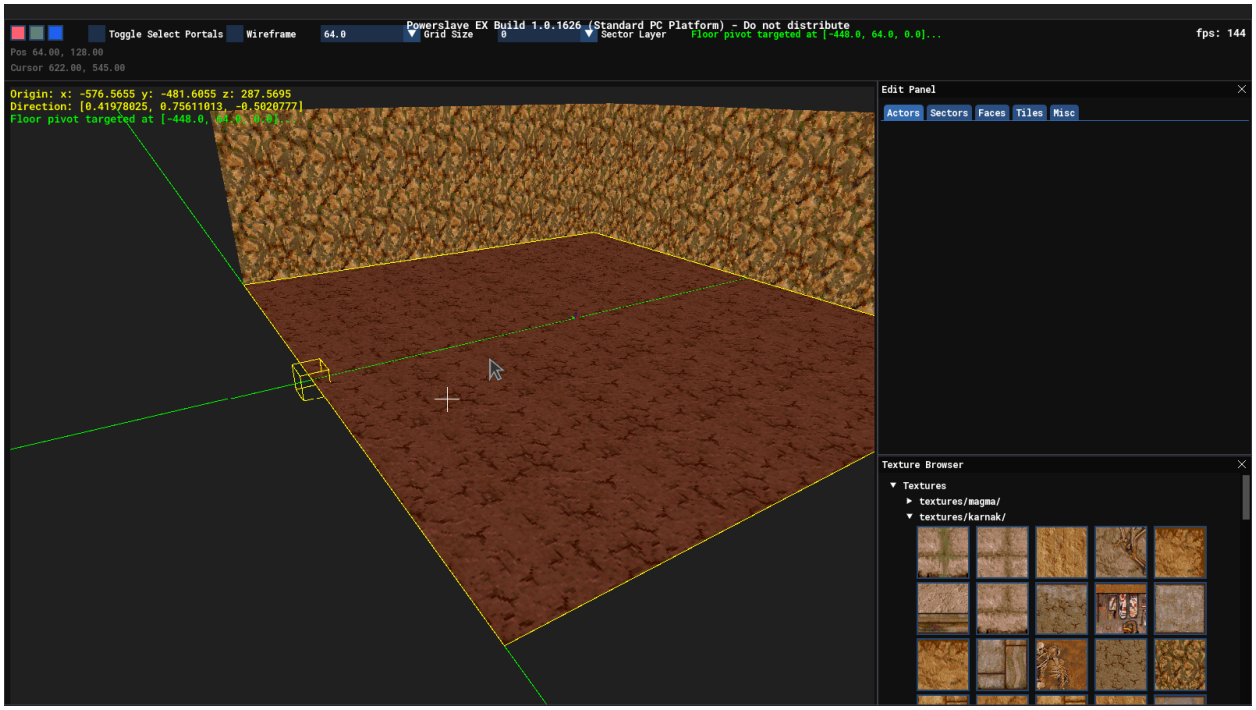
Sector splitting has many uses, one common usage is creating a staircase.



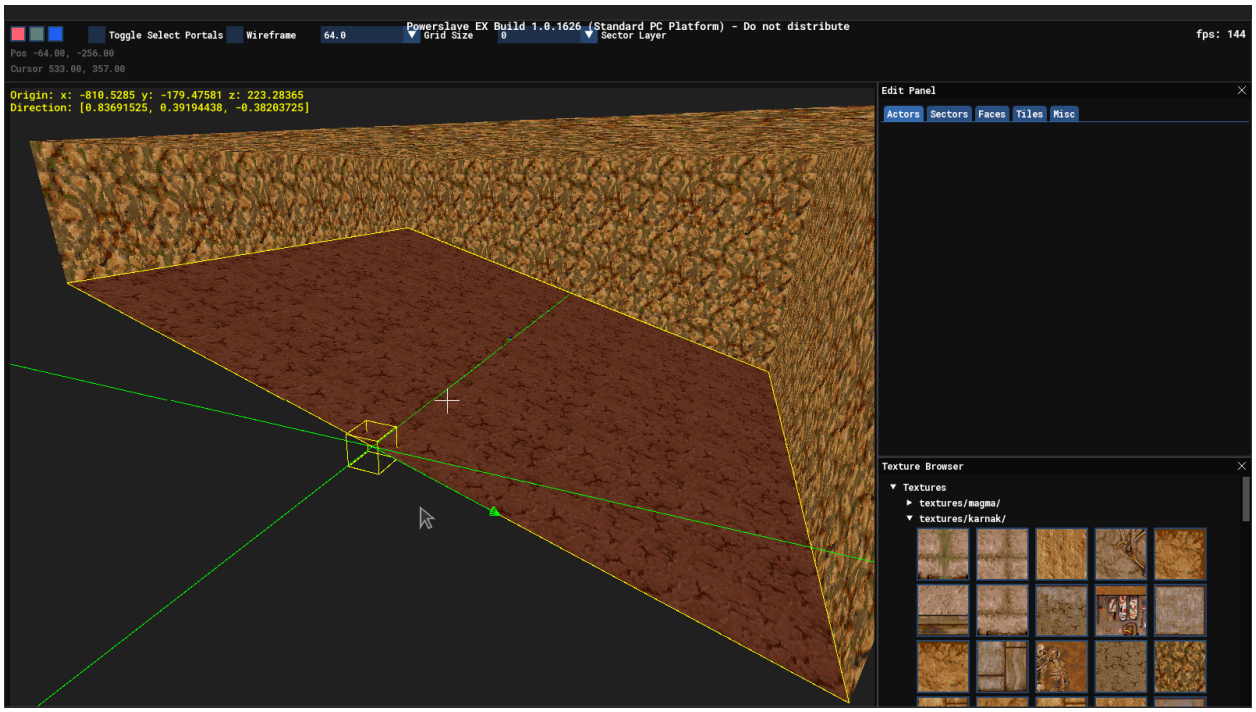


Slope Manipulation

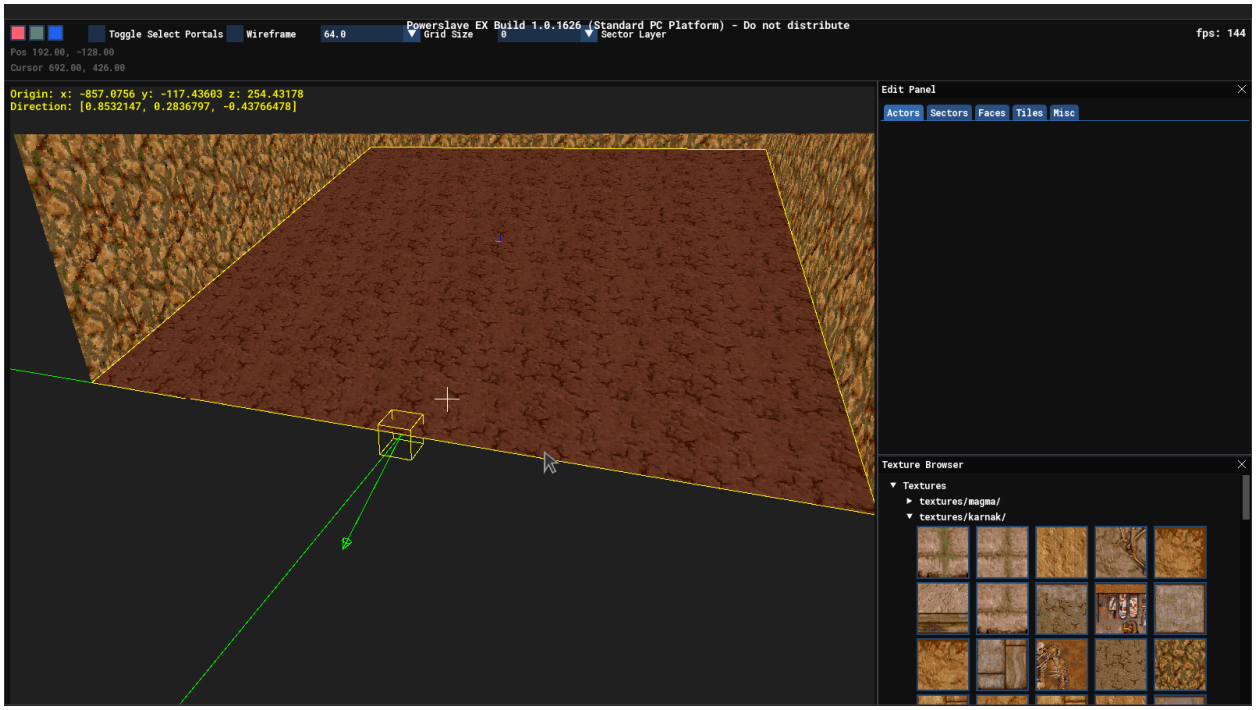
Modifying the slope of a ceiling or floor is pretty straightforward. While having a floor face selected, press **ctrl + left mouse button** to set the marker to a position that will serve as the **pivot point** to rotate the surface on and then press **keypad multiply** to set the pivot point.



While the surface is still selected, pressing **shift + mouse wheel** will rotate the surface around its pivot point. There will be a green arrow to indicate the direction of the slope.

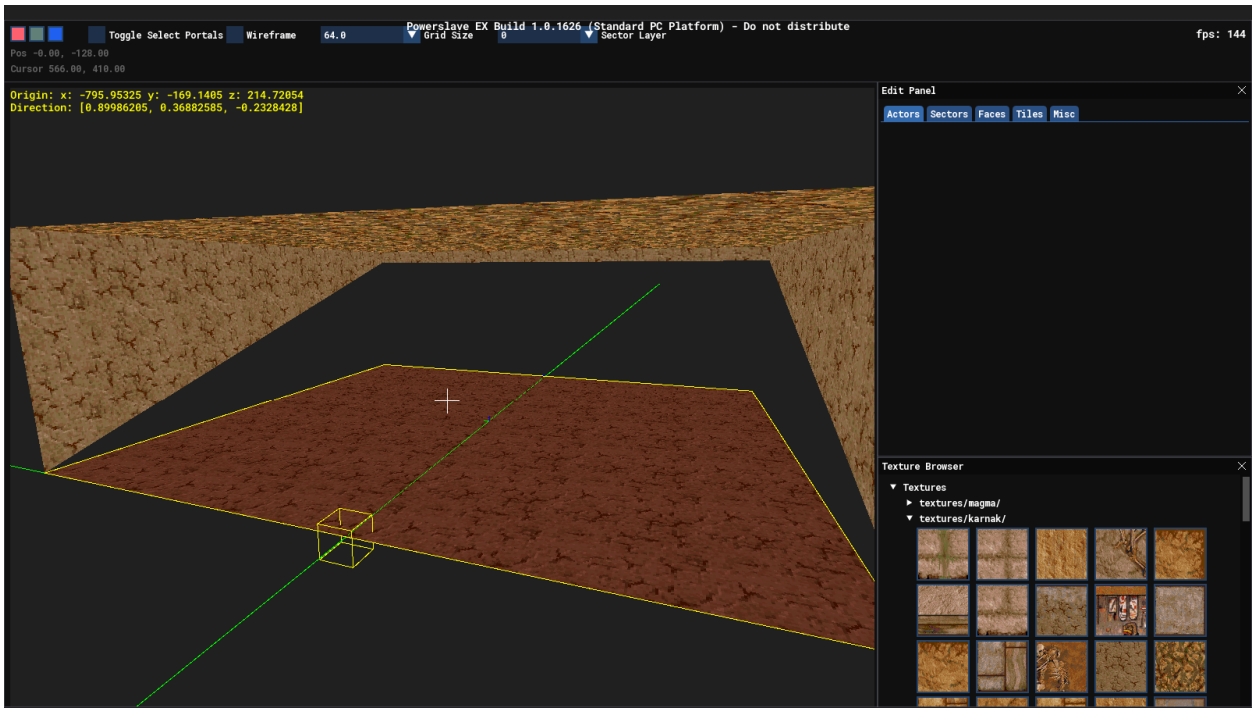


Pressing **ctrl + mouse wheel** will rotate on the xy-axis



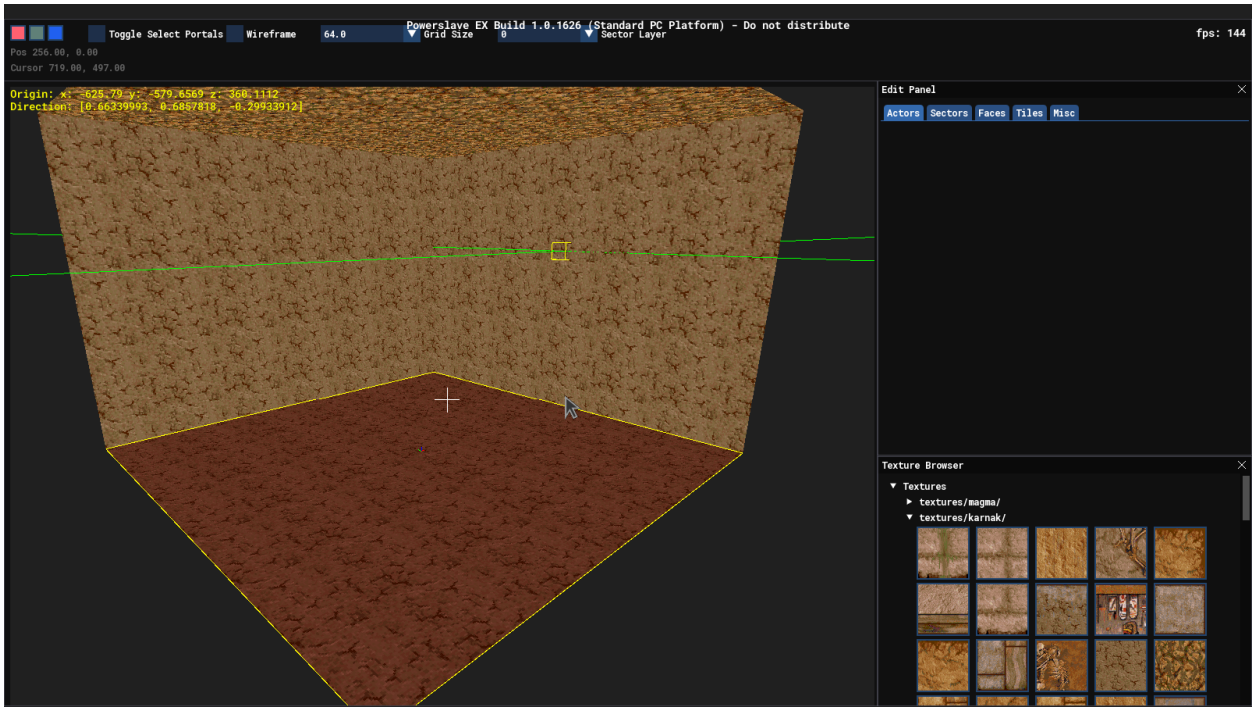
Pressing **shift + end** will reset the floor slope. Pressing **shift + home** will reset the ceiling slope.

BUG: Currently upon resetting the slope, you will notice that the faces do not update to reflect the change.

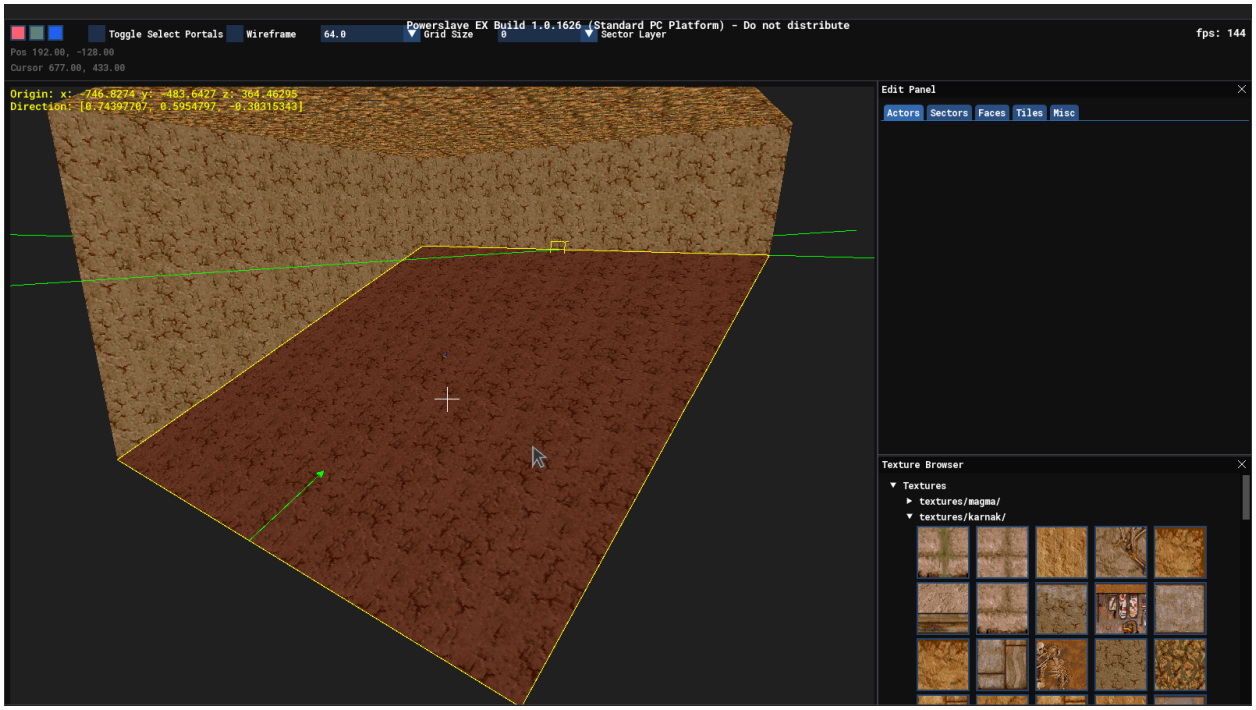


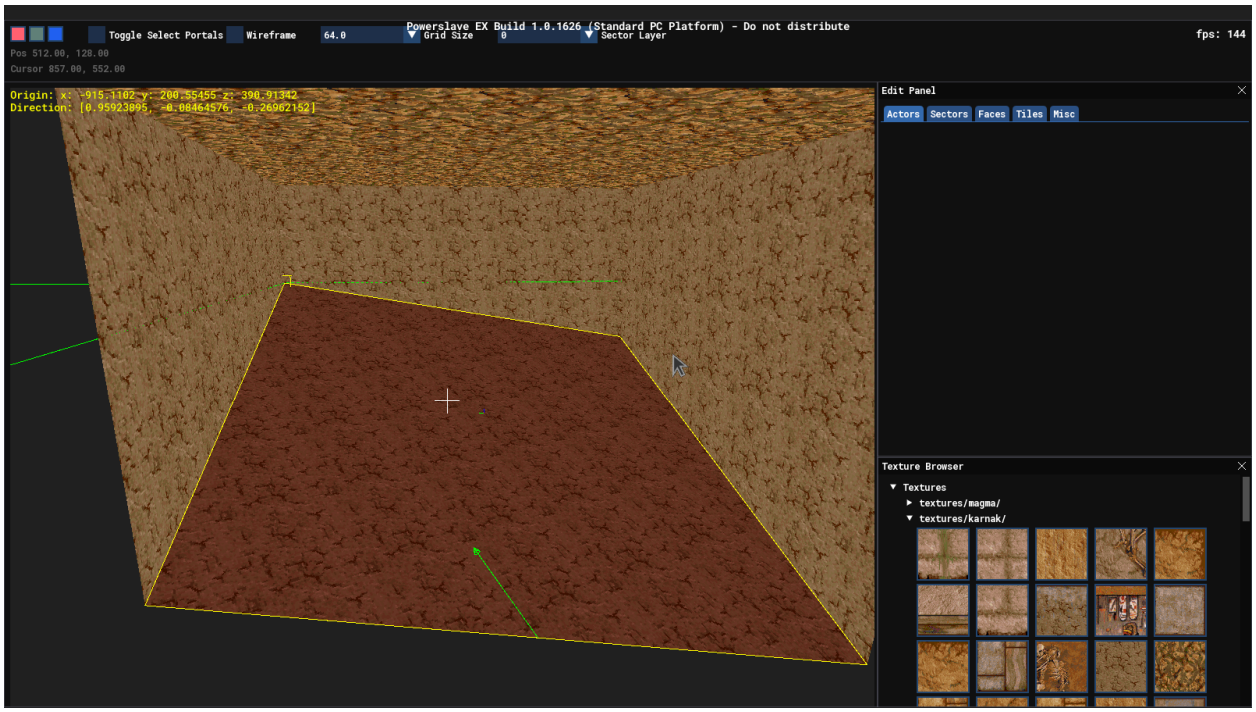
To fix it, simply nudge the floor or ceiling up and down with the **mouse wheel** to force the sector to refresh. Another way is to select a face and then pressing **space** to select the sector, then pressing **r** to force all faces to refresh.

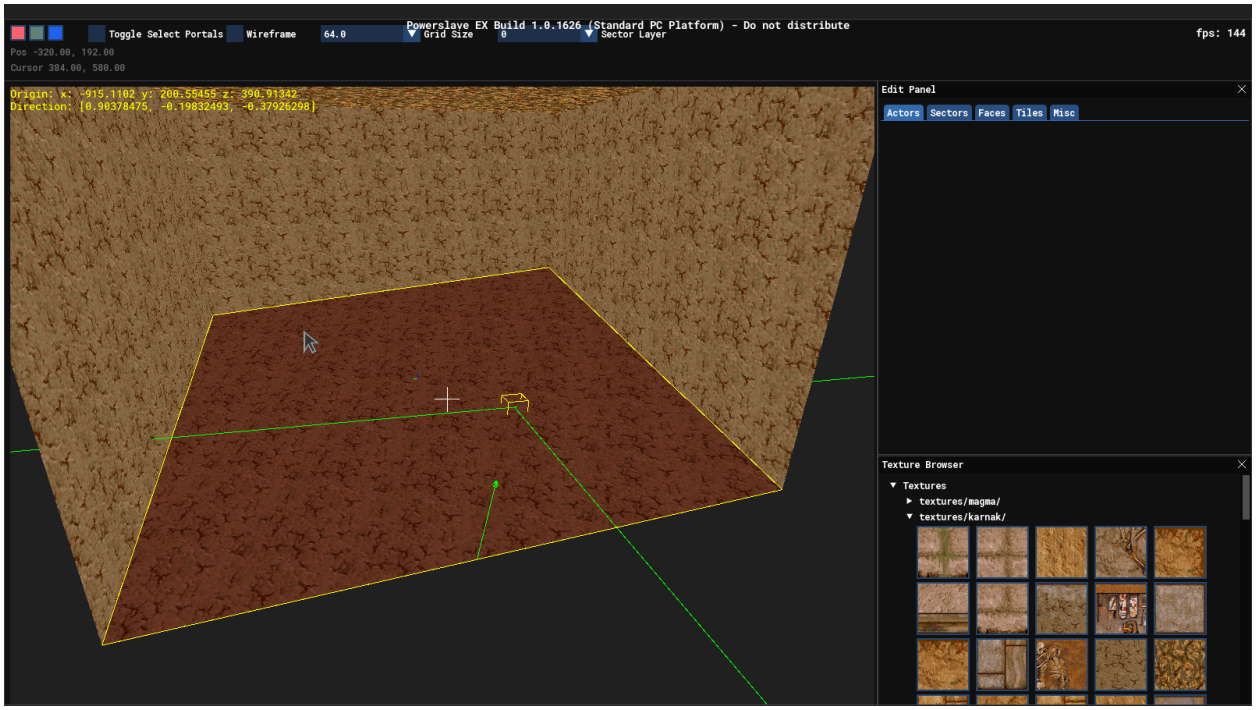
To have much finer control and to ensure alignment on the grid, you can place a marker at a desired location.



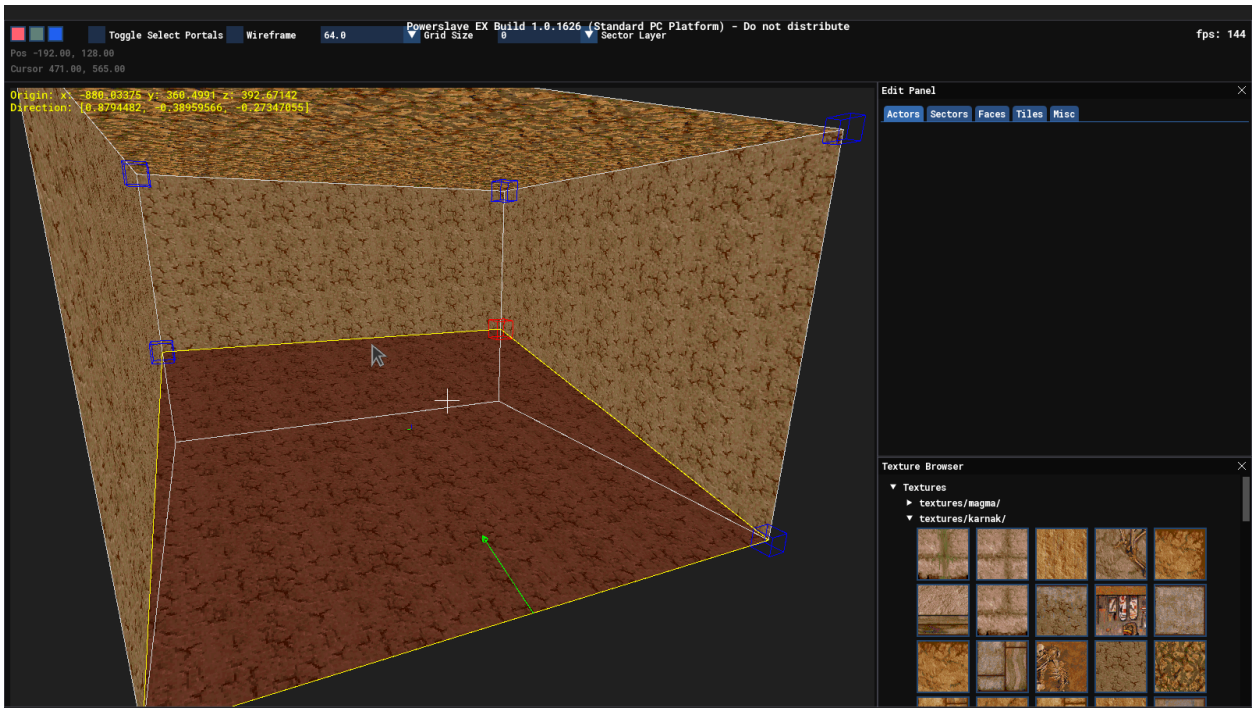
Then press **end** while the floor is selected, or **home** while the ceiling is selected, to make the slope point to that marker location. Below are several examples:


Toggle Select Portals
Wireframe
64.0
Grid Size
0
Sector Layer
Powerslave EX Build 1.0.1626 (Standard PC Platform) - Do not distribute
fps: 144
Pos 192.00, -128.00
Cursor 677.00, 433.00
Origin: x: -746.8274 y: -483.6427 z: 364.46295
Direction: [0.7439707, 0.5954797, -0.30315343]
Edit Panel
Actors
Sectors
Faces
Tiles
Misc
Texture Browser
Textures
textures/magma/
textures/karnak/

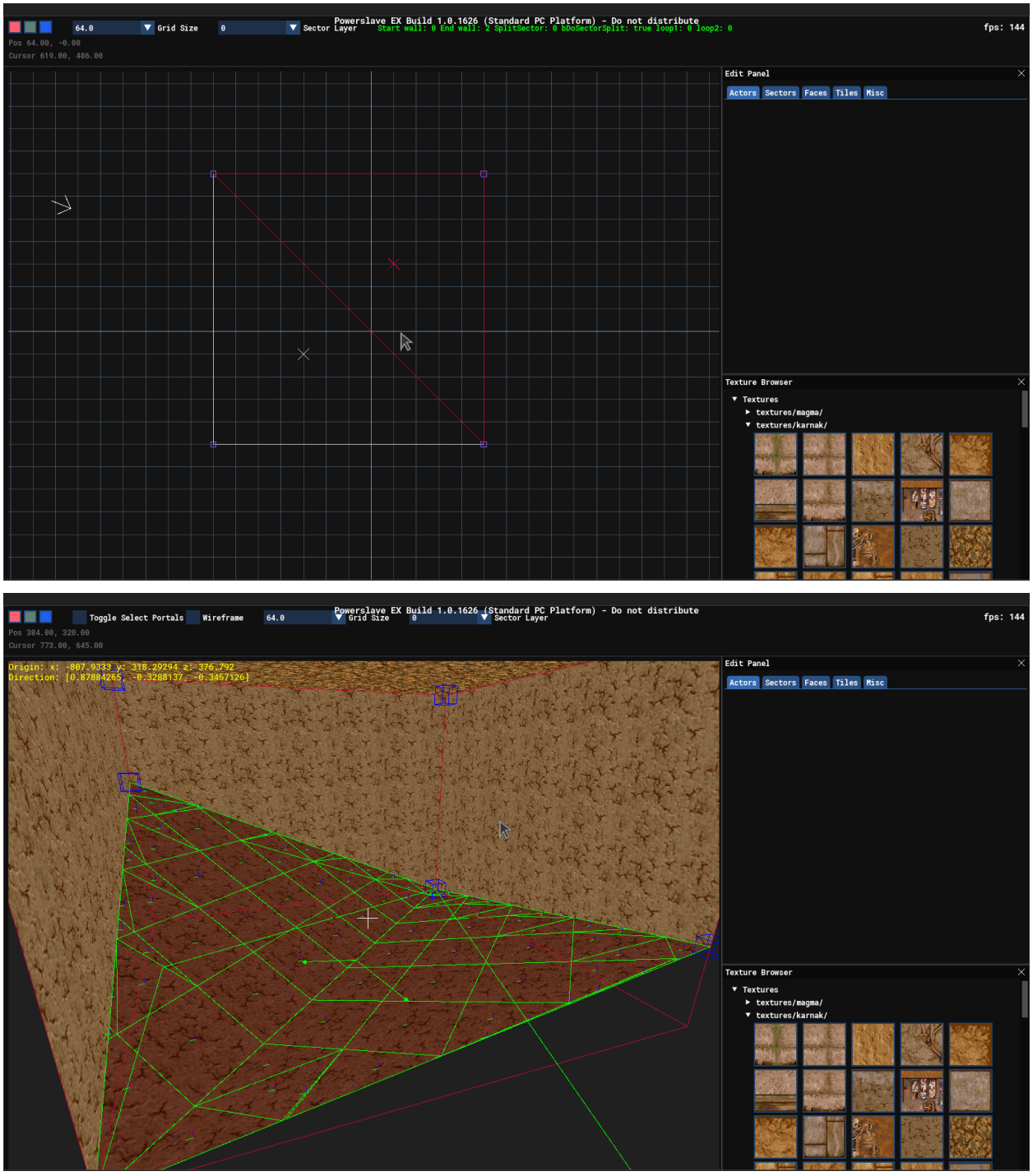

Toggle Select Portals
Wireframe
64.0
Grid Size
0
Sector Layer
Powerslave EX Build 1.0.1626 (Standard PC Platform) - Do not distribute
fps: 144
Pos 512.00, 128.00
Cursor 857.00, 552.00
Origin: x: -915.1102 y: 200.55455 z: 390.91342
Direction: [0.95923895, -0.08464576, -0.26962152]
Edit Panel
Actors
Sectors
Faces
Tiles
Misc
Texture Browser
Textures
textures/magma/
textures/karnak/



When a sector is selected, it will display the boundary vertices of the ceiling/floor surfaces. You can select these and then nudge them up/down using the **arrow keys**



This method is largely preferred for creating terrain. However it's recommended to make the **sector consist of 3 points only**, this way you can have better control on the terrain.

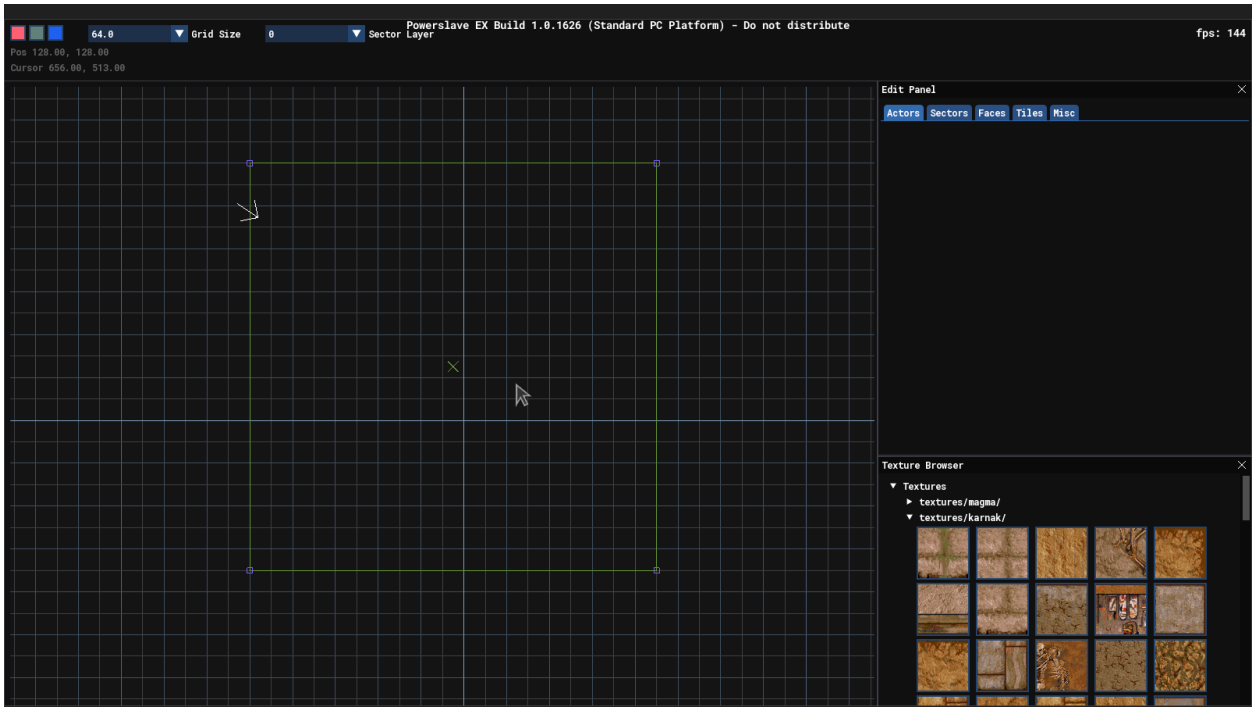


If done right, you can create some really interesting terrain geometry.

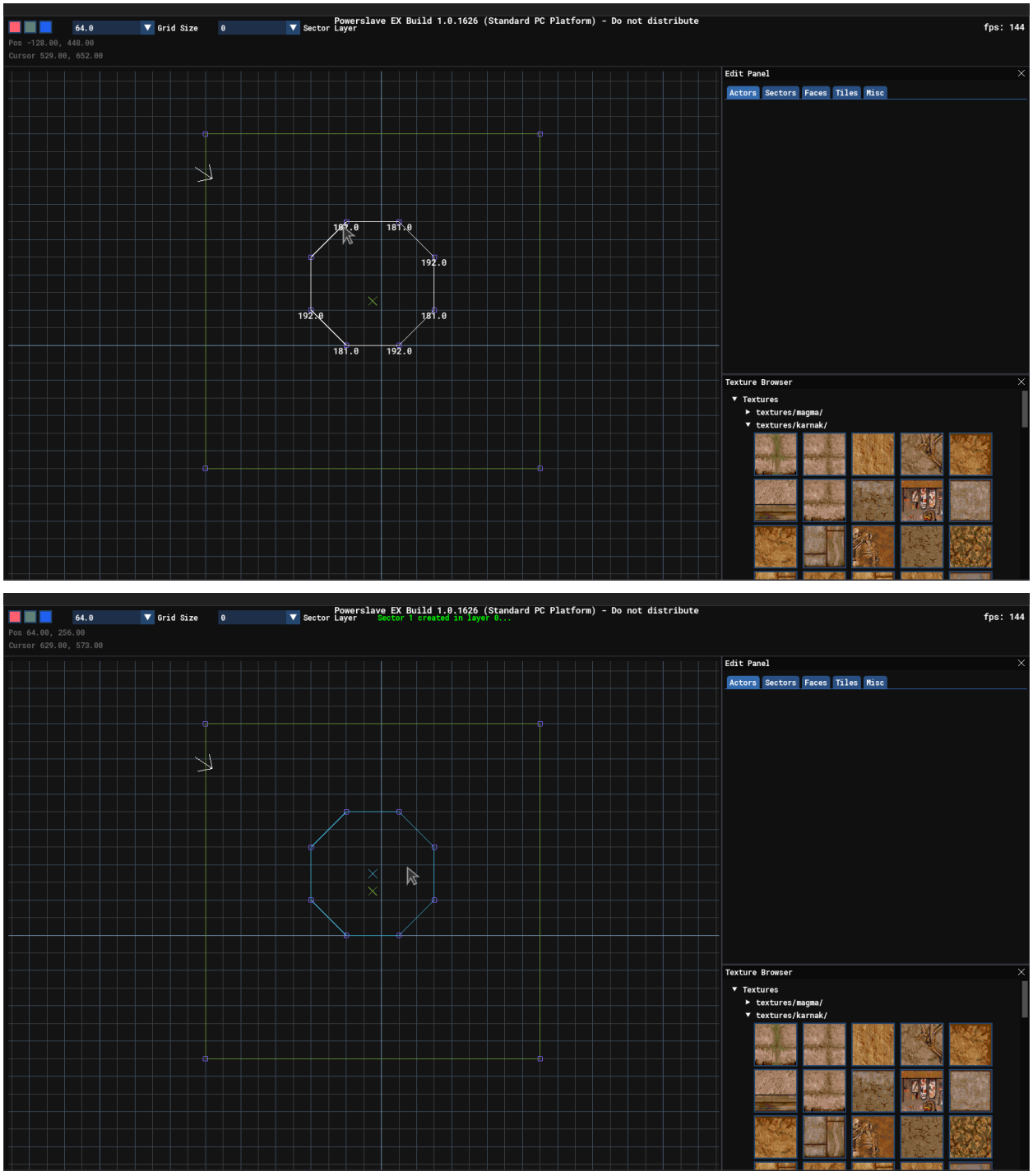
CSG Operations (Experimental)

When designing a room, if you intend to add pillars or some other geometry inside a sector, that usually means that you need to chop the sector up into convex shapes and rearrange some things. This utility is meant to help speed up this process. Below is a quick demonstration of this tool.

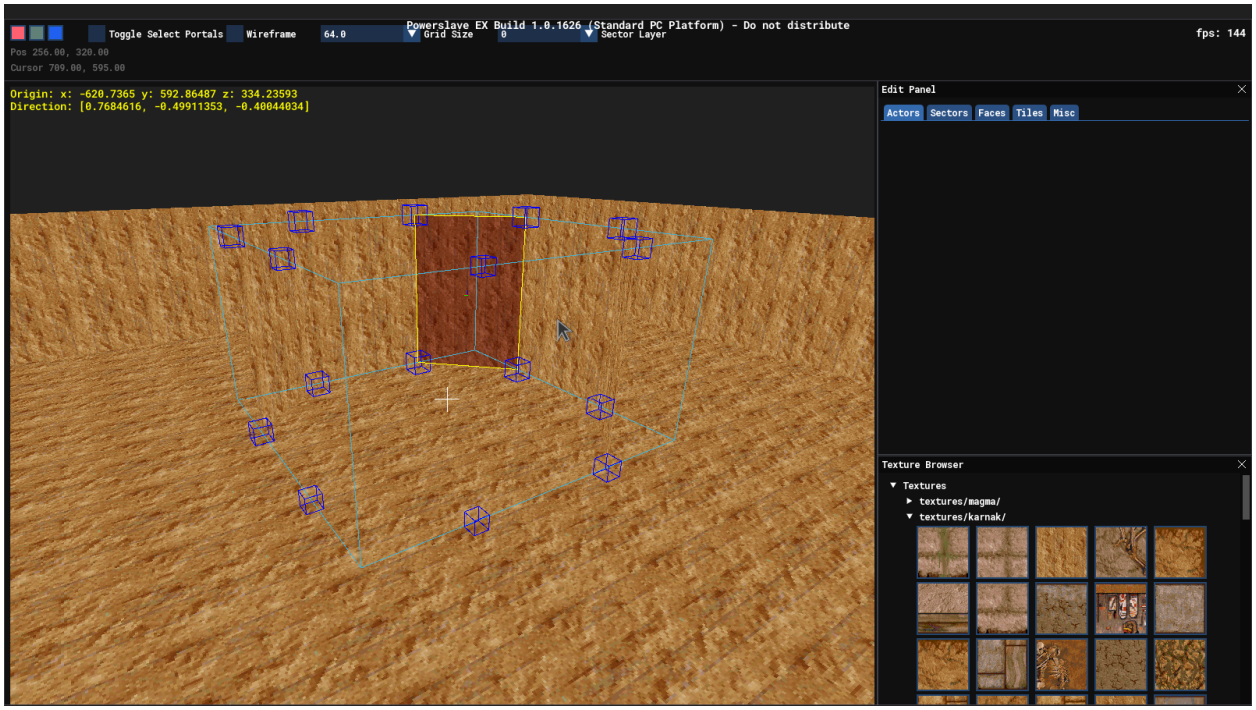
Start off with a simple 4-sided room.



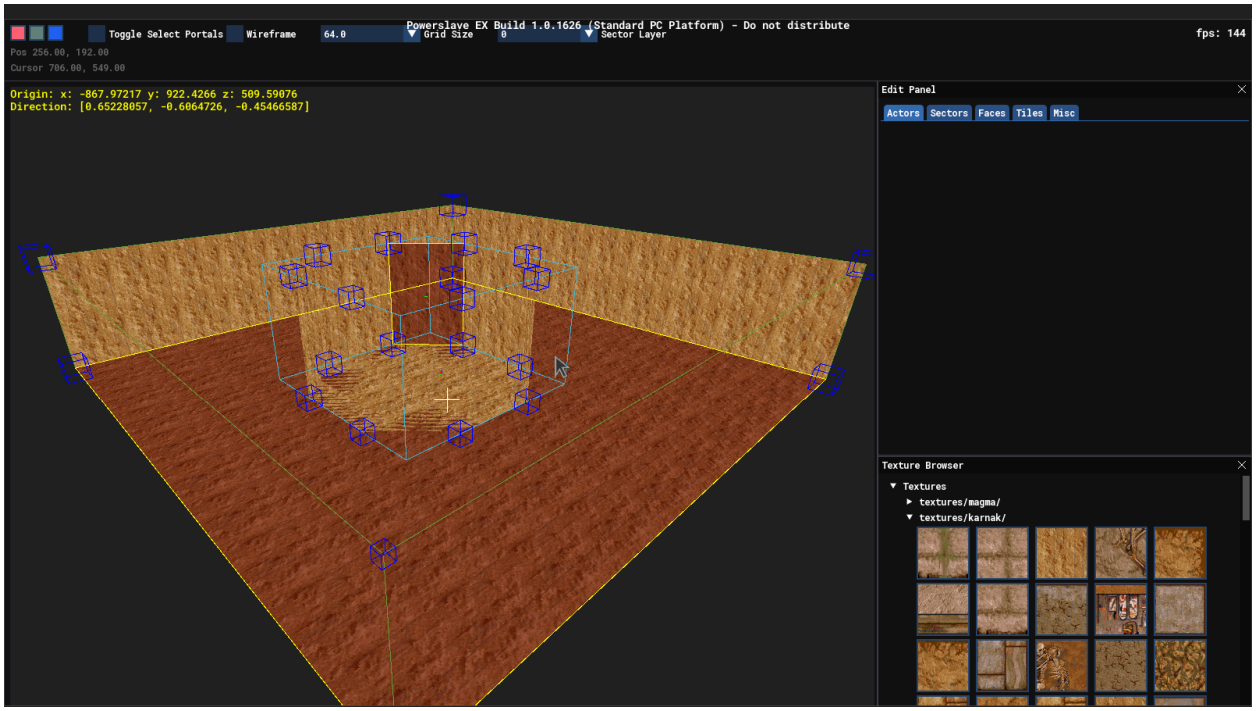
Next create another sector that is meant to be your pillar



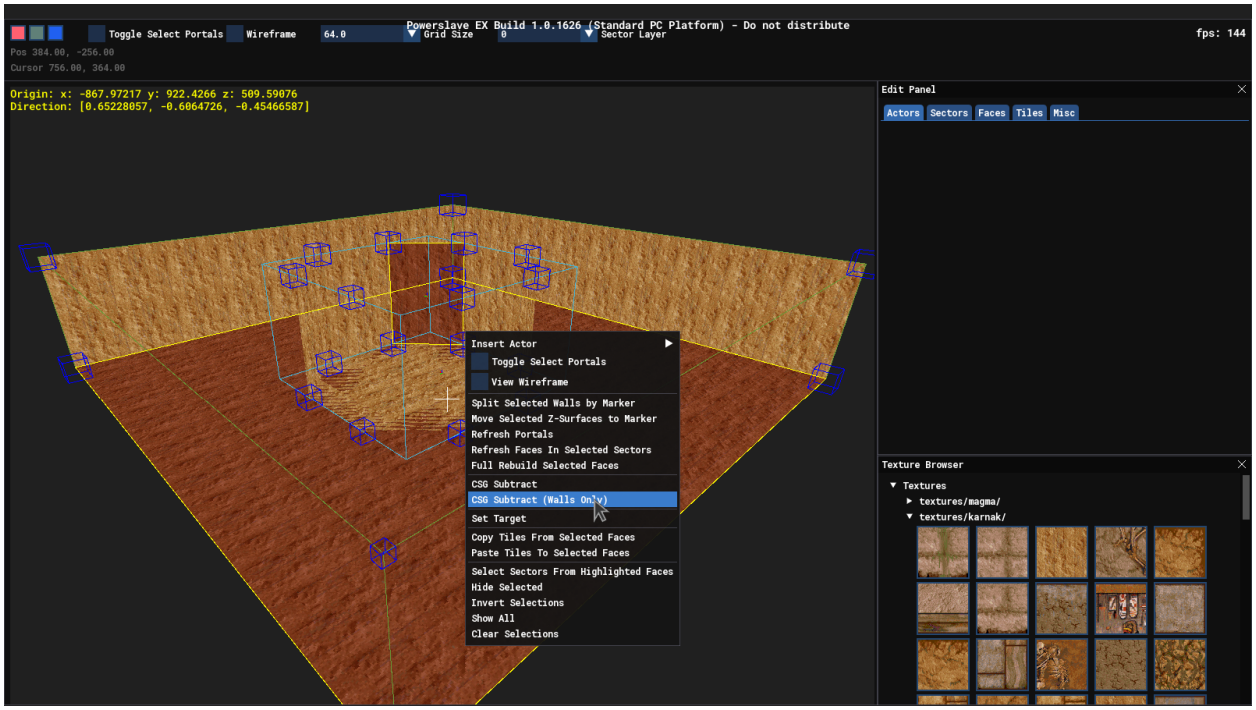
Enter 3D mode and select the sector used for ‘carving’, which in this case, the pillar.



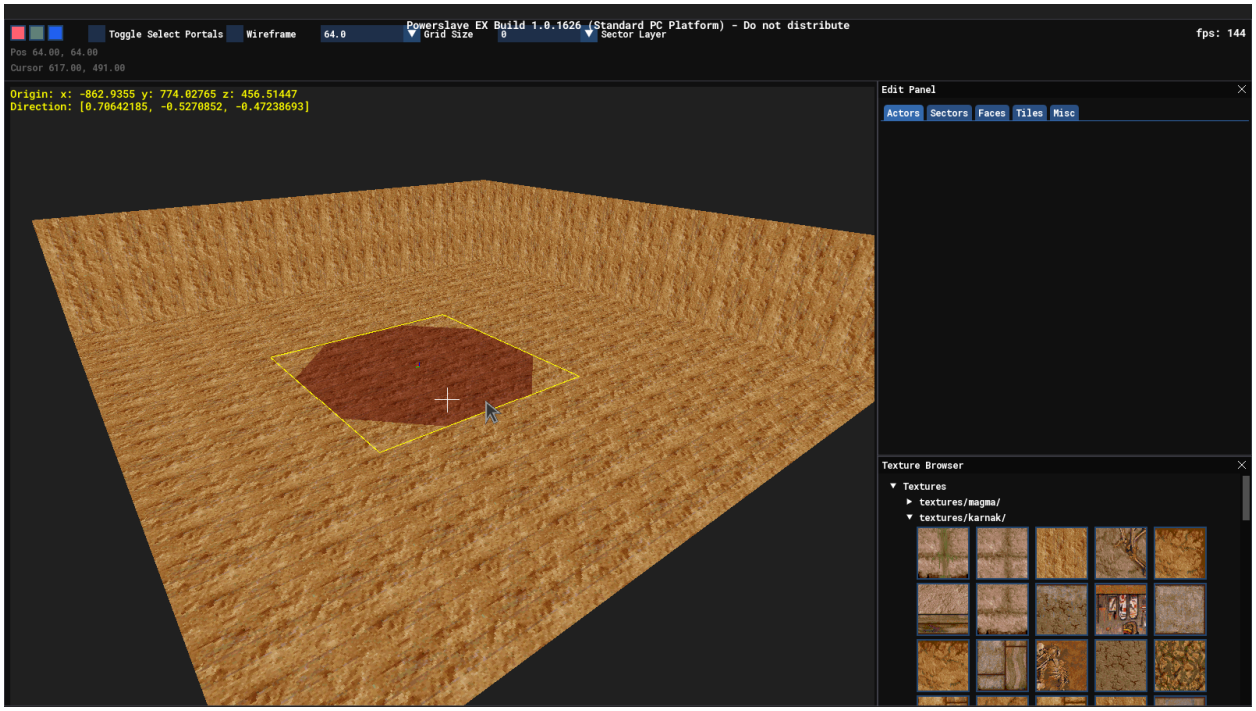
Next select the sector that you want to be chopped up.



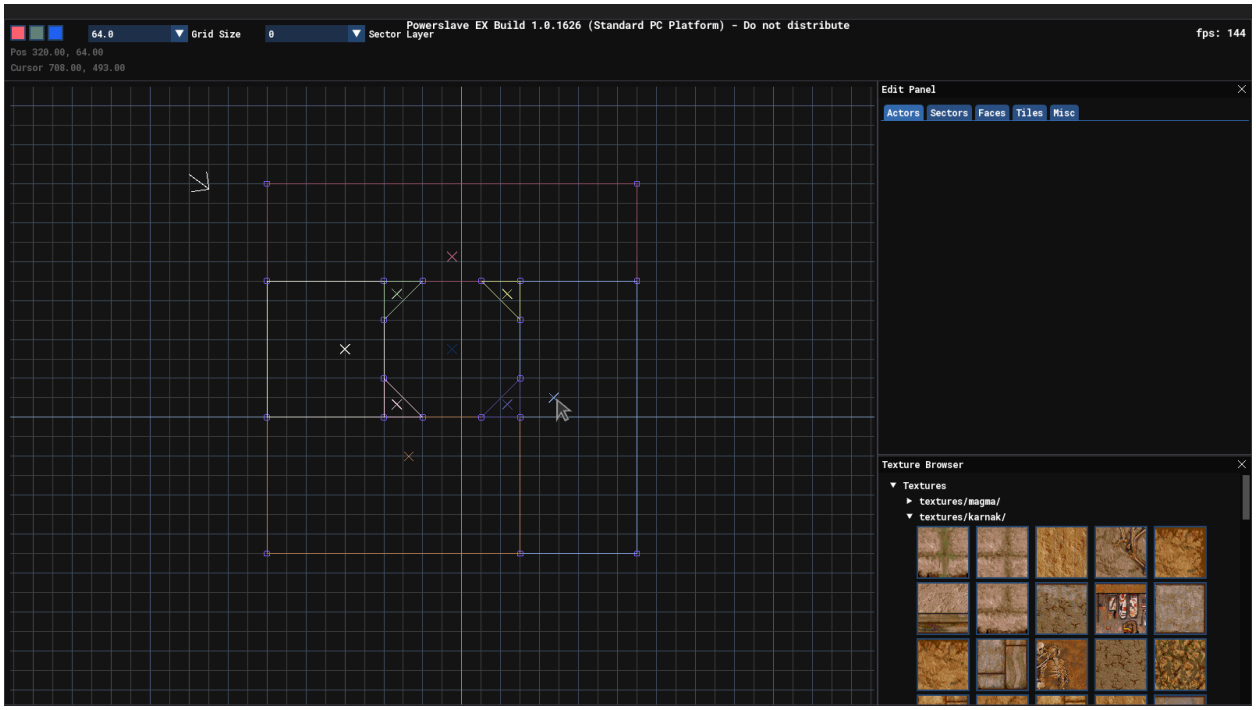
Next, exit camera active mode, press the right **mouse button** to bring up the context menu and select **CSG Subtract (Walls Only)**.



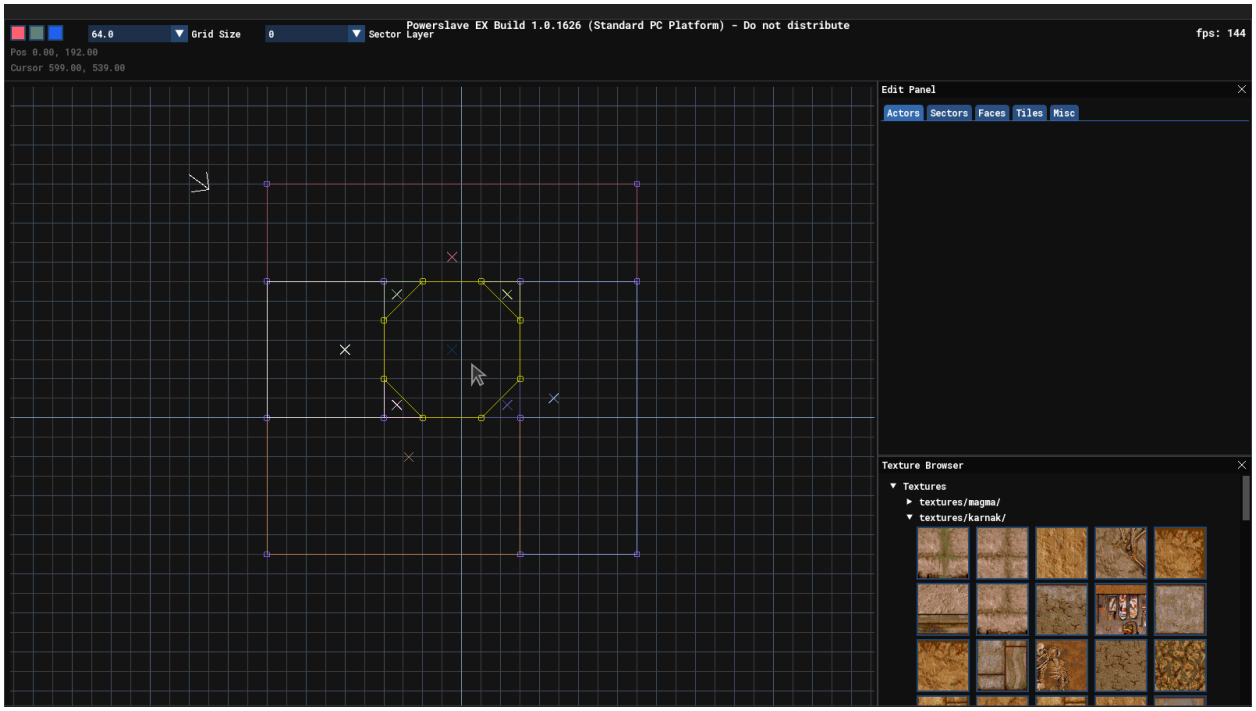
Doing this will now chop up the outer sector and the pillar sector you added is now carved in.



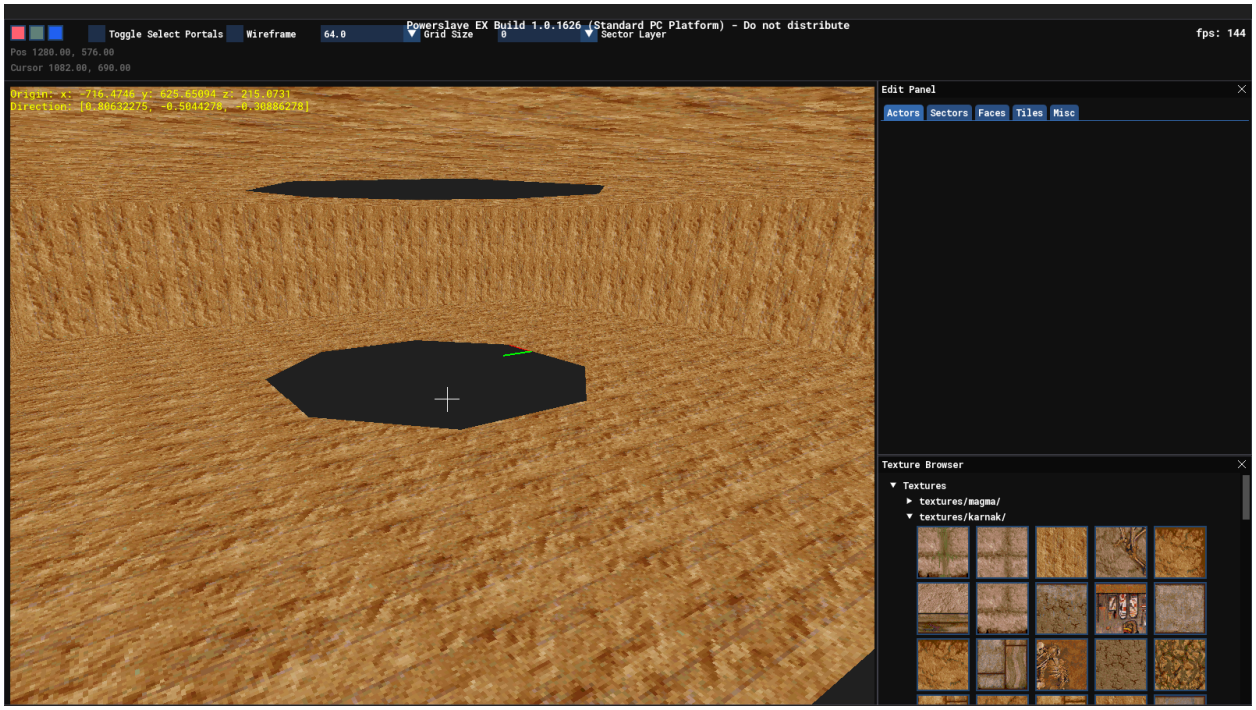
In 2D view, your map now looks like this:



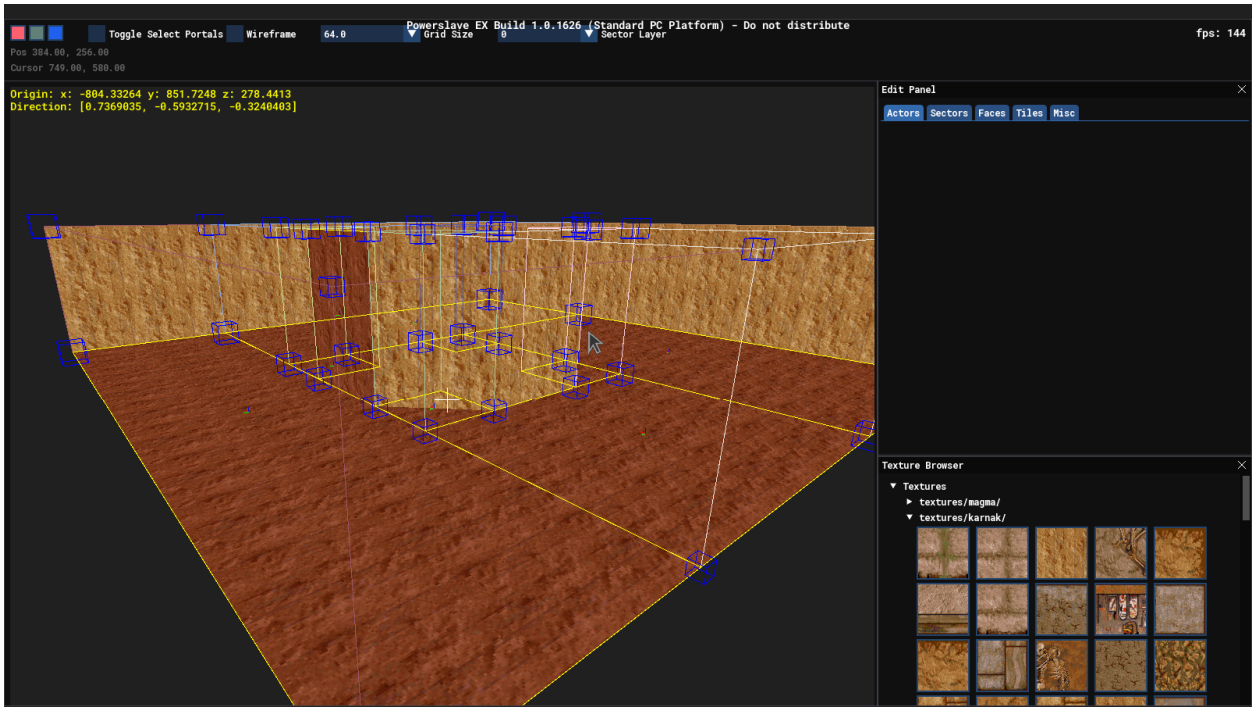
Next up, select and delete the pillar sector since we want this to be solid space



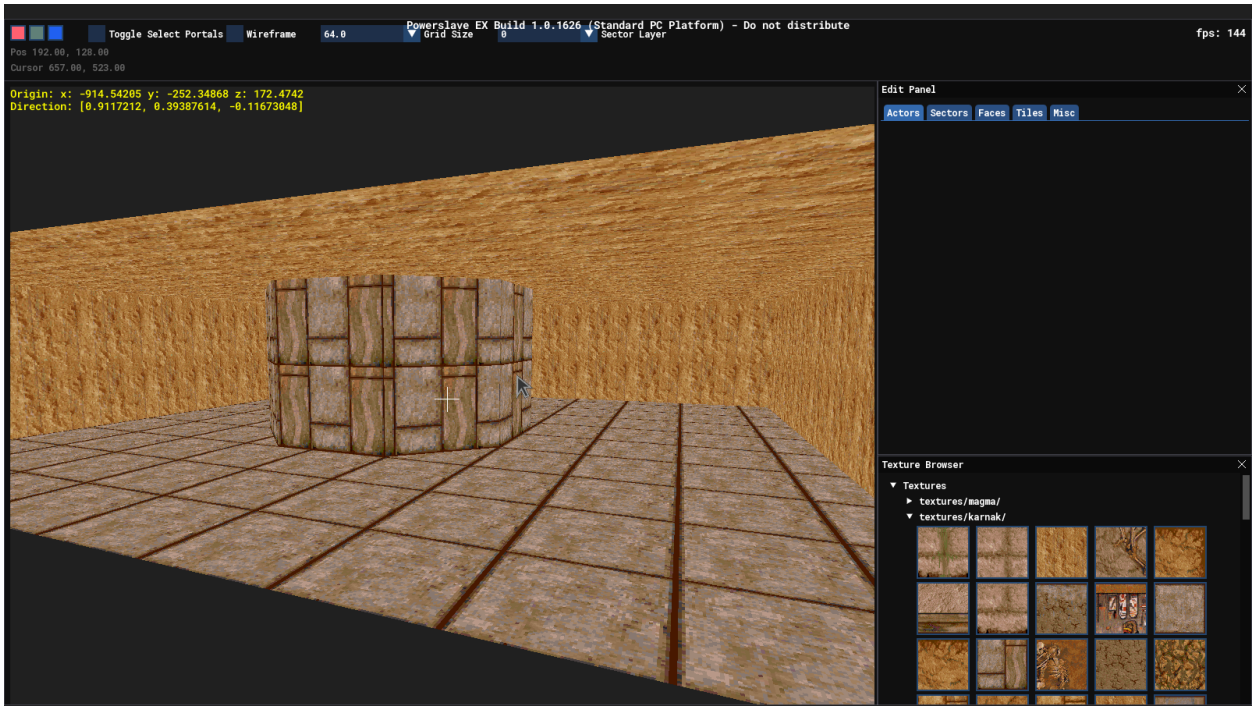
Upon doing this, the faces will **not be updated** and will look like this:



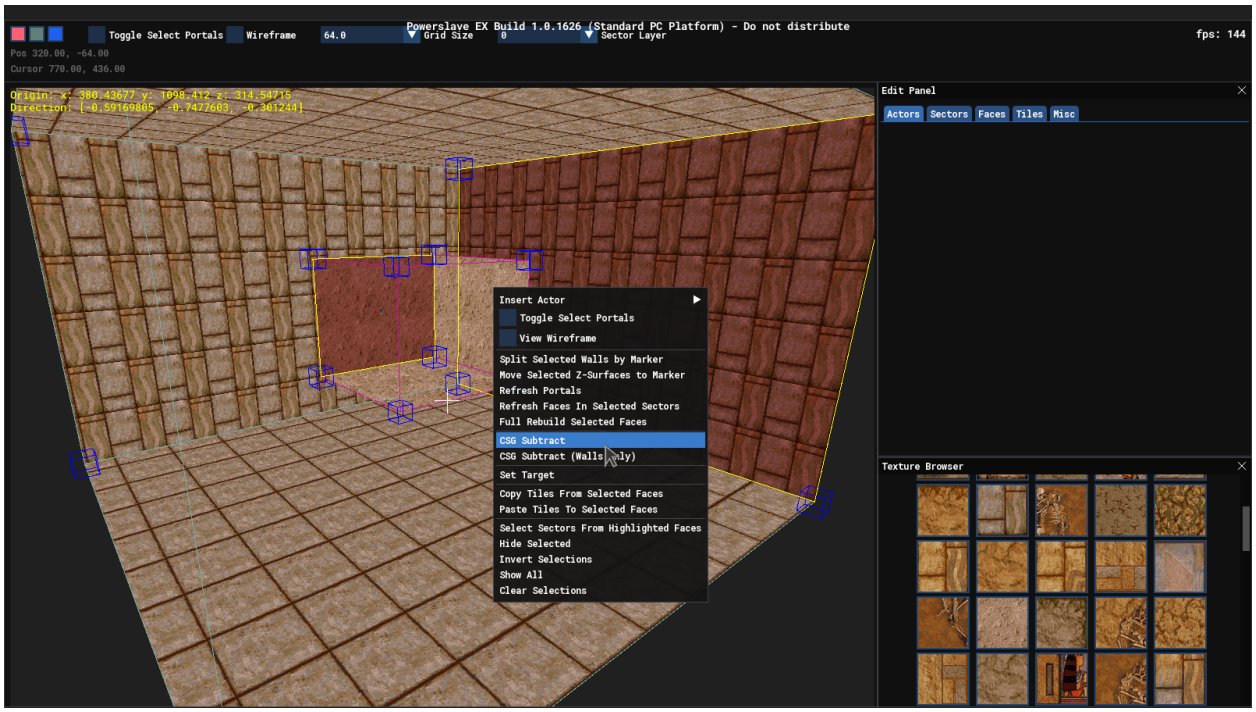
This is because the portal faces that used to link into the inner sector will still have its tiles missing, but will be automatically flagged as solid if the face fails to find a neighboring sector. To quickly restore the missing tiles, select all sectors and then press **r** to force all faces to refresh. This will result in the faces missing the tiles to rebuild them.



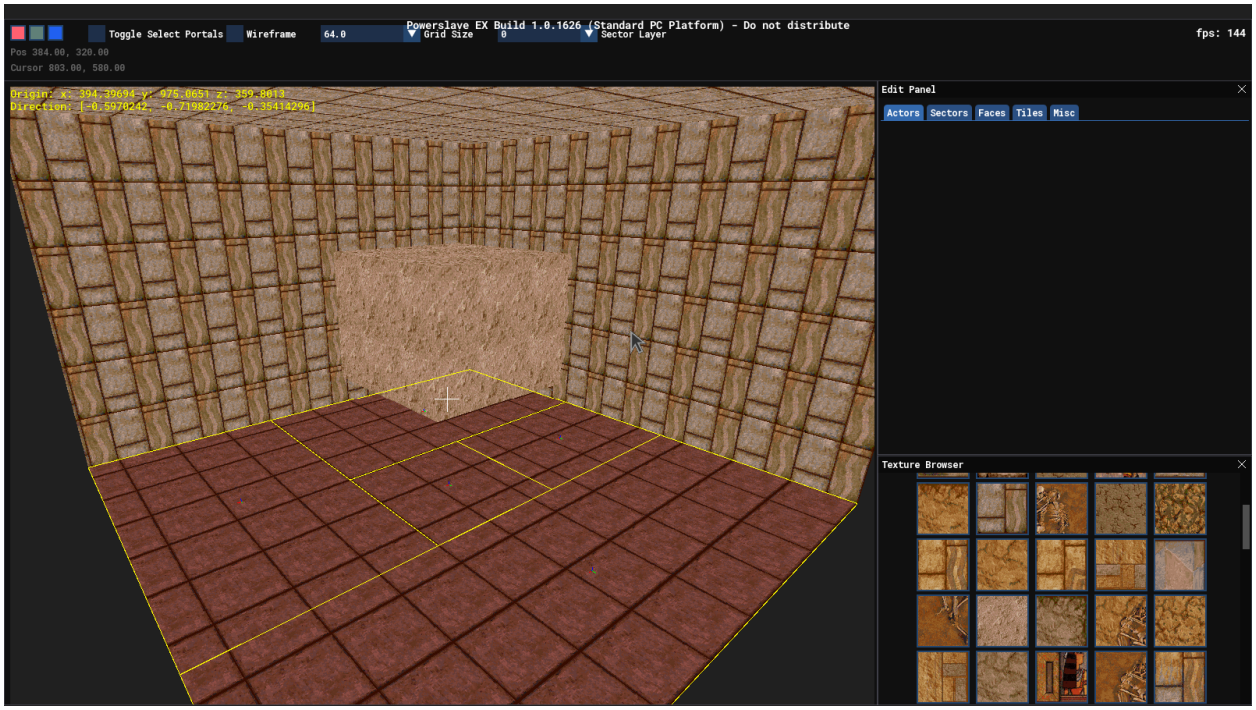
Congrats, you've added a pillar into your room.



The other option, simply called **CSG Subtract**, is used if you want to carve 3D geometry. Below is a demonstration of what it does:



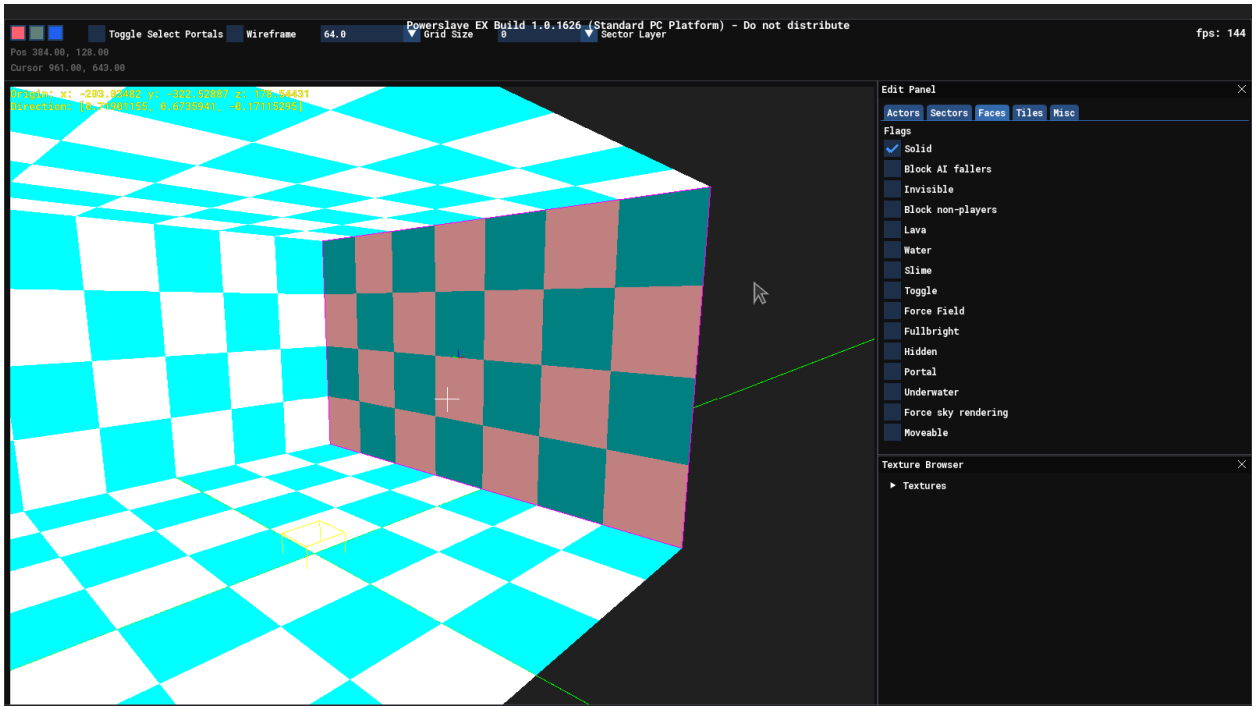
End result will look like this:



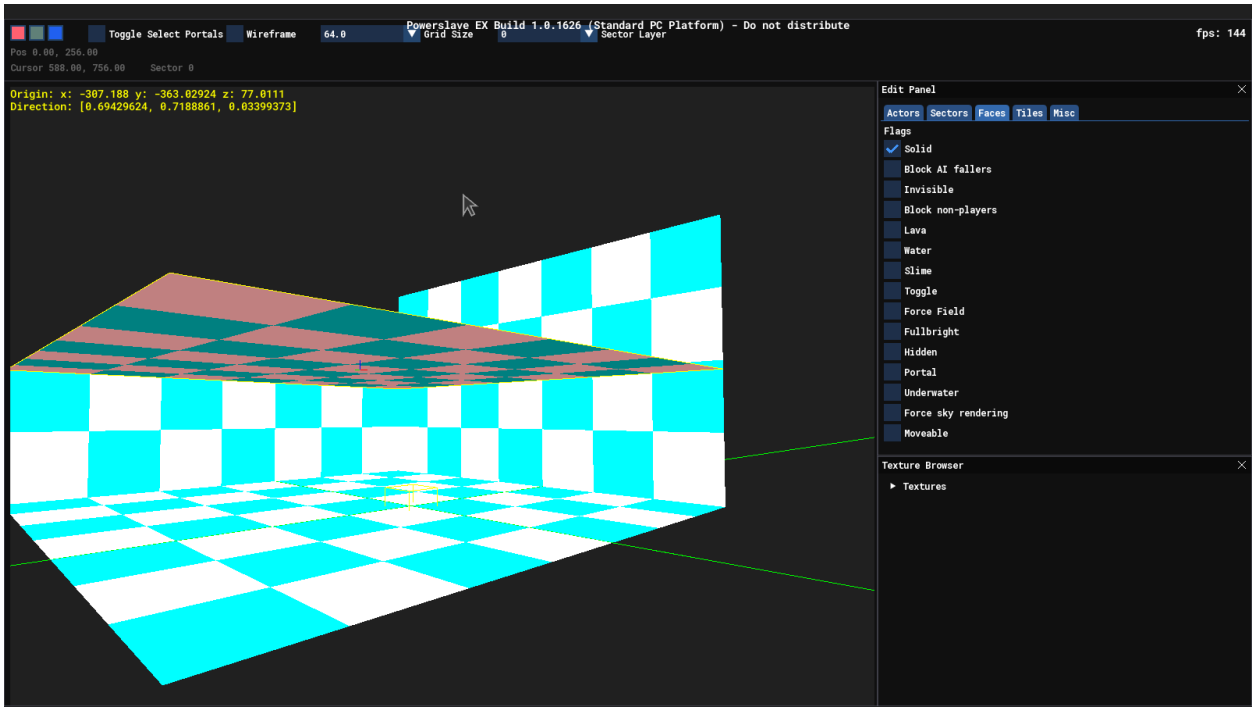
Face Locking

In 3D mode, pressing **l** on a selected face will toggle lock mode on that face. When lock mode is enabled, the face will highlight pink to indicate that it is locked. While locked, the face **will never update** when changing the sector. This is usually used in conjunction with creating doors or lifts.

Screenshot shows a locked face

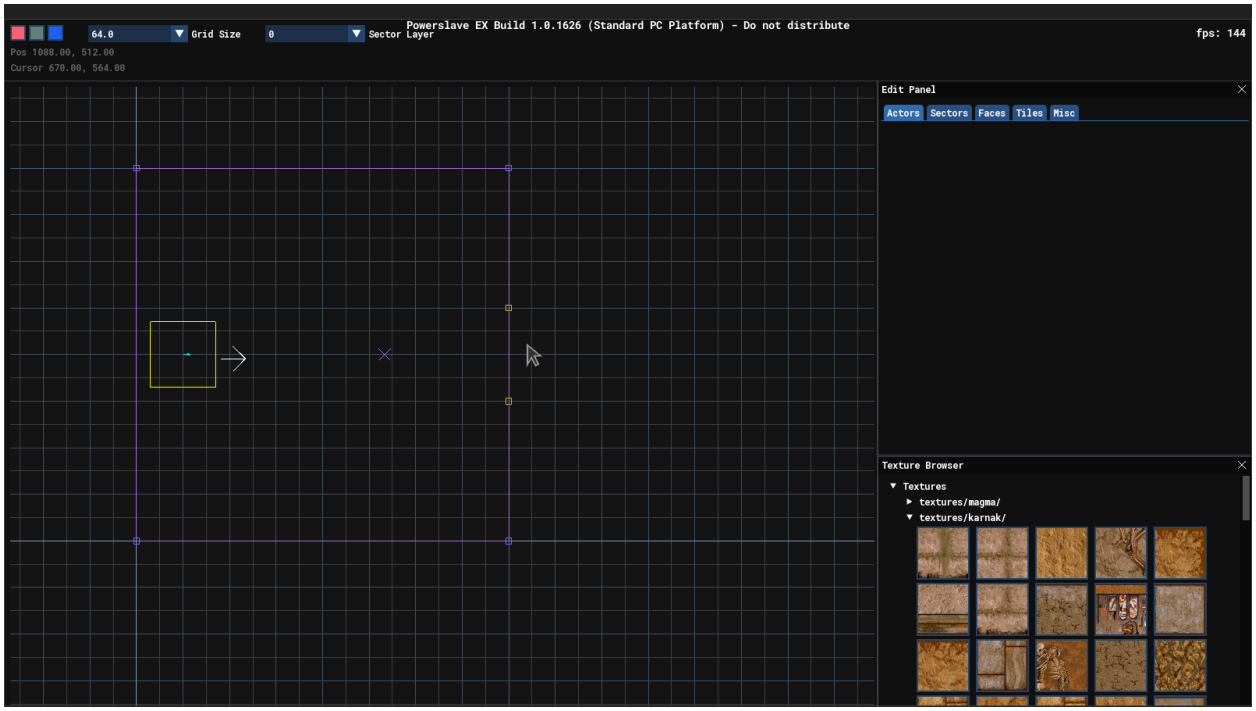


While locked, changing the sector will have no effect on the locked face.

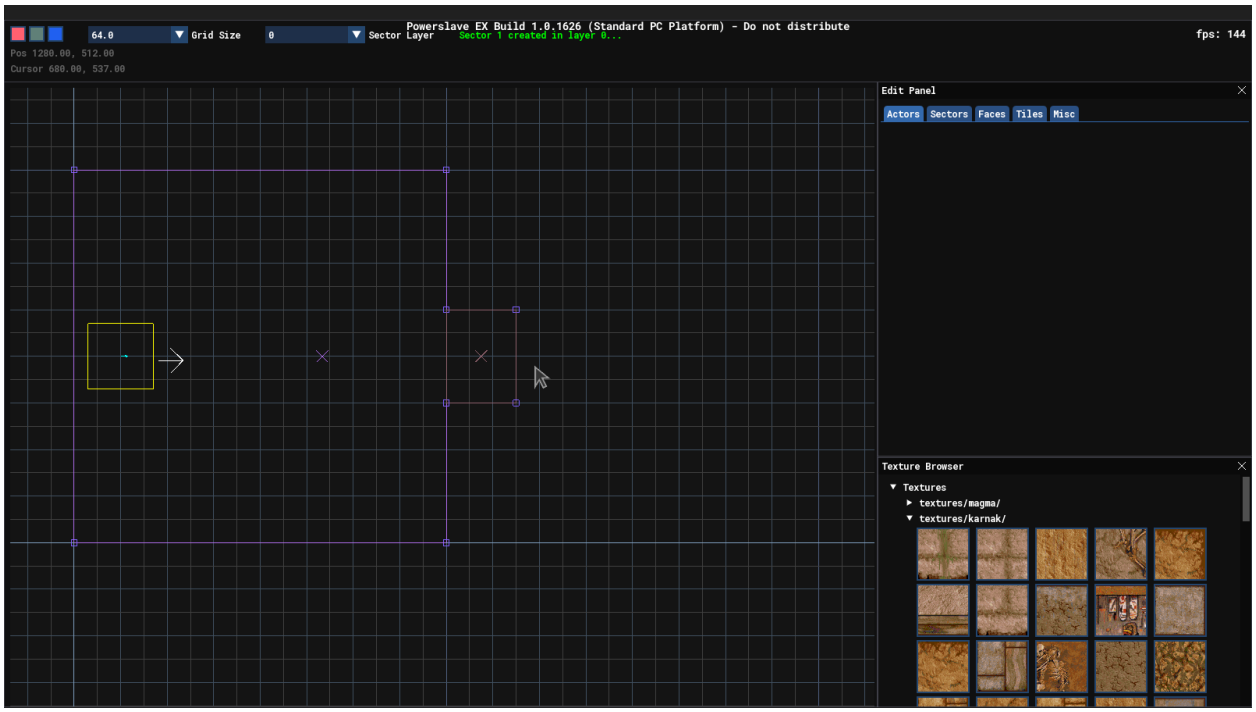


How to make a door

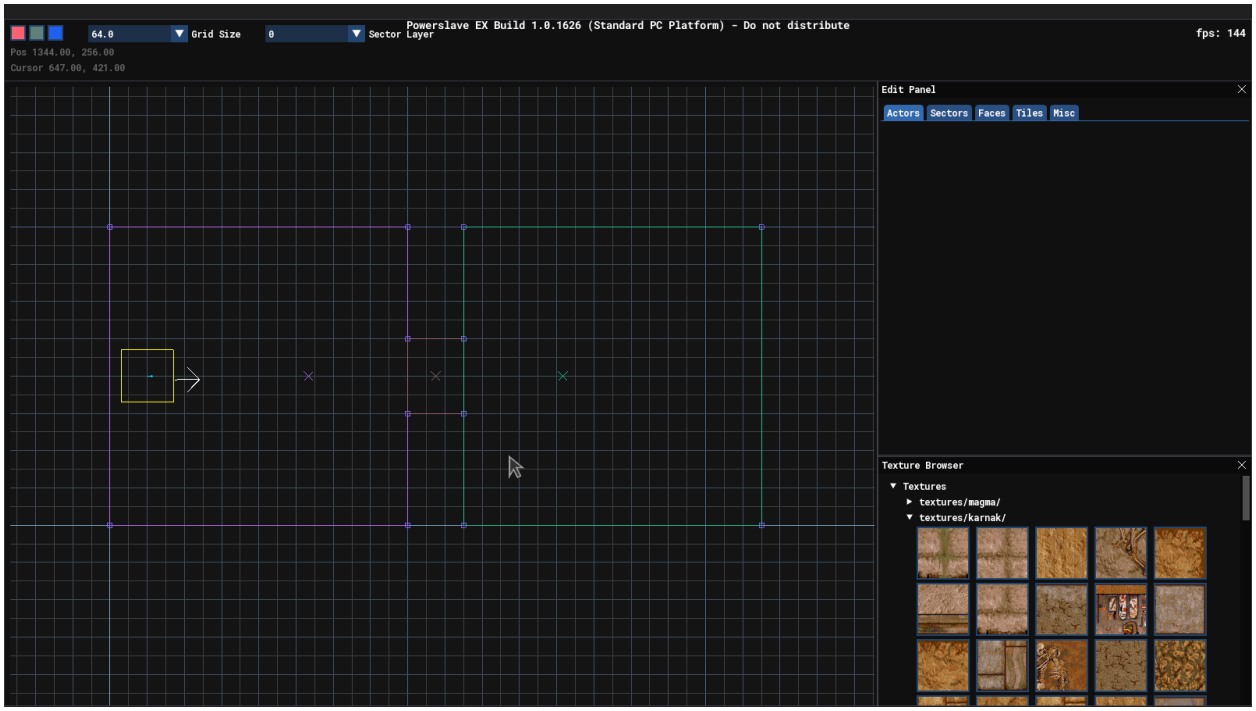
Create a 1024x1024 room. At one end of the room, split the walls with two vertices as shown



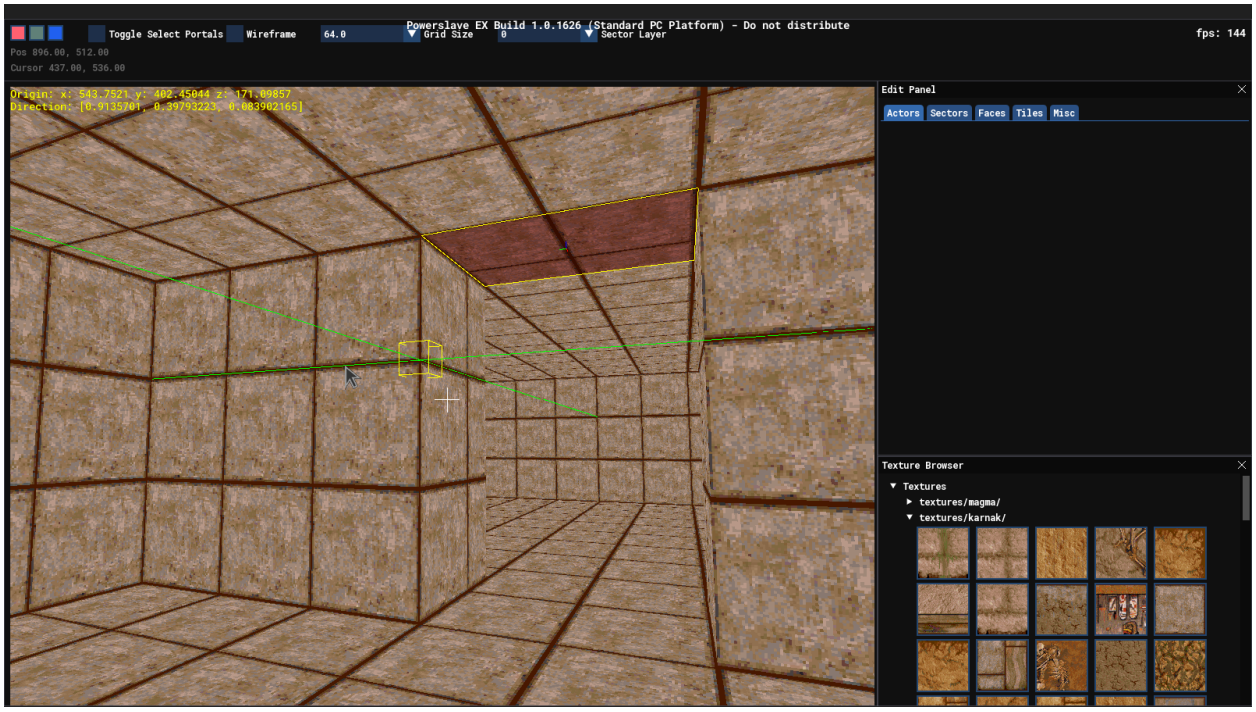
Next create a 192x256 sector that connects to the split wall as shown

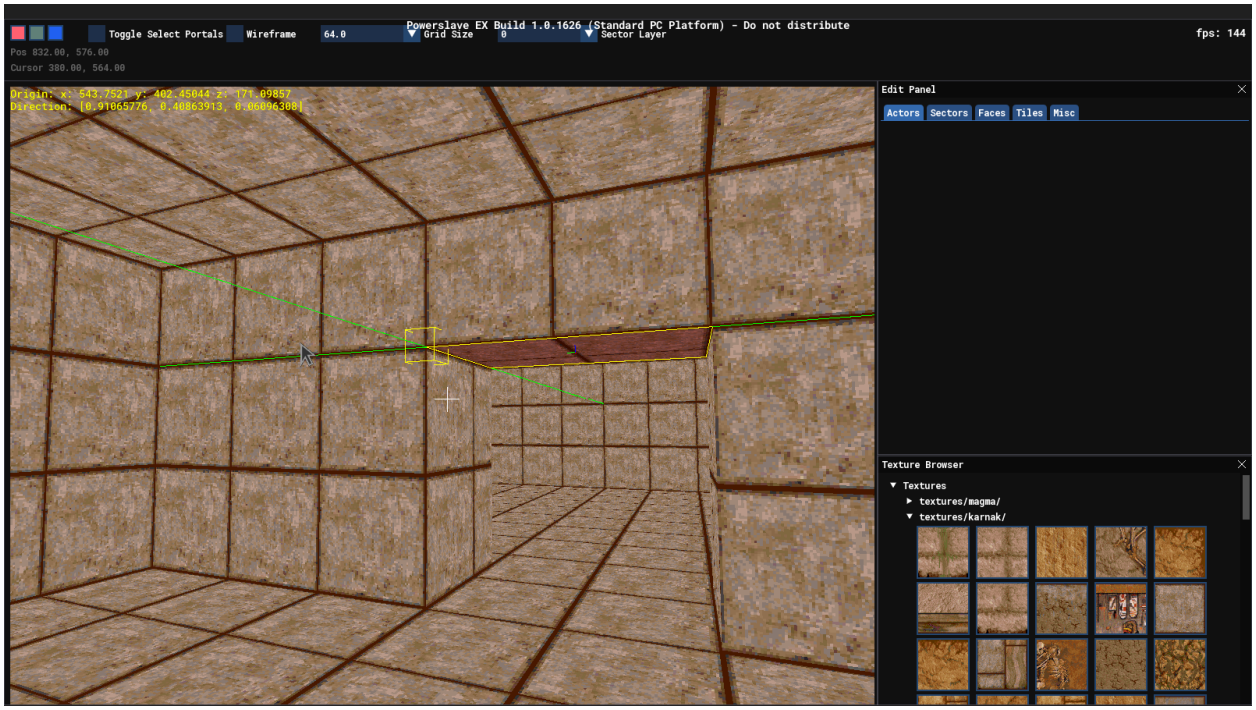


Create another room on the other side

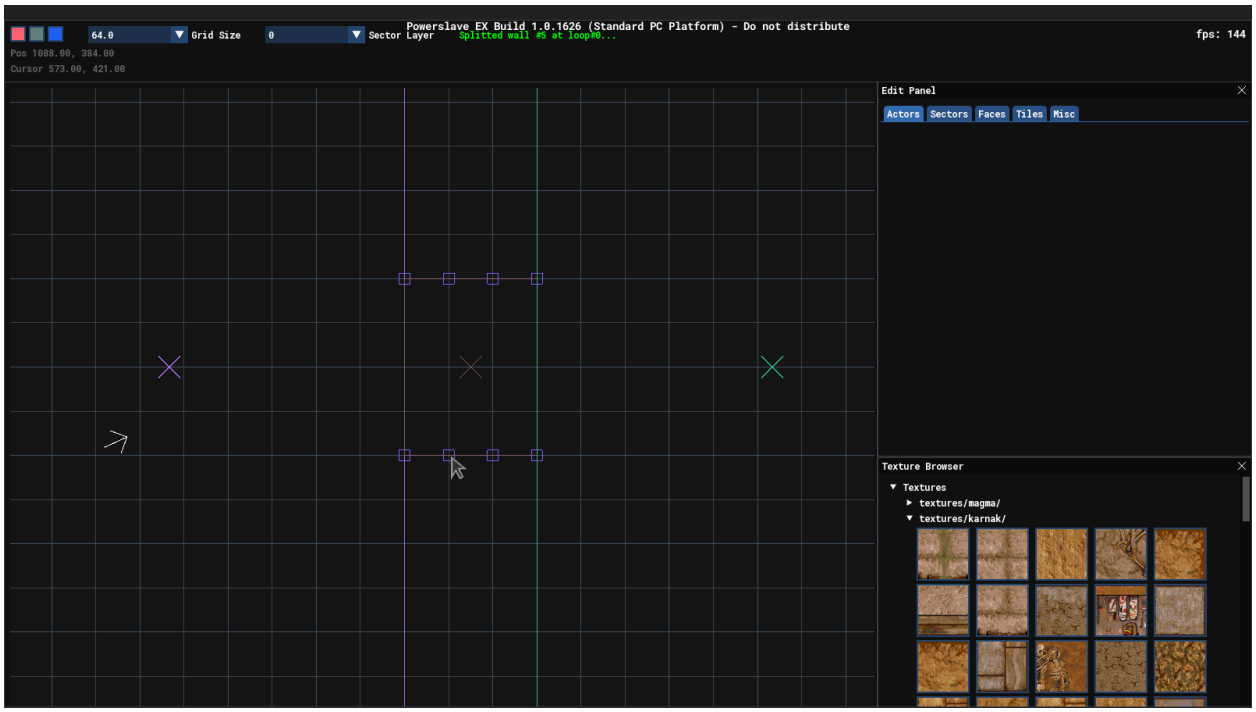


In 3D mode, select the mid sector and lower its ceiling 128 units (1 tile worth)

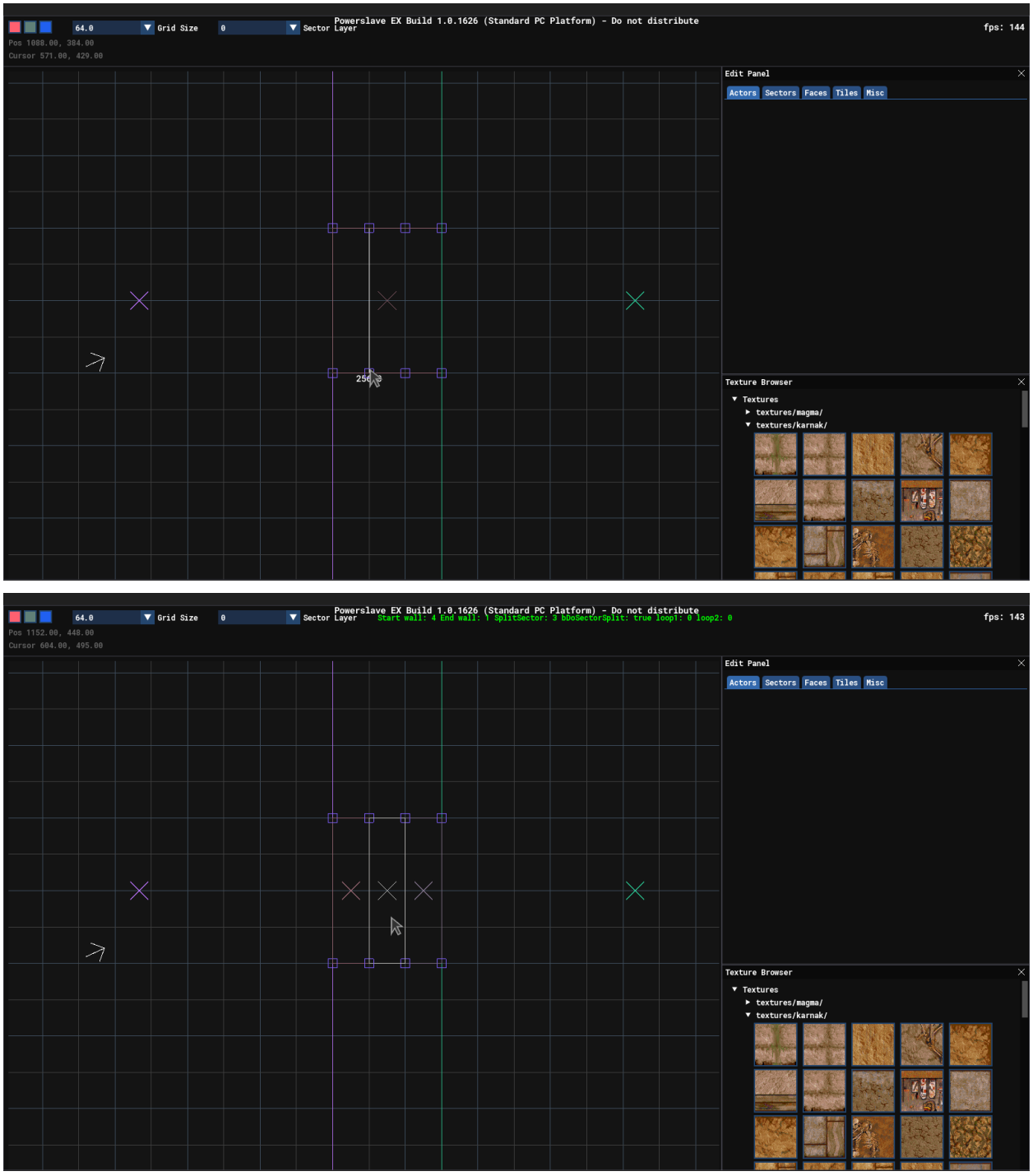




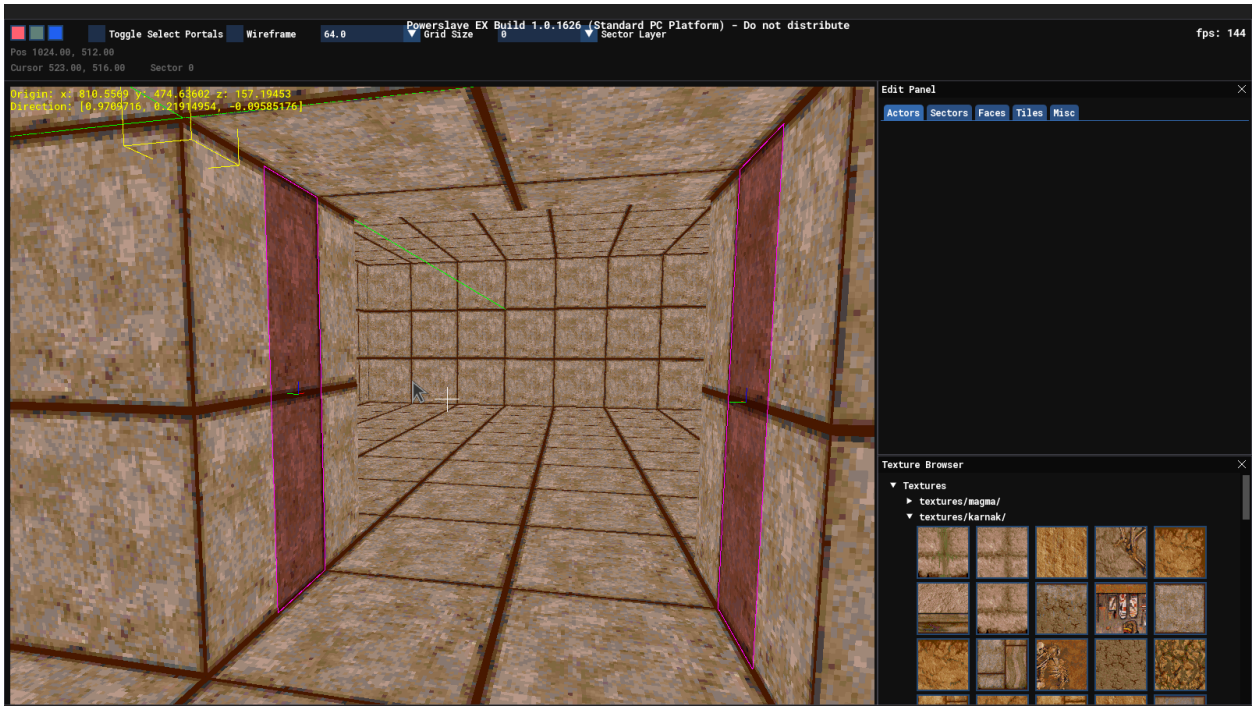
Next we will be creating the sector that makes up the door itself. Inside the mid sector, insert two vertices in the center of each wall as shown



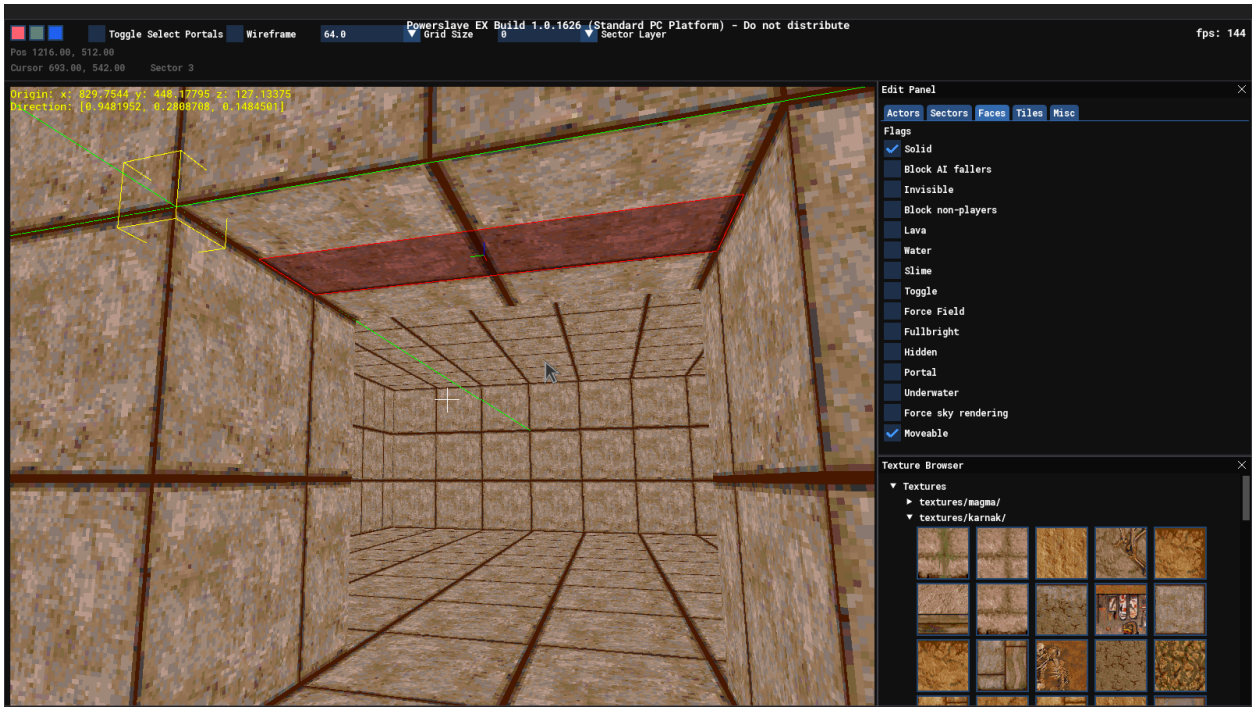
Then enter draw mode and draw a single line from each inserted vertex to split into 3 sectors



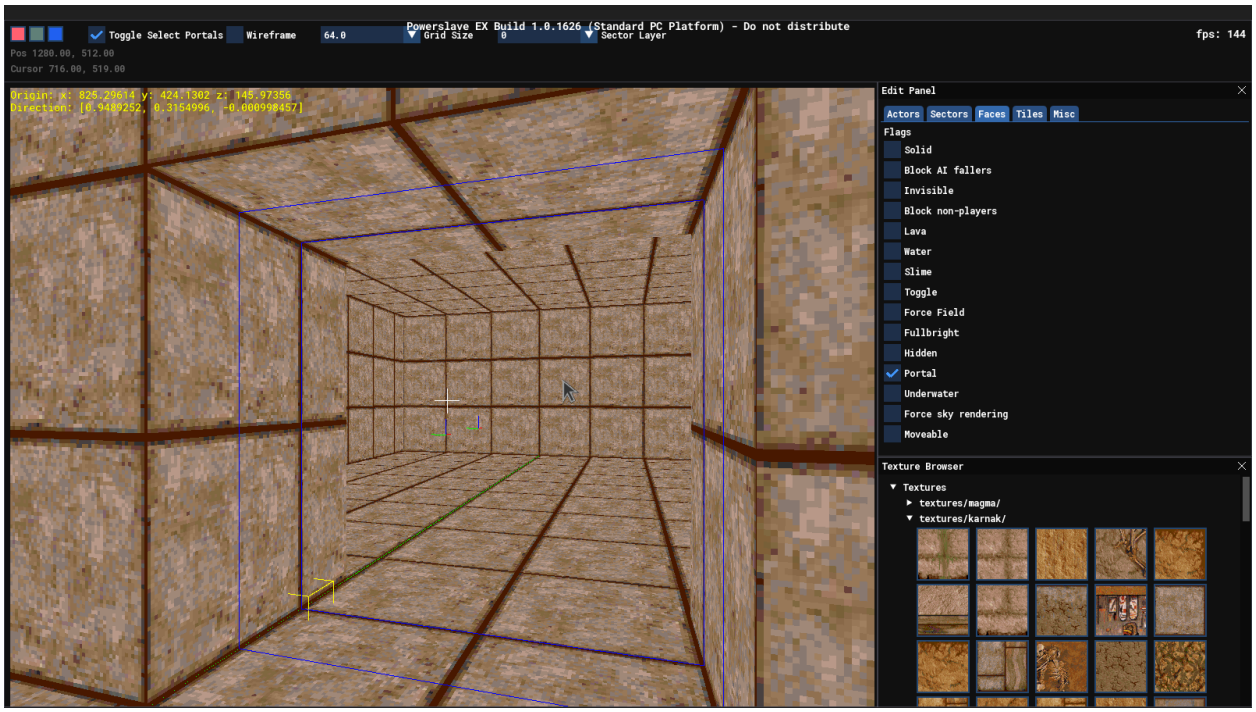
Next enter 3D mode and select both sides of the door sector **and lock them**.

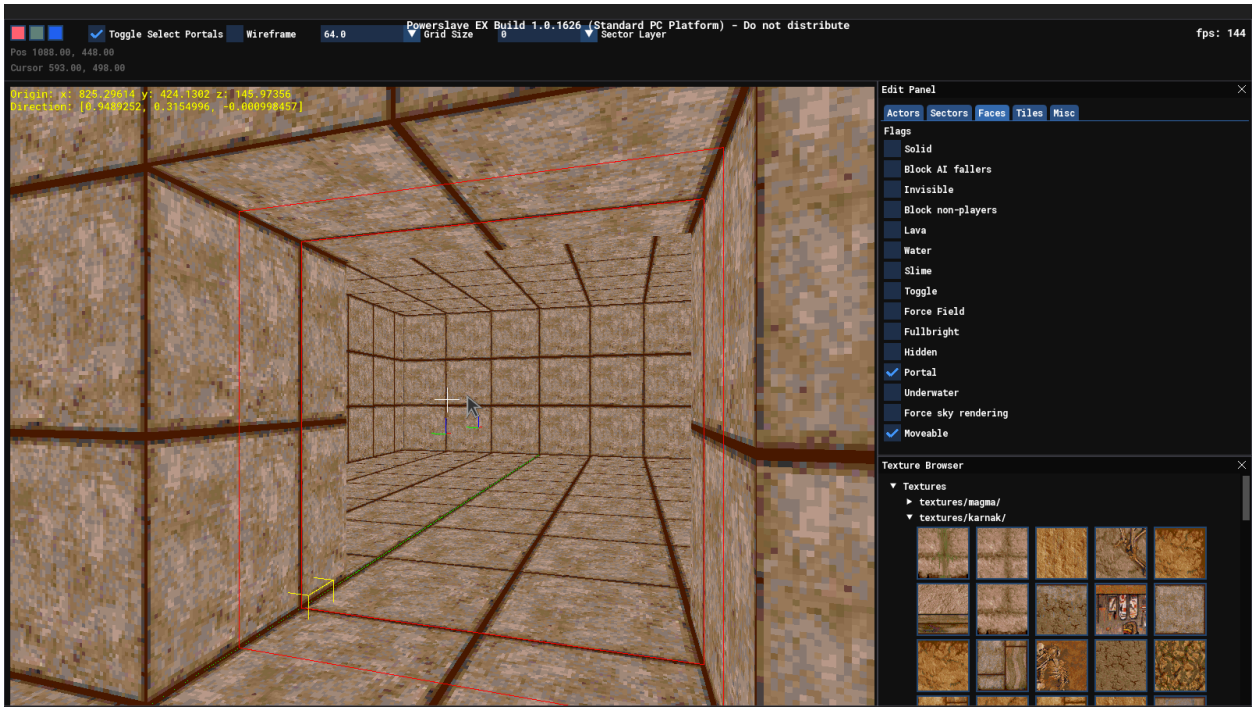


Select the ceiling of the door sector and flag as **moveable**

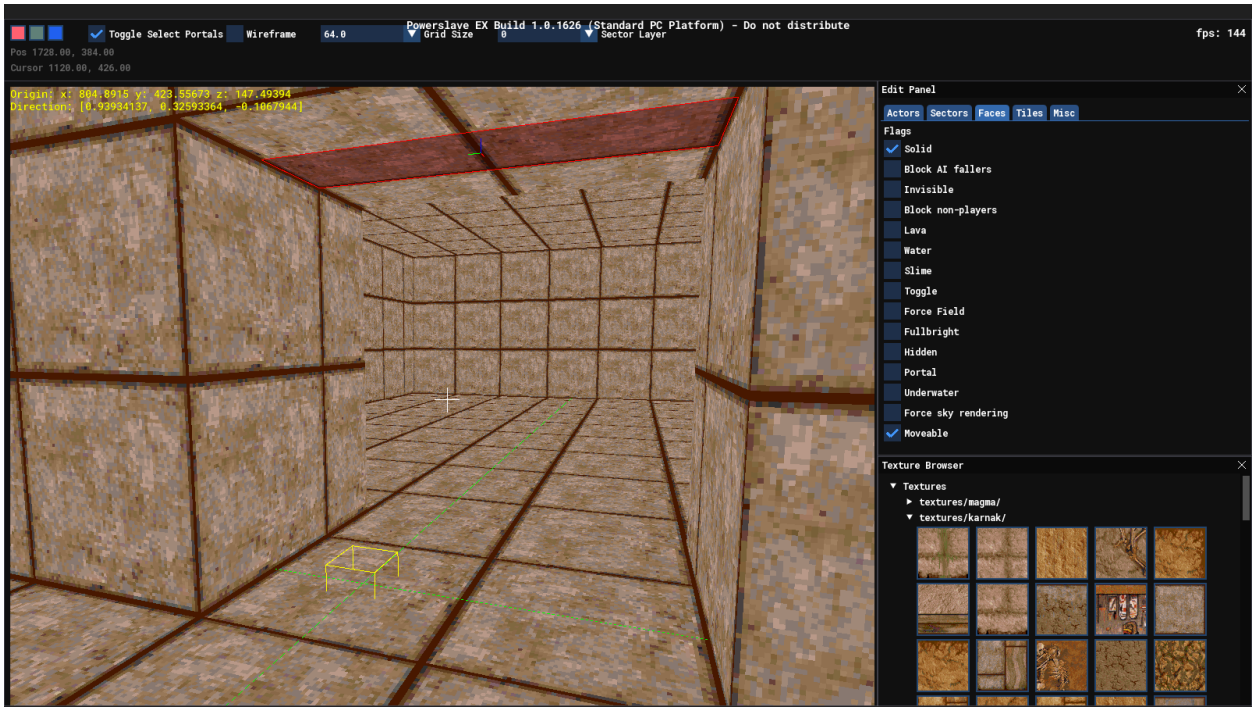


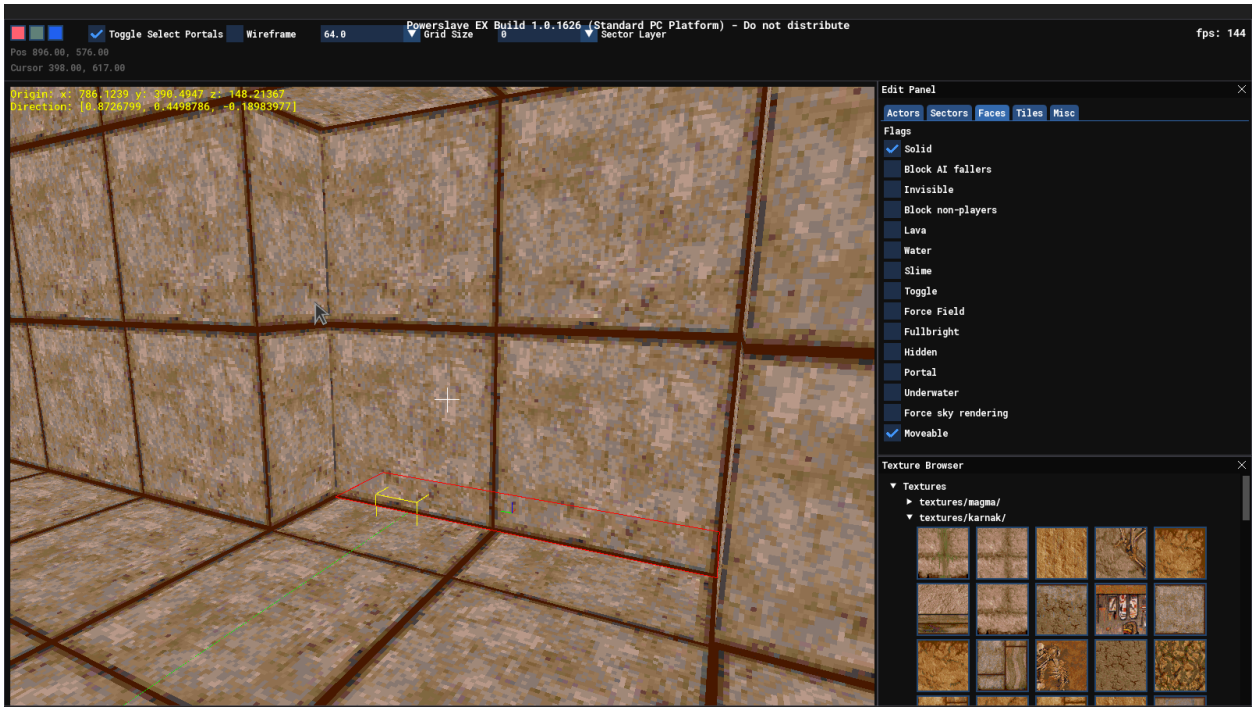
The next step might be tricky to get, but you must select **both sides** of the portal faces that connect the door sector and flag them as **moveable.** Don’t forget to do the same for the portal faces inside the door sector that connects to the outer sectors.



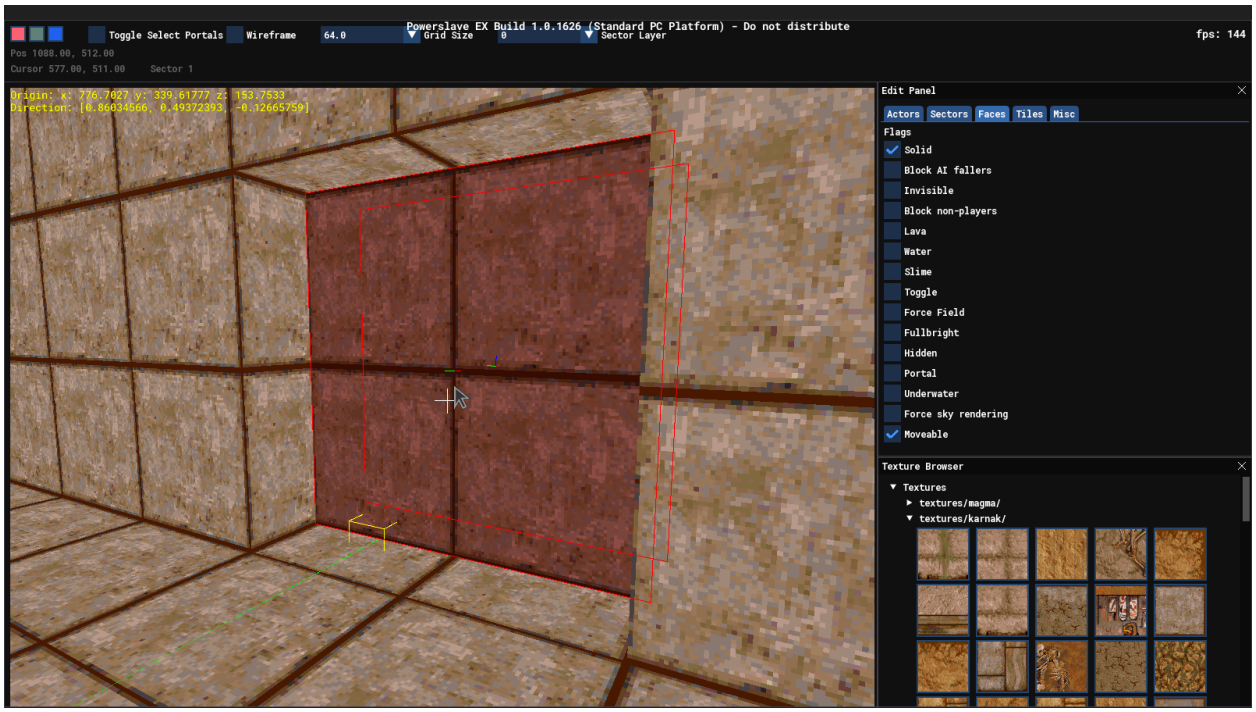


Press **C** to clear selections, then select the ceiling of the door sector and lower it to the floor with the **mouse wheel**

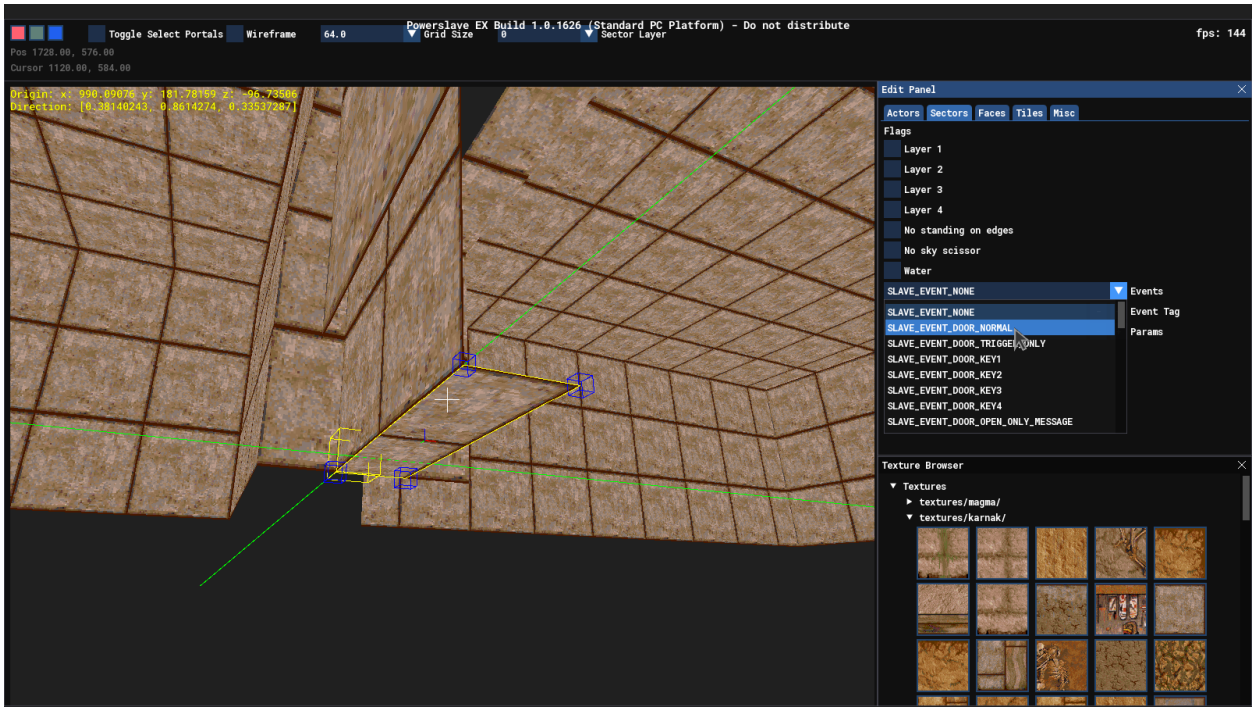




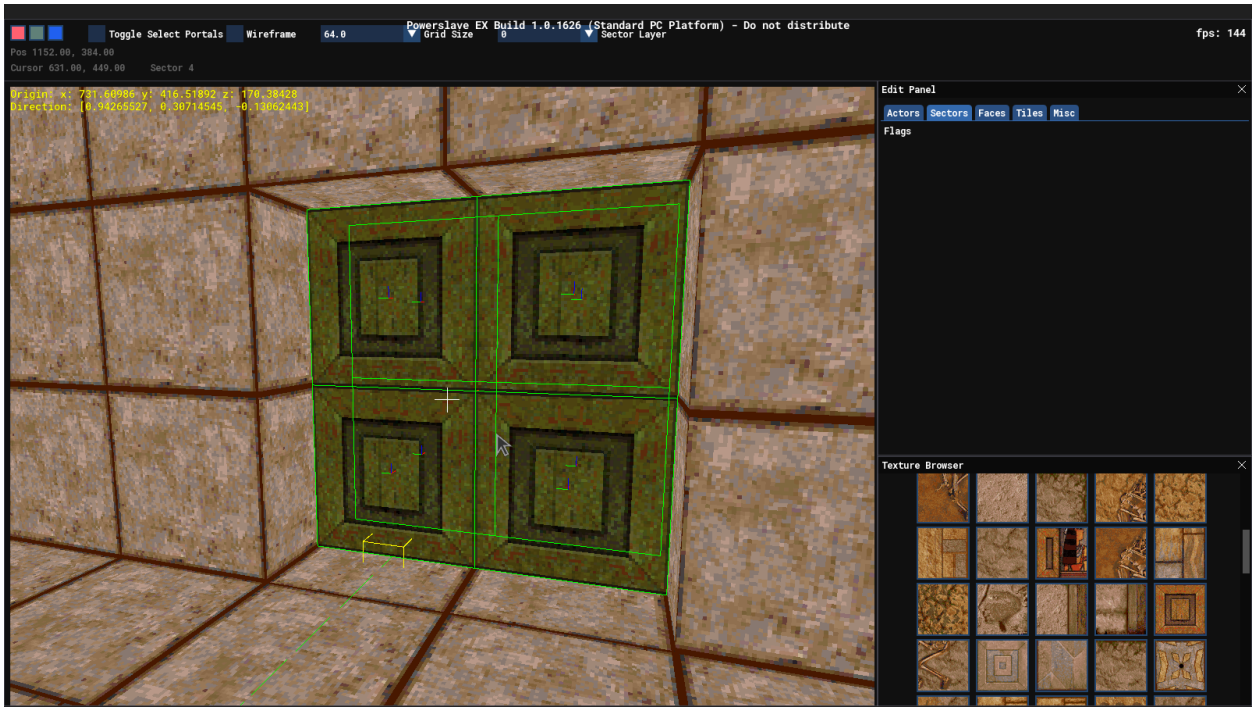
Then select the outer faces of the door sector and flag them as **moveable.** Press **C** to clear selections.



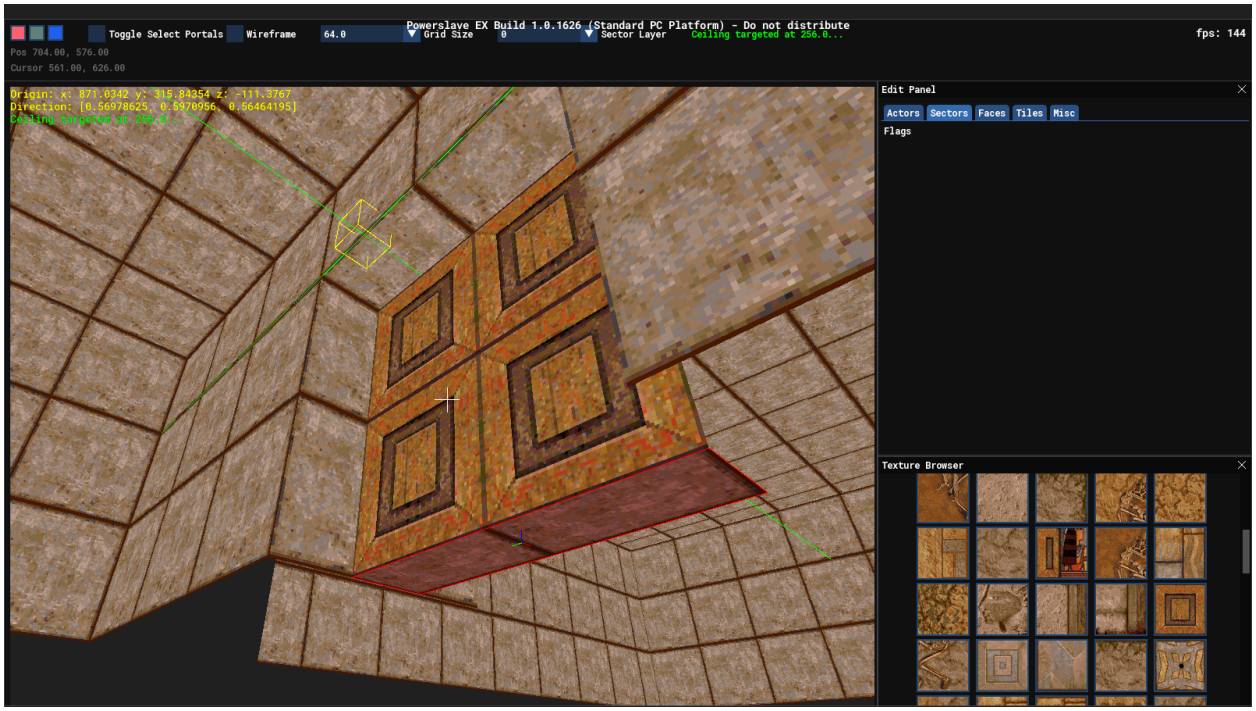
Next select the door sector by selecting the ceiling face and then pressing **space**. In the **Edit Panel**, select the **Sectors Tab** and under **Events** select **SLAVE_EVENT_DOOR_NORMAL**



Next texture the outer faces of the door to your preference. When done, press **C** to clear selections.



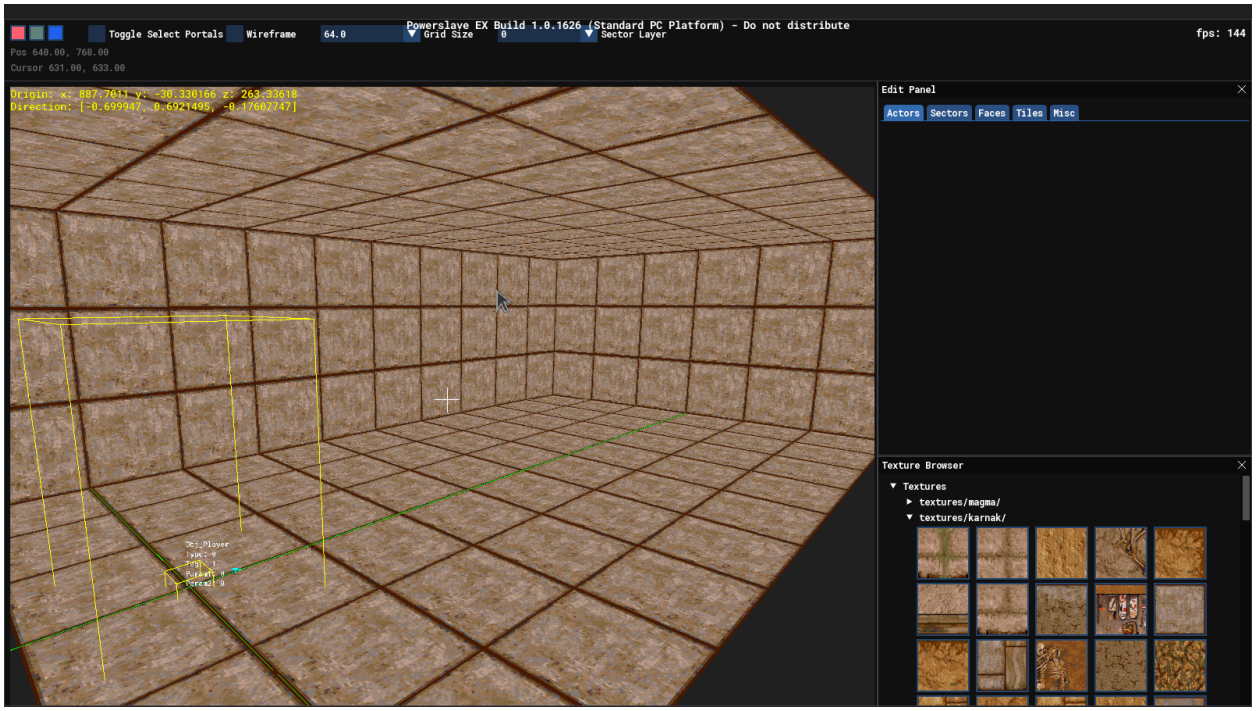
Select the ceiling face of the door sector again and press **space** to select the sector and then place your marker **where you want the door to raise to** and press **T** to target at the designated height. Press **C** to clear selections when done.



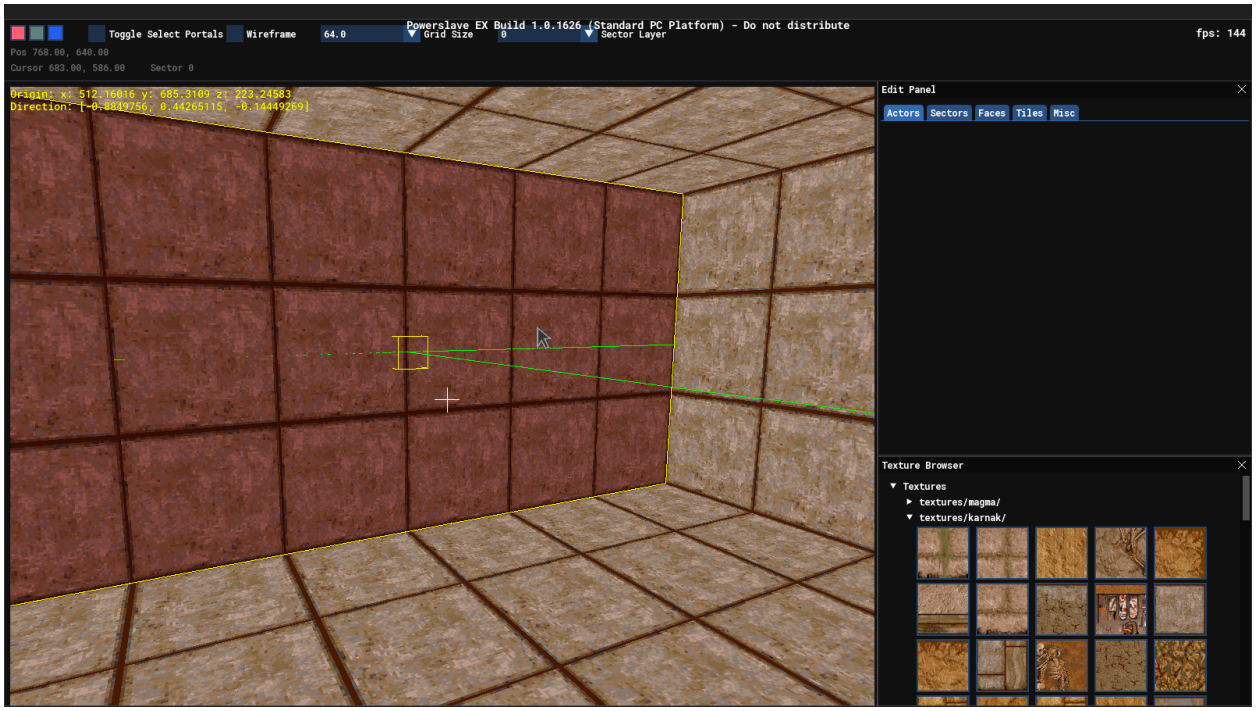
Finally, save your map and launch. If all done correctly, pressing the use button on the door will cause it to raise, wait then lower.

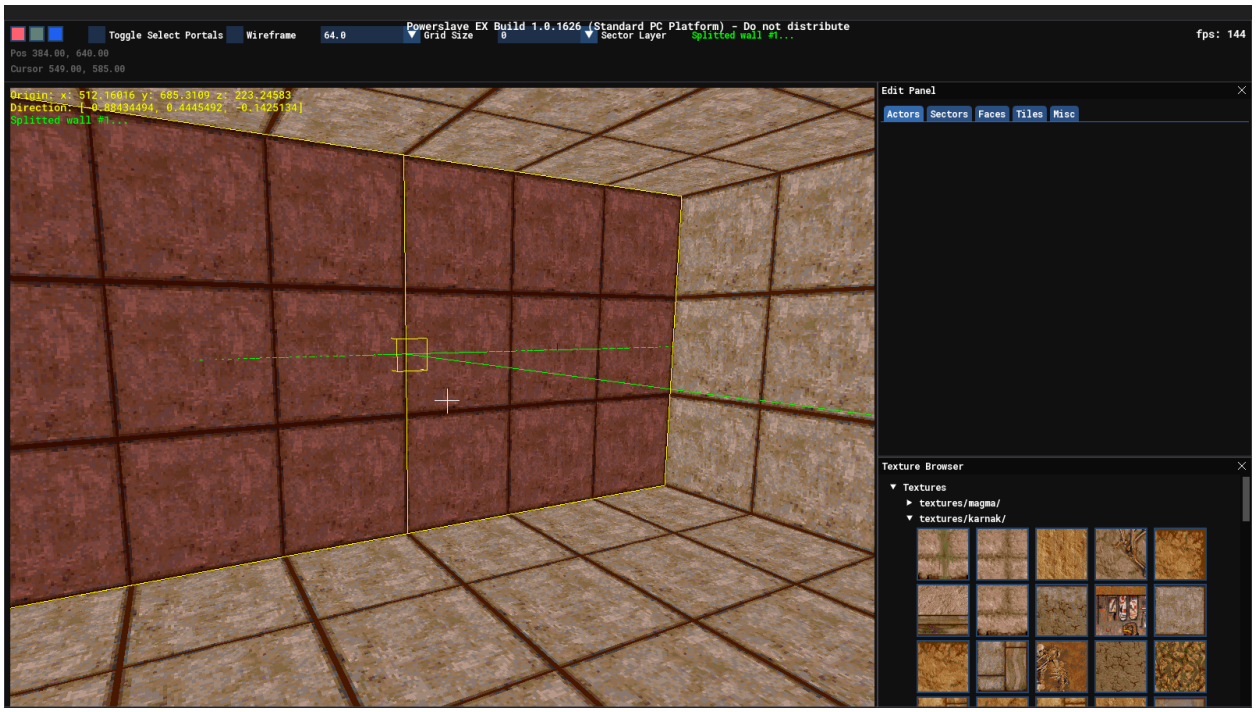
Creating projectile traps

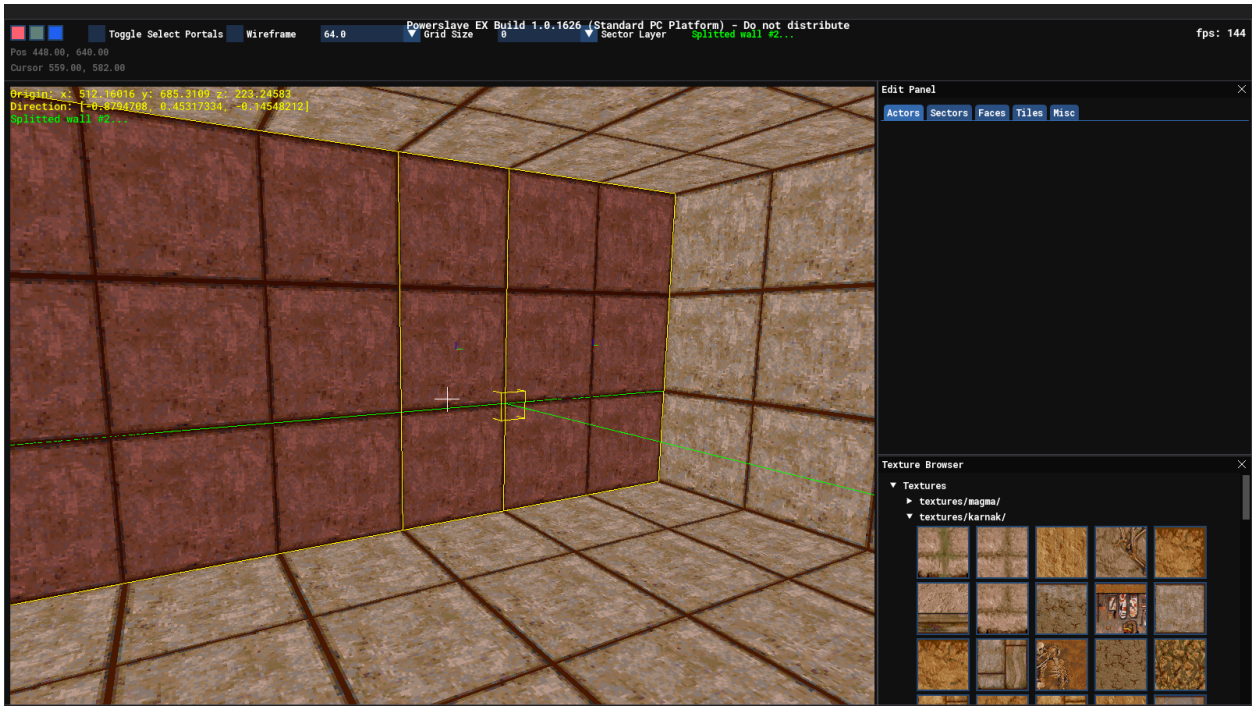
Create a room as shown



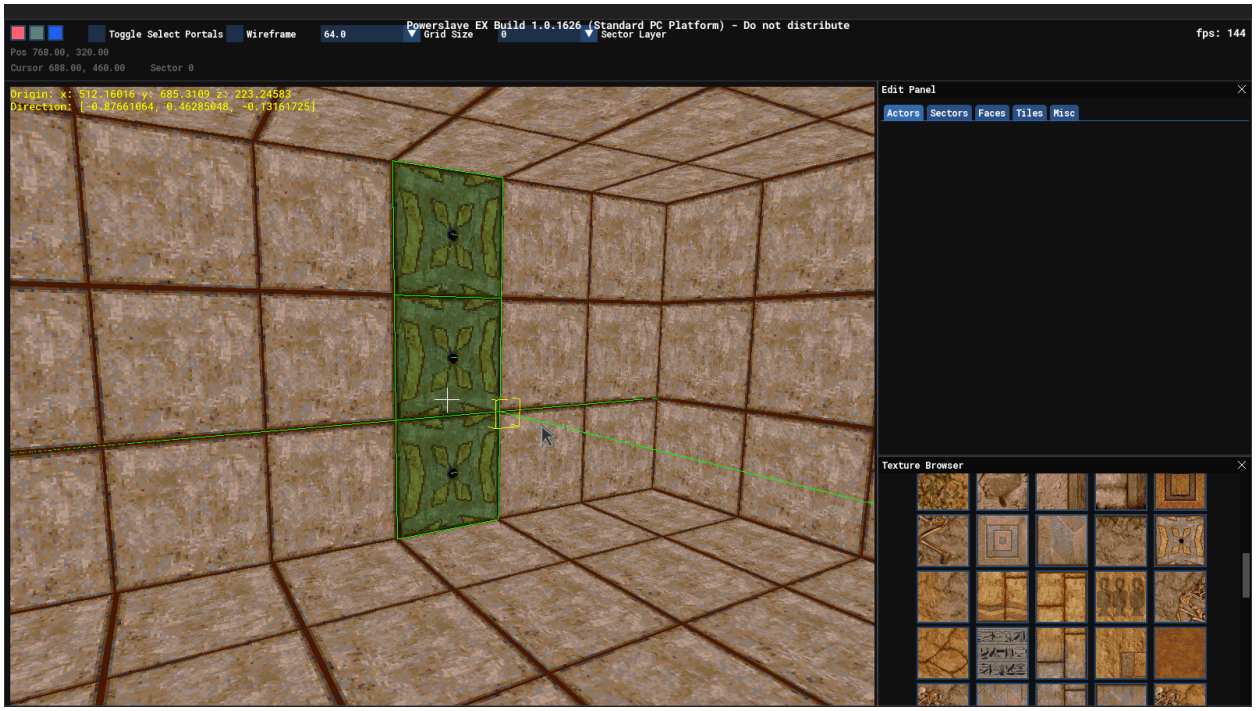
Next we will be creating a face that will make up a spawning point for the projectiles to come from. Split up the wall as shown



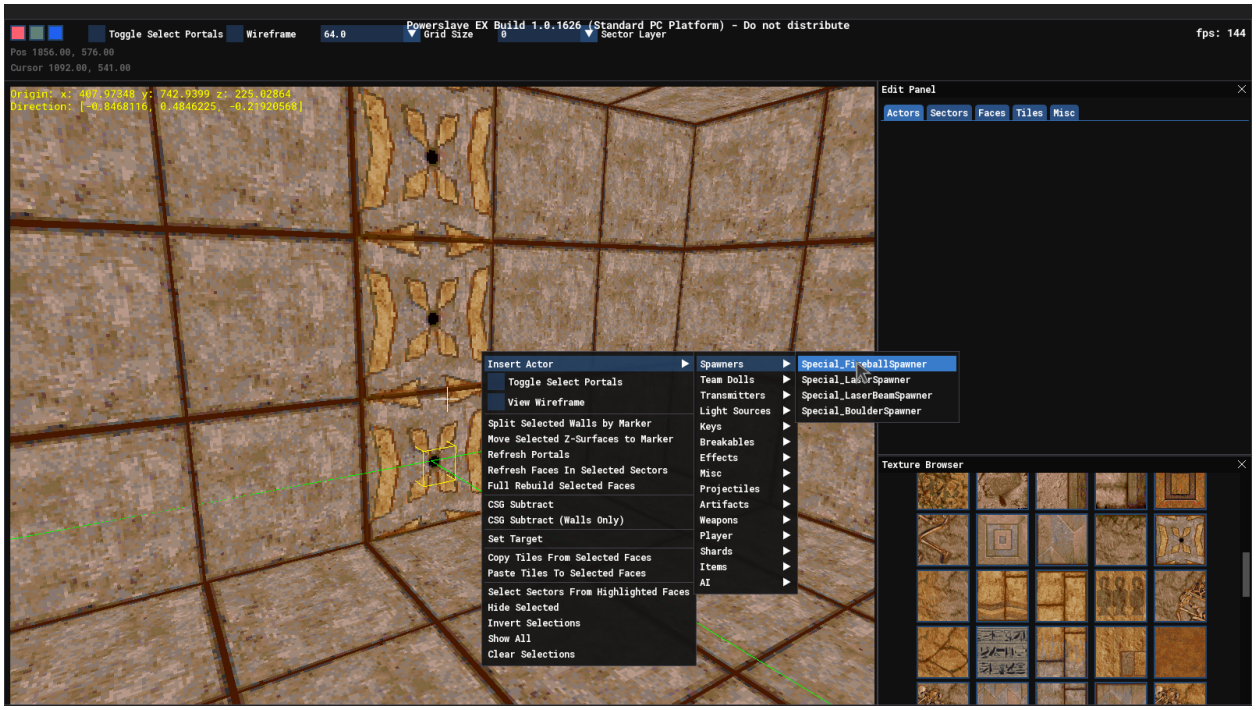




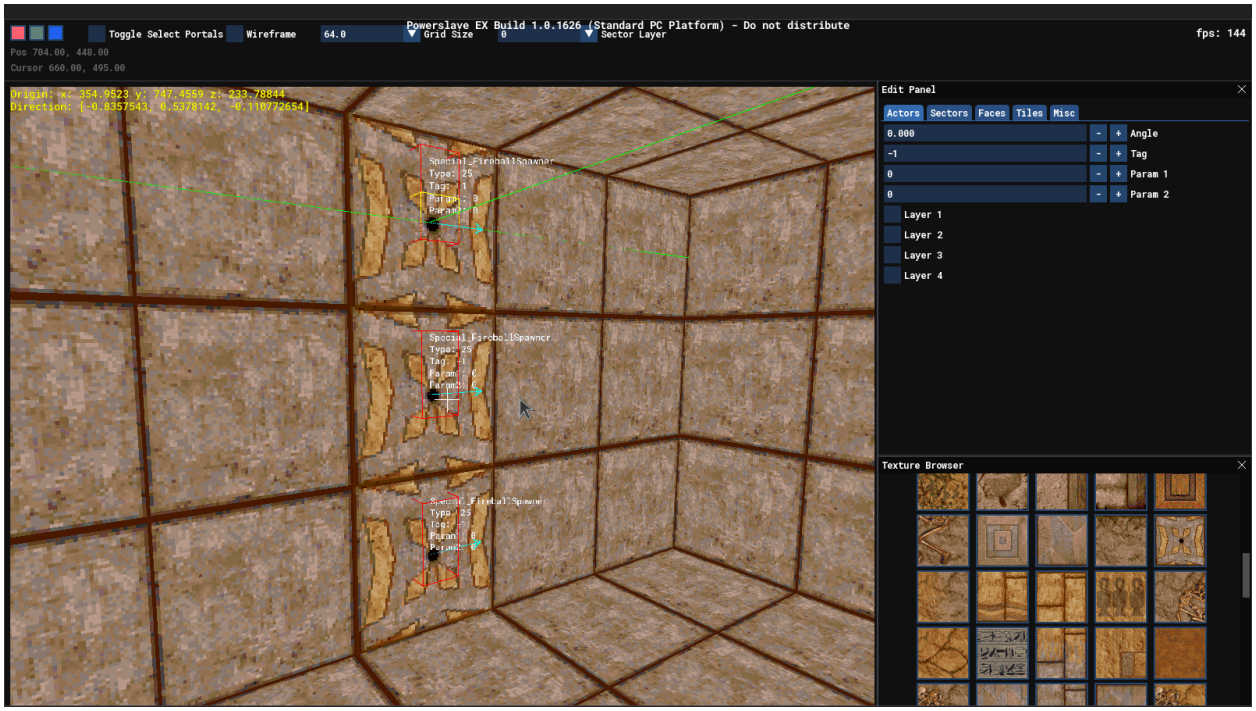
Then give this face a unique texture



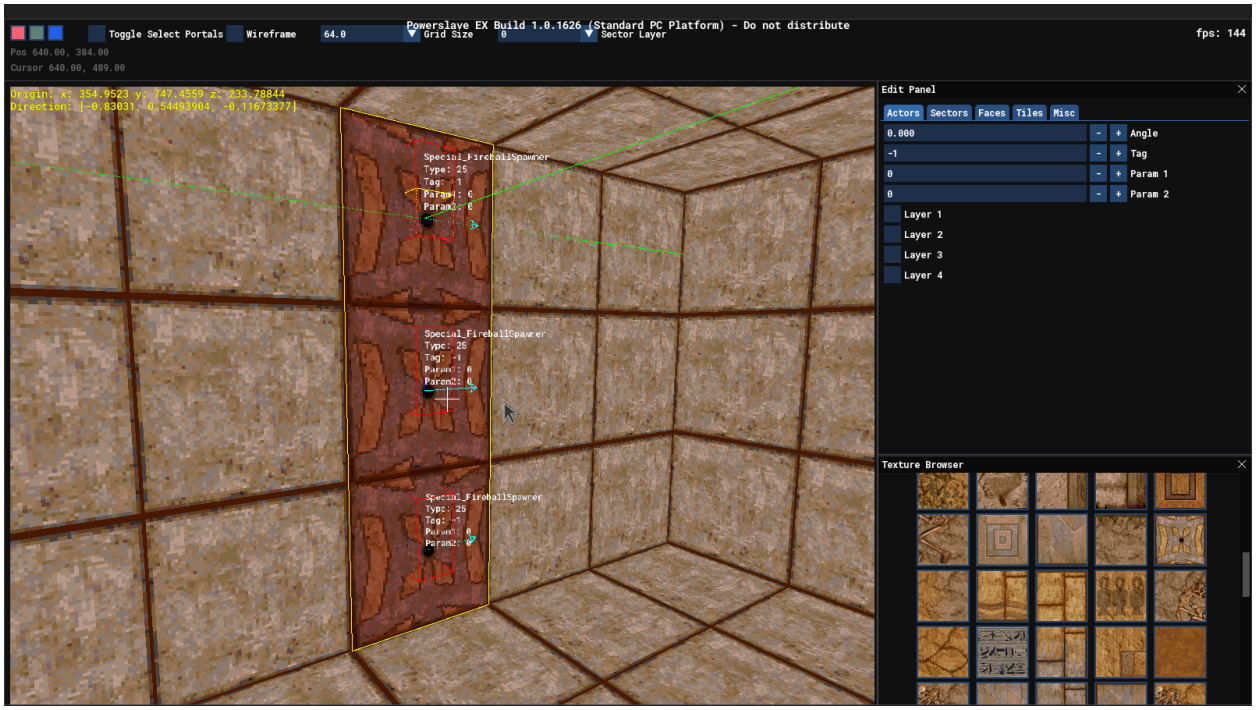
Next, we will be placing three projectile spawners at the center of each tile. Place the marker at the center of one of the tiles and exit **camera active mode**. Right click and then select **Insert Actor->Spawners->Special_FireballSpawner**



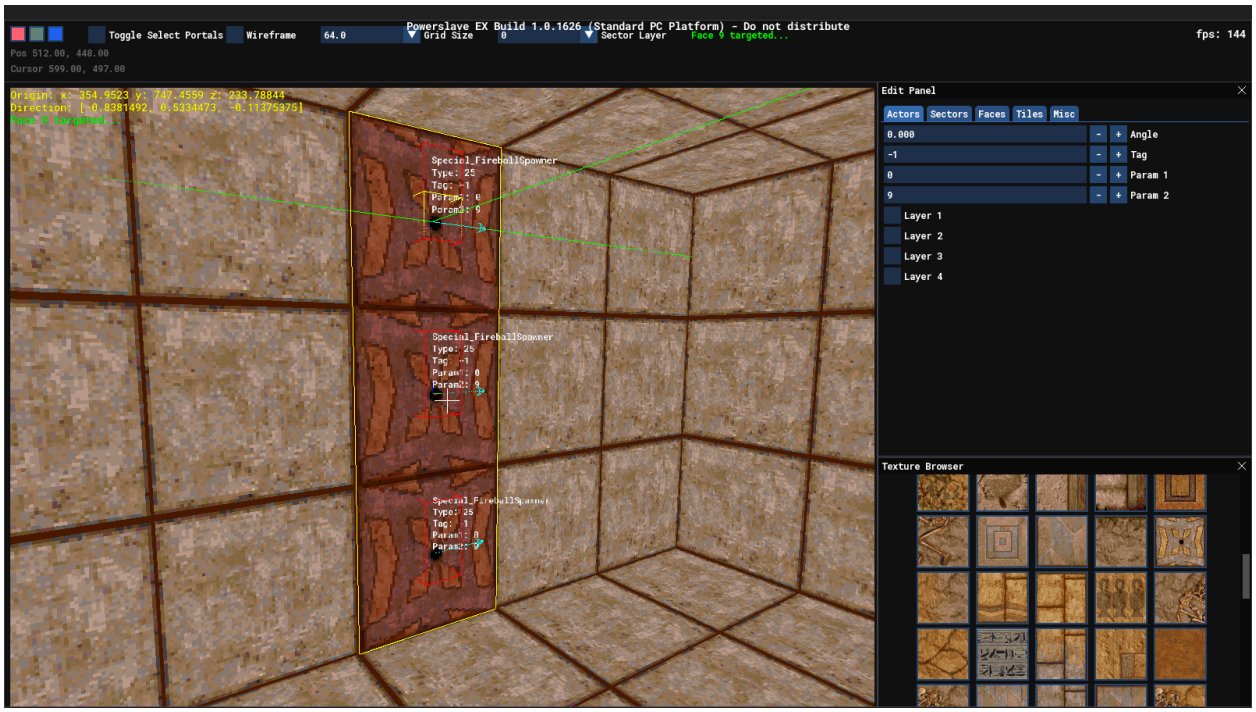
If you want, place a few more, preferably placing one at the center of each tile of the face.



Next, select the spawner actors and then while still selected, select the face.

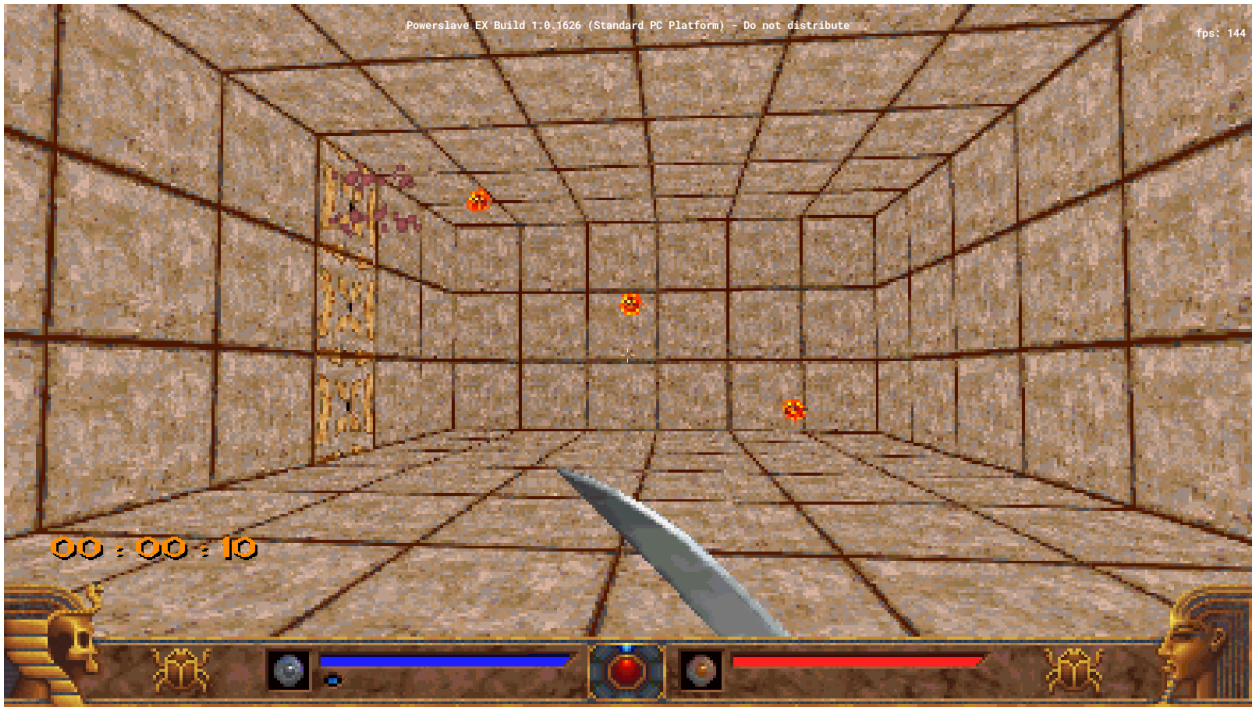


Press **T** to have the spawners target the face. This will set up the spawners to fire projectiles in the direction of the face.



Press **C** to clear selections. Then select the spawner actors again and in the **Edit Panel** set **Param 1** to **-1.** This field controls the initial state of the spawners. A value of **-1** will cause them to immediately begin firing projectiles when the level has loaded. Any other value will cause the

spawners to idle until they are triggered. Next save your map and load it in the game. If done right, the projectiles will begin firing. Also note that the order in which you placed the actors, will affect the sequential order that the projectiles will fire in.



In regards to **Special_LaserBeamSpawner**, it has special spawn parameters that are different compared to the other spawners. Below is a chart indicating what action it will perform depending on what value is set for **Param 1**.

0	Start in its ON state. Ignores triggers
-1	Start in its ON state. When triggered, it will turn OFF, wait a few seconds then turn back ON
-3	Start in its OFF state
-4	Start in its ON state. When triggered, it will turn OFF

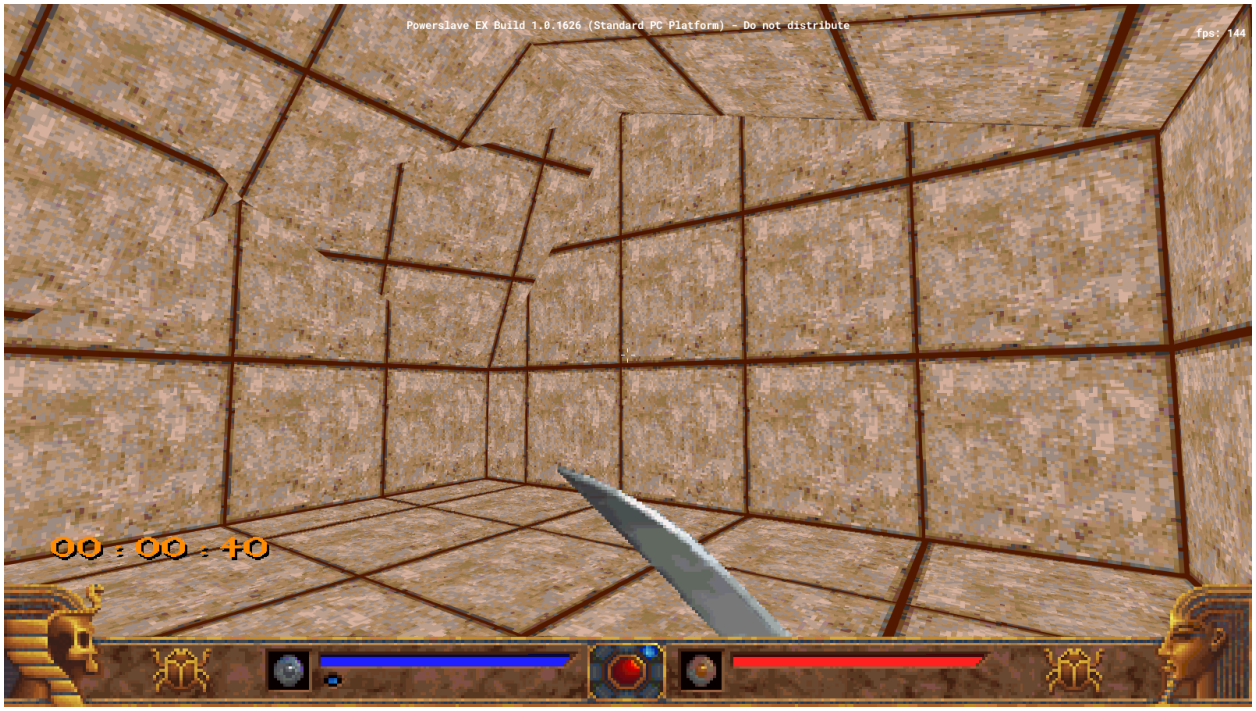
T-Junction Removal

In 3D geometry, a T-junction is a spot where two polygons meet along the edge of another polygon. T-junctions will most probably appear as *discontinuities* in the surface and will show up as gaps in the rendering. This issue will become a lot more apparent with angled surfaces and

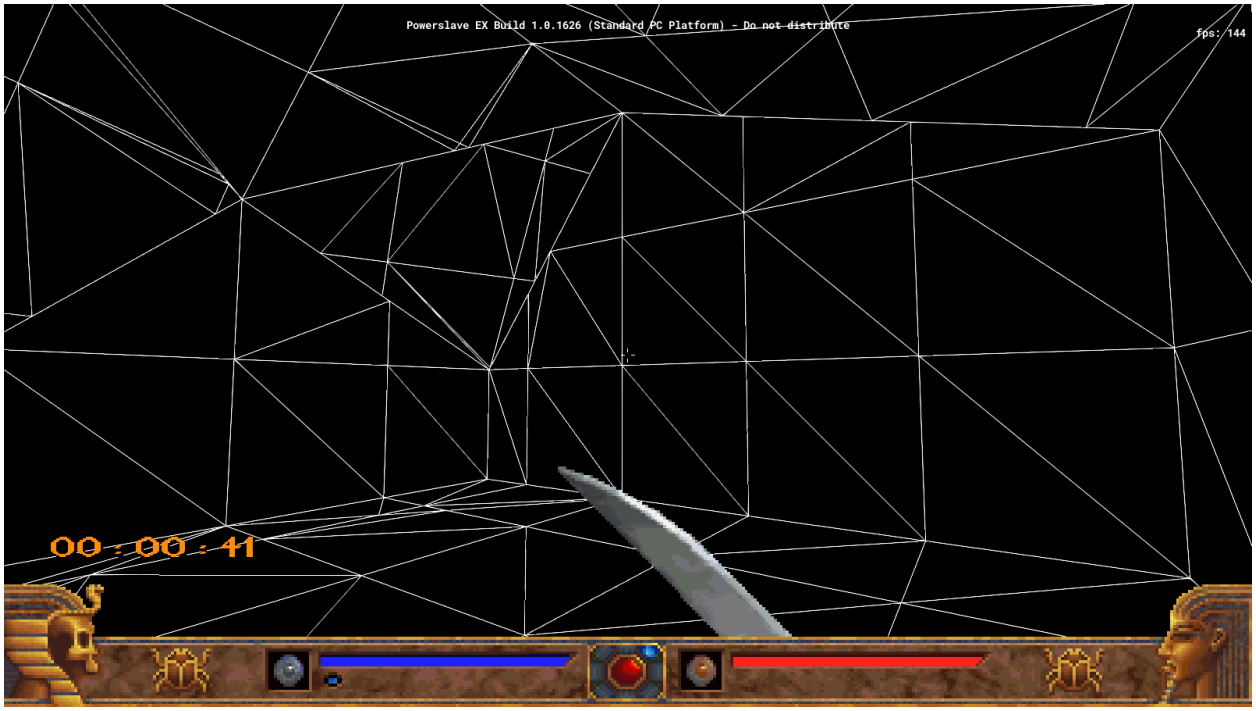
while it might not be a huge deal, it does reveal a lot of holes and gaps when integer vertex emulation is enabled in the game. This was also an issue that existed on the PSX version that also attempted to eliminate t-junctions by triangulating the geometry.

Screenshots below will help further explain this artifact

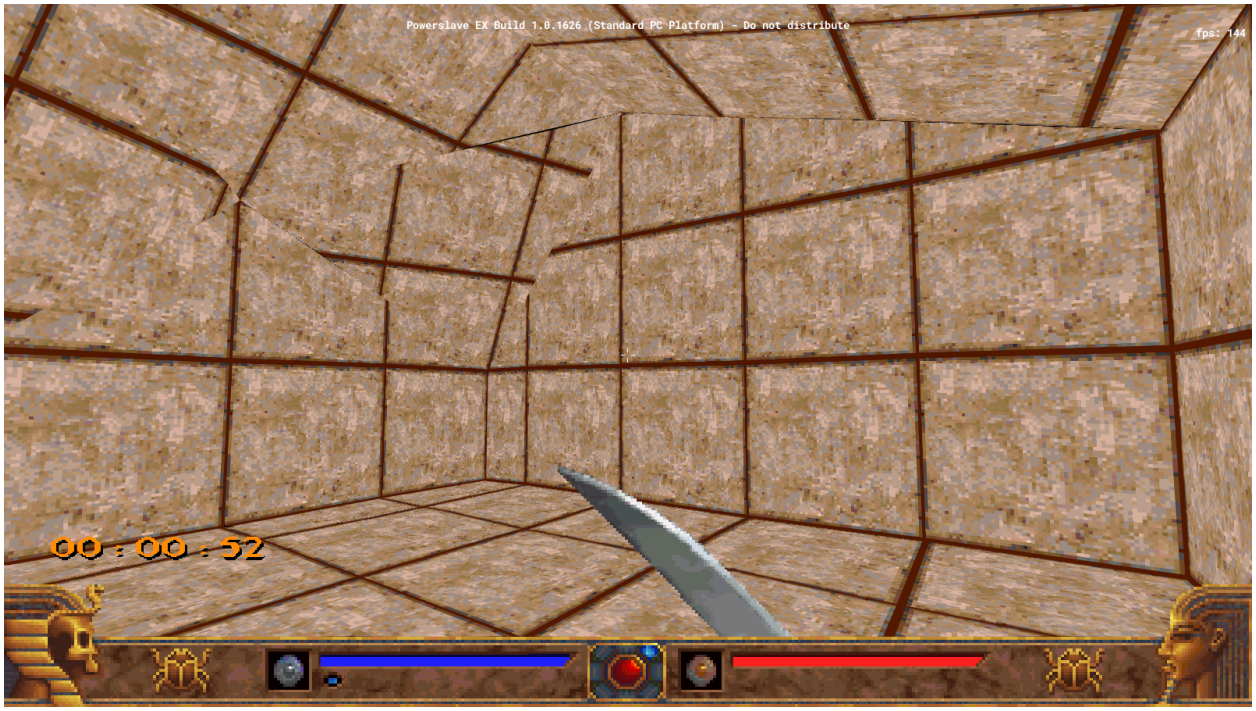
Scene with lots of angled surfaces



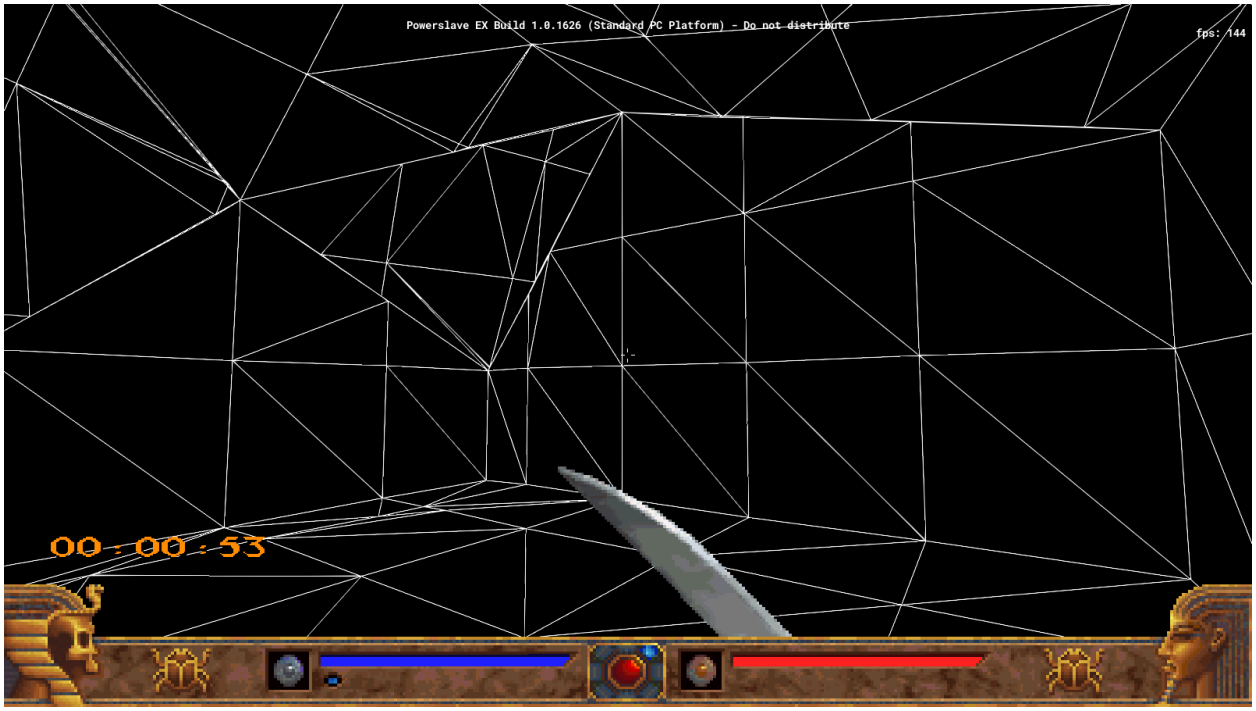
Same scene in wireframe mode



Same scene with integer vertex emulation enabled. Note the gaps seen in the ceiling portions of the room.

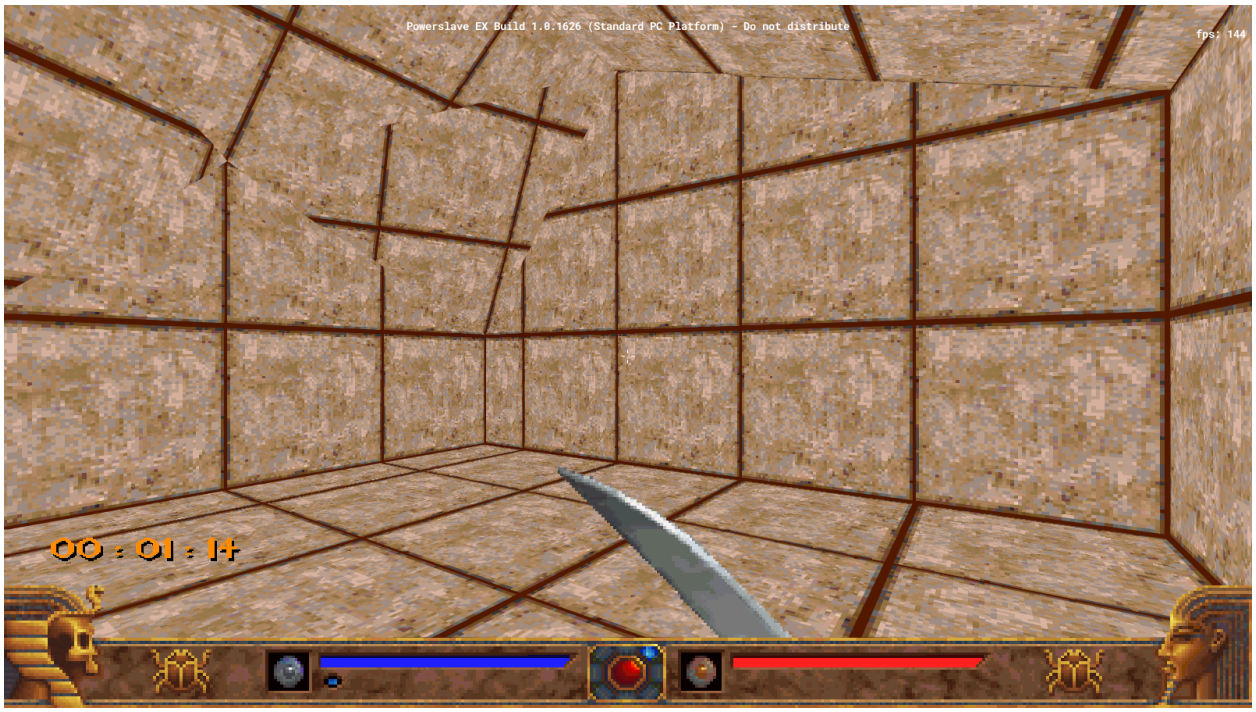


Same scene with integer vertex emulation enabled in wireframe mode

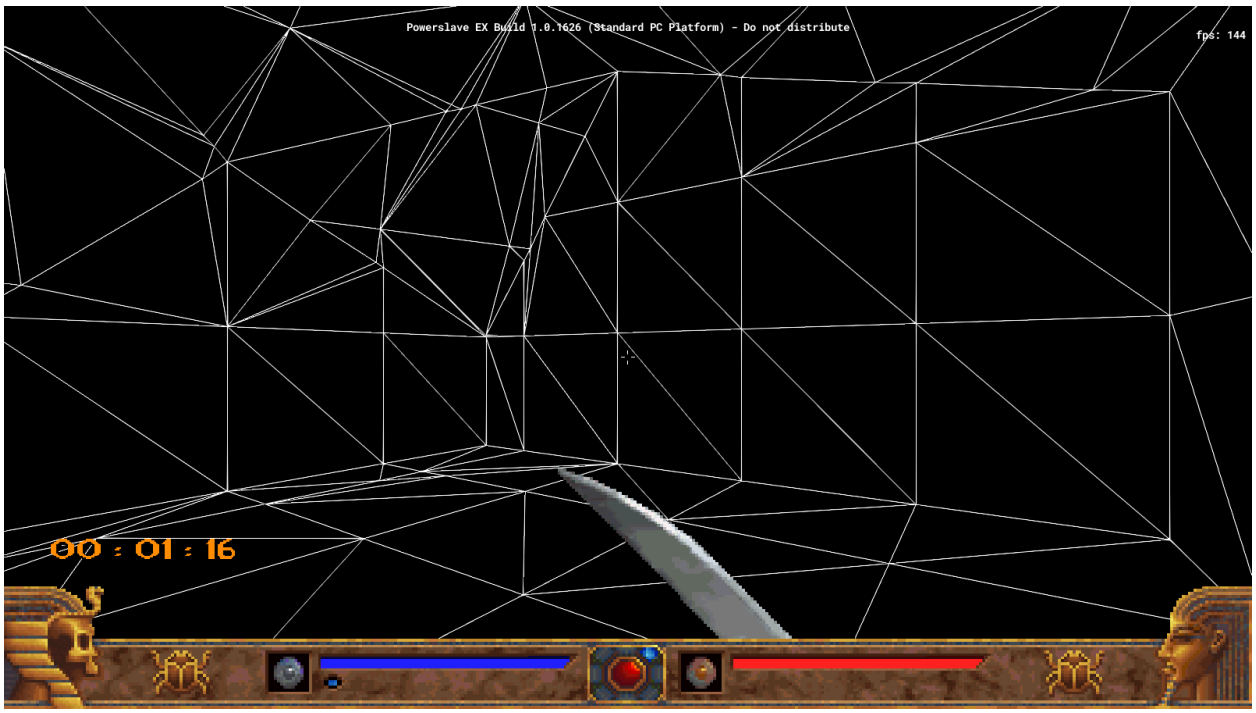


To fix this, after saving your map and before building lights, in the console, type the command: **removetjuncts <my_level.map>**. This will load your .MAP file and will attempt to remove these t-junctions in the map. You will need to re-run this tool every time you save the map in the editor since it will discard the updated geometry. Ideally it's best to run this tool when your map is nearly complete and/or ready for release.

Below are screenshots of the level after t-junctions have been removed



Same scene but in wireframe mode. Note the extra triangulated polygons caused by the t-junctions. This will mitigate gaps in the geometry when integer vertex emulation is enabled.



Other Examples

Example test levels can be found in **C:\Program Files (x86)\Steam\steamapps\common\PowerSlave Exhumed\docs\examples**